

Formato UMAM Digital

Registro de Atención Prehospitalaria

Documentación técnica y código fuente completo

Contenido

- Descripción, arquitectura y diagrama entidad-relación
- Diccionario de datos, API REST y flujos de funcionamiento
- Frontend: Angular 20 + Angular Material (carpeta src/)
- Backend: NestJS 11 + TypeORM + PostgreSQL 17 (carpeta backend/)
- Código fuente: 63 archivos. 5.604 líneas
- Fecha de generación: 08/10/2026

Desarrollado por

Ingeniero Mario Isai Moreno Duran

Número 6564121593

Índice

1. Descripción general del sistema	4
2. Arquitectura	5
3. Modelo de datos (entidad-relación)	6
3.1 Diseño: una tabla con columnas JSONB	6
3.2 Diagrama entidad-relación	7
3.3 Diccionario de datos: tabla registros	7
3.4 Tipos de campo y cómo se guardan	8
3.5 Ejemplo de un registro y consultas SQL	8
3.6 Contenido de cada sección	9
4. API REST del backend	15
5. Cómo funciona: flujos principales	16
6. Estructura del proyecto: qué hace cada archivo	17
7. Instalación, configuración y ejecución	19
8. Código fuente	20
.editorconfig	17 lín. 20
.gitattributes	2 lín. 21
.gitignore	43 lín. 22
README.md	41 lín. 23
angular.json	106 lín. 24
package.json	53 lín. 26
tsconfig.app.json	15 lín. 27
tsconfig.json	34 lín. 28
tsconfig.spec.json	14 lín. 29
src/app/app.config.ts	25 lín. 30
src/app/app.html	11 lín. 31
src/app/app.scss	27 lín. 32
src/app/app.spec.ts	78 lín. 33
src/app/app.ts	30 lín. 35
src/app/core/data/secciones.data.spec.ts	145 lín. 36
src/app/core/data/secciones.data.ts	300 lín. 38
src/app/core/forms/construir-formulario.spec.ts	70 lín. 43
src/app/core/forms/construir-formulario.ts	45 lín. 44
src/app/core/models/campo-formulario.model.ts	33 lín. 45
src/app/core/models/registro.model.spec.ts	21 lín. 46
src/app/core/models/registro.model.ts	57 lín. 47
src/app/core/services/registro-api.service.spec.ts	50 lín. 48
src/app/core/services/registro-api.service.ts	63 lín. 49
src/app/core/services/registro-local-storage.service.spec.ts	60 lín. 50
src/app/core/services/registro-local-storage.service.ts	47 lín. 51
src/app/core/services/registro.service.ts	11 lín. 52
src/app/features/exportacion/excel-exportador.service.spec.ts	73 lín. 53
src/app/features/exportacion/excel-exportador.service.ts	71 lín. 55
src/app/features/exportacion/pdf-vista/pdf-vista.component.spec.ts	208 lín. 56
src/app/features/exportacion/pdf-vista/pdf-vista.component.ts	1176 lín. 59
src/app/features/formulario/formulario.component.spec.ts	275 lín. 77
src/app/features/formulario/formulario.component.ts	270 lín. 81

Índice (cont.)

src/app/features/historial/historial.component.spec.ts	160 lín.	85
src/app/features/historial/historial.component.ts	149 lín.	88
src/app/shared/campo-formulario/campo-formulario.component.spec.ts	374 lín.	91
src/app/shared/campo-formulario/campo-formulario.component.ts	506 lín.	97
src/app/shared/seccion-paso/seccion-paso.component.spec.ts	40 lín.	104
src/app/shared/seccion-paso/seccion-paso.component.ts	56 lín.	105
src/app/shared/tabla-repetible/tabla-repetible.component.spec.ts	52 lín.	106
src/app/shared/tabla-repetible/tabla-repetible.component.ts	58 lín.	107
src/app/shared/theme/umam-theme.scss	23 lín.	108
src/index.html	15 lín.	109
src/main.ts	6 lín.	110
src/styles.scss	36 lín.	111
backend/.env.example	19 lín.	112
backend/.gitignore	4 lín.	113
backend/.prettierrc	4 lín.	114
backend/README.md	37 lín.	115
backend/docker-compose.yml	22 lín.	116
backend/eslint.config.mjs	34 lín.	117
backend/nest-cli.json	8 lín.	118
backend/package.json	77 lín.	119
backend/tsconfig.build.json	4 lín.	121
backend/tsconfig.json	25 lín.	122
backend/src/app.module.ts	44 lín.	123
backend/src/main.ts	37 lín.	124
backend/src/registros/dto/guardar-registro.dto.ts	56 lín.	125
backend/src/registros/registro.entity.ts	72 lín.	126
backend/src/registros/registros.controller.ts	57 lín.	127
backend/src/registros/registros.module.ts	21 lín.	128
backend/src/registros/registros.service.ts	51 lín.	129
backend/test/app.e2e-spec.ts	77 lín.	130
backend/test/jest-e2e.json	9 lín.	132

1. Descripción general del sistema

Formato UMAM Digital es una aplicación web que digitaliza el **Registro de Atención Prehospitalaria** de UMAM: el formato que llena el personal de ambulancia al atender a un paciente. Está pensada para capturarse rápido desde celular o tableta, en el lugar del servicio.

¿Qué permite hacer?

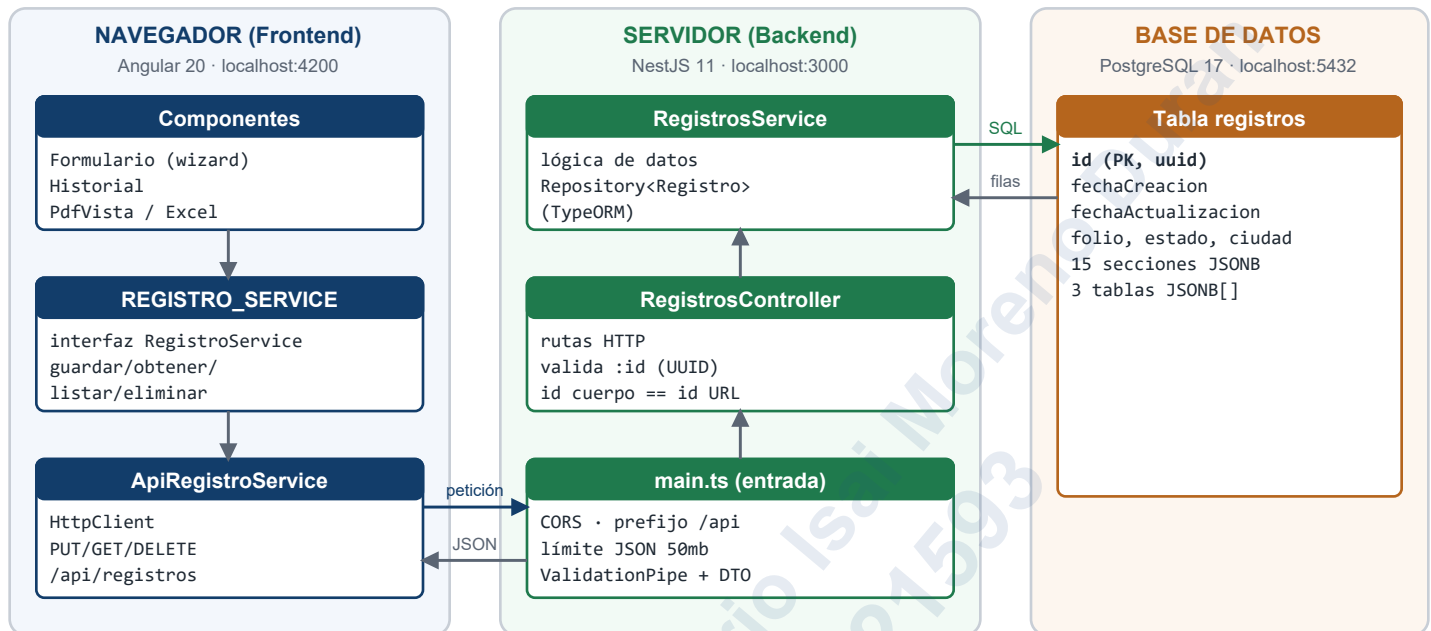
- Capturar el formato por pasos (wizard), una sección a la vez: datos del servicio, del paciente, causa traumática o clínica, parto, evaluaciones, signos vitales, tratamiento, traslado, datos legales, hospital receptor y consentimiento.
- Firmar en pantalla con el dedo o con lápiz óptico (paciente, testigo, quien entrega y quien recibe, paramédico).
- Adjuntar fotos de EKG, radiografías o laboratorios tomadas con la cámara del dispositivo.
- **Autoguardado**: un segundo después de cada cambio el registro se envía al servidor, sin botón de "guardar".
- Consultar el **Historial** de trámites, reabrirlos para seguir editando o eliminarlos.
- Exportar a **PDF** idéntico a la hoja física oficial (2 páginas carta) y a **Excel** (una hoja por sección).

Tecnologías utilizadas

Capa	Tecnología	Para qué se usa
Frontend	Angular 20 + Angular Material	Interfaz de usuario, formularios reactivos, calendarios y selectores.
Frontend	ExcelJS + file-saver	Generar y descargar el archivo .xlsx.
Backend	NestJS 11 (Node.js 24, TypeScript)	API REST: recibe, valida y responde las peticiones.
Backend	TypeORM 0.3 + driver pg	ORM: traduce entre clases TypeScript y tablas SQL.
Backend	class-validator	Validar el JSON recibido antes de guardarlo.
Base de datos	PostgreSQL 17 (Docker)	Almacenamiento persistente; columnas JSONB para las secciones.
Pruebas	Jasmine/Karma (front) · Jest/Supertest (back)	113 pruebas unitarias + 3 pruebas de API de punta a punta.

2. Arquitectura

El sistema sigue una arquitectura de **tres capas**. El navegador nunca habla con la base de datos: todo pasa por la API, que valida los datos antes de guardarlos.



Capa 1 · Frontend (navegador). Los componentes (Formulario, Historial, exportadores) no saben dónde se guardan los datos: solo piden el servicio abstracto REGISTRO_SERVICE. En `app.config.ts` se decide la implementación: `ApiRegistroService` (servidor, la activa) o `LocalStorageRegistroService` (solo navegador). Esto es **inyección de dependencias**: cambiar el almacenamiento no obliga a tocar ningún componente.

Capa 2 · Backend (servidor NestJS). Cada petición pasa por un "pipeline": CORS (solo acepta al frontend autorizado), límite de tamaño, validación con el DTO, y luego el **controlador** (rutas) llama al **servicio** (lógica), que usa el **repositorio** de TypeORM para hablar SQL con PostgreSQL.

Capa 3 · Base de datos (PostgreSQL). Una tabla registros guarda cada trámite completo. Corre en un contenedor Docker con un volumen, así los datos sobreviven a reinicios.

¿Por qué el id lo genera el frontend?

Al crear un borrador, el navegador genera un UUID con `crypto.randomUUID()`. Así el autoguardado puede enviar siempre **PUT /api/registros/{id}** desde el primer cambio, sin esperar a que el servidor asigne un id, y repetir el mismo PUT no duplica registros (operación *idempotente*).

3. Modelo de datos (entidad-relación)

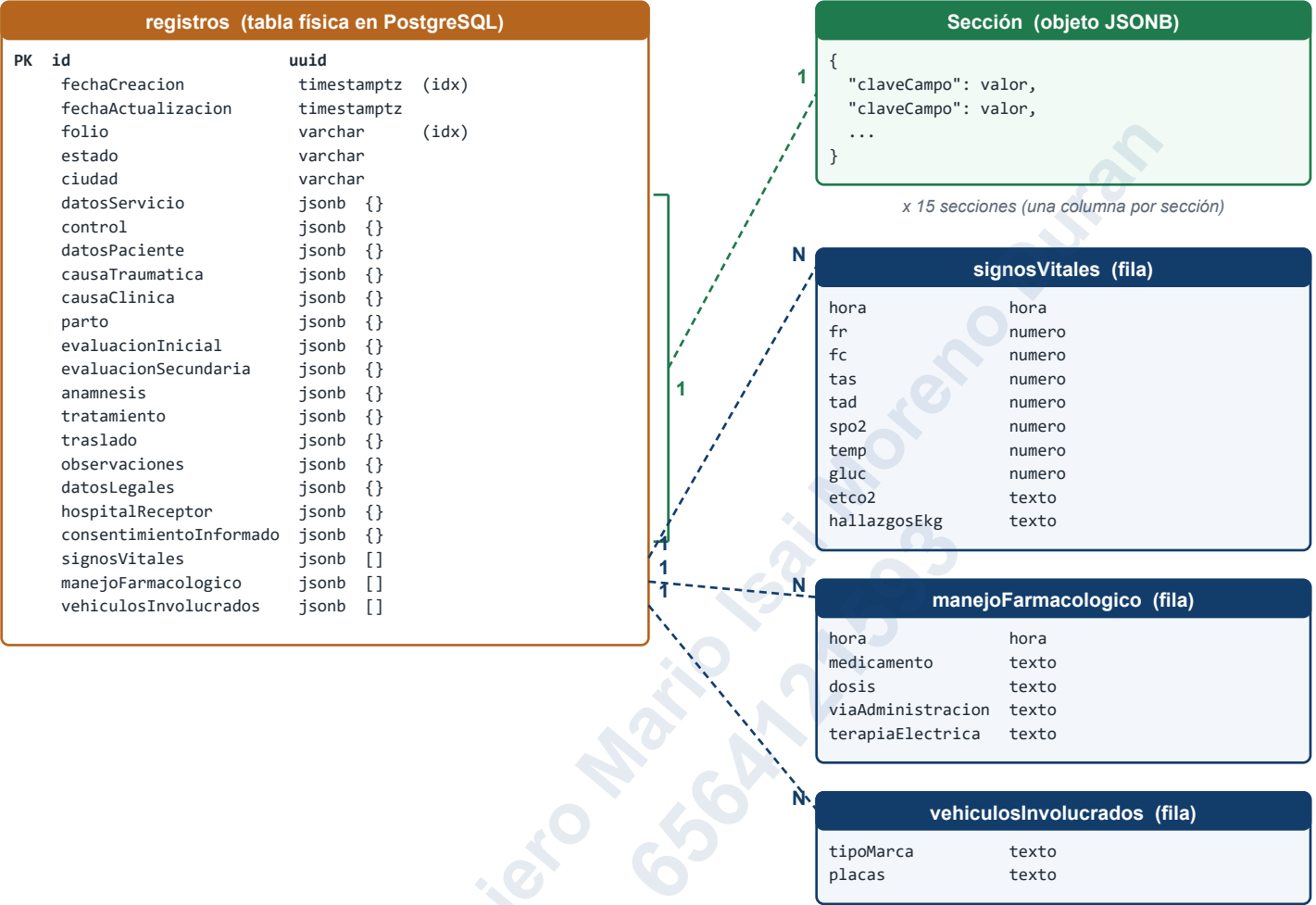
3.1 Diseño: una tabla con columnas JSONB

El formato tiene 159 campos repartidos en 16 secciones (más 3 tablas repetibles), y sus campos se definen en el frontend (secciones.data.ts). En lugar de crear una columna por campo (o una tabla por sección), cada sección se guarda en una columna **JSONB** de PostgreSQL:

- **Agregar o quitar un campo** del formulario no requiere modificar la base de datos ni el backend.
- Un registro se lee y se guarda completo en **una sola operación** (es justo como lo usa el autoguardado).
- JSONB es binario e indexable: se puede consultar dentro del JSON con SQL si se necesita (ver 3.5).
- Los datos más usados para buscar (**folio, estado, ciudad, fechas**) sí son columnas normales, con índice en folio y fecha.

Las **tablas repetibles** (signos vitales, manejo farmacológico, vehículos) son relaciones **1 a N**: un registro tiene muchas filas. Se guardan como un arreglo JSONB dentro del mismo registro, porque esas filas nunca existen ni se consultan fuera de su registro. Las secciones son relaciones **1 a 1** embebidas.

3.2 Diagrama entidad-relación



Naranja: tabla física. Azul y verde: entidades lógicas guardadas dentro de las columnas JSONB (línea punteada = relación embebida, con su cardinalidad).

3.3 Diccionario de datos: tabla registros

Columna	Tipo	Restricciones	Descripción
id	uuid	PK, NOT NULL	Identificador único del trámite. Lo genera el frontend.
fechaCreacion	timestampz	NOT NULL, índice	Momento en que se creó el borrador. Ordena la lista.
fechaActualizacion	timestampz	NOT NULL, automático	Última vez que se guardó (la pone TypeORM).
folio	varchar	NOT NULL, default "", índice	Folio del formato (sección I).
estado	varchar	NOT NULL, default "	Estado de la república (sección I).
ciudad	varchar	NOT NULL, default "	Ciudad (sección I).
datosServicio	jsonb	NOT NULL, default '{}'	Sección II. Datos del Servicio (13 campos).
control	jsonb	NOT NULL, default '{}'	Sección III. Control (3 campos).
datosPaciente	jsonb	NOT NULL, default '{}'	Sección IV. Datos del Paciente (11 campos).

Columna	Tipo	Restricciones	Descripción
causaTraumatica	jsonb	NOT NULL, default '{}'	Sección V. Causa Traumática (14 campos).
causaClinica	jsonb	NOT NULL, default '{}'	Sección VI. Causa Clínica (4 campos).
parto	jsonb	NOT NULL, default '{}'	Sección VII. Parto (20 campos).
evaluacionInicial	jsonb	NOT NULL, default '{}'	Sección VIII. Evaluación Inicial (13 campos).
evaluacionSecundaria	jsonb	NOT NULL, default '{}'	Sección IX. Evaluación Secundaria (4 campos).
anamnesis	jsonb	NOT NULL, default '{}'	Sección X. Anamnesis (16 campos).
tratamiento	jsonb	NOT NULL, default '{}'	Sección XI. Tratamiento (27 campos).
traslado	jsonb	NOT NULL, default '{}'	Sección XII. Traslado (9 campos).
observaciones	jsonb	NOT NULL, default '{}'	Sección XIII. Observaciones (5 campos).
datosLegales	jsonb	NOT NULL, default '{}'	Sección XV. Datos Legales (5 campos).
hospitalReceptor	jsonb	NOT NULL, default '{}'	Sección XVI. Hospital Receptor (6 campos).
consentimientoInformado	jsonb	NOT NULL, default '{}'	Sección XVII. Consentimiento Informado (6 campos).
signosVitales	jsonb	NOT NULL, default '[]'	Tabla repetible Signos vitales y monitoreo de IX. Evaluación Secundaria: arreglo de filas (10 columnas).
manejoFarmacologico	jsonb	NOT NULL, default '[]'	Tabla repetible Manejo farmacológico y terapia eléctrica de XI. Tratamiento: arreglo de filas (5 columnas).
vehiculosInvolucrados	jsonb	NOT NULL, default '[]'	Tabla repetible Vehículos involucrados de XV. Datos Legales: arreglo de filas (2 columnas).

3.4 Tipos de campo y cómo se guardan

Cada campo tiene un **tipo** que define cómo se dibuja en pantalla y qué valor se guarda en el JSON:

Tipo	En pantalla	Valor guardado	Ejemplo
texto	Cuadro de texto de una línea	string	"Juan Pérez"
numero	Cuadro numérico (con sufijo opcional: kg, cm, %)	número o string	72
fecha	Calendario (mat-datepicker)	string YYYY-MM-DD	"2026-10-08"
hora	Selector de hora (mat-timepicker)	string HH:mm	"14:35"
textarea	Texto libre de varias líneas	string	"Penicilina"
checkbox	Casilla Sí/No	boolean	true
checkbox-grupo	Chips de selección múltiple	arreglo de strings	["Hogar", "Trabajo"]
radio-grupo	Una sola opción entre varias	string	"Estable"
select	Lista desplegable	string	"Femenino"
imagenes	Fotos (cámara o archivo) de EKG, Rx, laboratorios	arreglo de data URLs JPEG	["data:image/jpeg;base64,..."]
firma	Lienzo para firmar con dedo, mouse o lápiz óptico	data URL PNG	"data:image/png;base64,..."

3.5 Ejemplo de un registro y consultas SQL

Así se ve (resumido) el JSON que viaja entre frontend y backend y que queda en la tabla:

```
{
  "id": "6f1c2a9e-3b7d-4c1e-9a55-2f0d8e7b1c34",
  "fechaCreacion": "2026-10-08T21:26:39.635Z",
  "folio": "F-001", "estado": "Chihuahua", "ciudad": "Juárez",
  "datosPaciente": { "nombreOMediaFiliacion": "Juan Pérez", "sexo": "Masculino", "edadAnios": 40 },
  "datosServicio": { "fecha": "2026-10-08", "horaSalida": "14:35", "lugarOcurrencia": ["Vía pública"] },
  "signosVitales": [ { "hora": "14:50", "fc": 88, "fr": 18, "spo2": 97 },
    { "hora": "15:05", "fc": 84, "fr": 17, "spo2": 98 } ],
  "hospitalReceptor": { "firmaQuienRecibe": "data:image/png;base64,iVBORw0..." },
  ...
}
```

Consultas útiles directamente en PostgreSQL (operador ->> = leer un campo del JSON como texto):

```
-- Registros de un paciente por nombre
```



```

SELECT folio, "fechaCreacion" FROM registros
WHERE "datosPaciente"->>'nombreOMediaFiliacion' ILIKE '%pérez%';

-- Cuántos servicios hubo por motivo de atención
SELECT "datosServicio"->>'motivoAtencion' AS motivo, count(*) FROM registros GROUP BY 1;

-- Desglosar las filas de signos vitales (relación 1:N) como si fueran una tabla
SELECT r.folio, sv->>'hora' AS hora, sv->>'fc' AS fc
FROM registros r, jsonb_array_elements(r."signosVitales") AS sv;

```

3.6 Contenido de cada sección (diccionario completo)

Generado automáticamente desde secciones.data.ts. La columna **Clave** es el nombre de la propiedad dentro del JSON de la sección. La sección I (Datos Generales) se guarda en las columnas folio, estado y ciudad.

I. Datos Generales → columnas folio, estado, ciudad

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
folio	Folio	texto	
estado	Estado	texto	
ciudad	Ciudad	texto	

II. Datos del Servicio → columna datosServicio

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
fecha	Fecha	fecha	
diaSemana	Día de la semana	texto	
horaSalida	Hora de salida	hora	
horaLlegada	Hora de llegada	hora	
horaTraslado	Hora de traslado	hora	
horaHospital	Hora de hospital	hora	
horaLiberacion	Hora de liberación	hora	
motivoAtencion	Motivo de la atención	radio-grupo	Enfermedad, Traumatismo, Gineco-Obstétrico
colonia	Colonia / Comunidad	texto	
delegacionMunicipio	Delegación política / Municipio	texto	
lugarOcurrencia	Lugar de la ocurrencia	checkbox-grupo	Hogar, Vía pública, Trabajo, Escuela, Deporte y recreación, Transporte público, Otro
lugarOcurrenciaOtro	Otro (especifique)	texto	
responsableEntregaPaciente	Responsable de entrega del paciente	texto	

III. Control → columna control

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
ambulanciaNumero	Número de ambulancia	texto	
operador	Operador	texto	
prestadoresServicio	Prestadores del servicio	texto	

IV. Datos del Paciente → columna datosPaciente

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
nombreOMediaFiliacion	Nombre o media filiación	texto	
sexo	Sexo	select	Masculino, Femenino
edadAnios	Edad (años)	numero	
edadMeses	Menores de 1 año (meses)	numero	
fechaNacimiento	Fecha de nacimiento	fecha	
colonia	Colonia / Comunidad	texto	

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
delegacionMunicipio	Delegación política / Municipio	texto	
telefono	Teléfono	texto	
ocupacion	Ocupación	texto	
numeroEmpleado	Número de empleado	texto	
tipoServicioMedico	Tipo de servicio médico	texto	

V. Causa Traumática → columna causaTraumatica

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
agenteCausal	Agente causal	checkbox-grupo	Automotor, Bicicleta, Maquinaria, Herramienta, Fuego, Sustancia caliente, Sustancia tóxica, Electricidad, Explosión, Producto biológico, Arma, Juguete, Ser humano, Animal, Otro
agenteCausalEspecifique	Especifique	texto	
lesionesCausadasPor	Lesiones causadas por	texto	
accidenteAutomovilistico	Accidente automovilístico	checkbox-grupo	Colisión, Volcadura, Automotor, Motocicleta, Bicicleta, Maquinaria
contraObjeto	Contra objeto	radio-grupo	Fijo, En movimiento
impacto	Impacto	radio-grupo	Frontal, Lateral, Posterior
hundimientoCms	Hundimiento	numero	(unidad: cm)
parabrisas	Parabrisas	radio-grupo	Íntegro, Roto/Doblado
volante	Volante	radio-grupo	Íntegro, Doblado
bolsasAire	Bolsas de aire	radio-grupo	Sí, No
cinturonSeguridad	Cinturón de seguridad	radio-grupo	Colocado, No colocado
dentroVehiculo	Dentro del vehículo	radio-grupo	Sí, No
eyectado	Eyectado	checkbox	
atropellado	Atropellado por	checkbox-grupo	Automotor, Motocicleta, Bicicleta, Maquinaria

VI. Causa Clínica → columna causaClinica

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
origenProbable	Origen probable	checkbox-grupo	Metabólico, Respiratorio, Cardiovascular, Neurológico, Urogenital, Gineco-obstétrico, Digestivo, Psico-emotivo, Oncológico, Infeccioso, Músculo esquelético, Otro
origenProbableEspecifique	Especifique	texto	
primeraVez	1era vez	radio-grupo	Sí, No
subsecuente	Subsecuente	texto	

VII. Parto → columna parto

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
gesta	Gesta	numero	
cesarea	Cesárea	numero	
para	Para	numero	
abortos	Abortos	numero	
semanaGestacion	Semana de gestación	numero	
fechaProbableParto	Fecha probable de parto	fecha	
membranas	Membranas	texto	
horaInicioContracciones	Hora de inicio de contracciones	hora	
frecuenciaContracciones	Frecuencia	texto	
duracionContracciones	Duración	texto	

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
horaNacimiento	Hora de nacimiento	hora	
lugarNacimientoParto	Lugar	texto	
placentaExpulsada	Placenta expulsada	radio-grupo	Sí, No
productoVivoMuerto	Producto	radio-grupo	Vivo, Muerto
sexoRecienNacido	Sexo del recién nacido	radio-grupo	Masculino, Femenino
apgar1min	APGAR 1 min	numero	
apgar5min	APGAR 5 min	numero	
apgar10min	APGAR 10 min	numero	
silverman1min	Silverman 1 min	numero	
silverman5min	Silverman 5 min	numero	

VIII. Evaluación Inicial → columna evaluacionInicial

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
nivelConciencia	Nivel de conciencia	radio-grupo	Consciente, Respuesta a estímulo verbal, Respuesta a estímulo doloroso, Inconsciente
viaAerea	Vía aérea	radio-grupo	Permeable, Comprometida
reflejoDeglucion	Reflejo de deglución	radio-grupo	Presente, Ausente
ventilacionObservacion	Ventilación — observación	checkbox-grupo	Automatismo regular, Automatismo irregular, Ventilación rápida, Ventilación lenta, Ventilación profunda, Ventilación superficial, Apnea
ventilacionAuscultacion	Ventilación — auscultación	checkbox-grupo	Ruidos respiratorios normales, Ruidos respiratorios disminuidos, Ruidos respiratorios anormales, Ruidos respiratorios ausentes
ventilacionAuscultacionEspecifique	Auscultación — especifique	texto	
alternacionesLocalizadas	Alternaciones localizadas en	checkbox-grupo	Hemitórax derecho, Hemitórax izquierdo, Ápices, Bases
presenciaPulsos	Presencia de pulsos	checkbox-grupo	Carotídeo, Radial
calidadPulso	Calidad del pulso	checkbox-grupo	Normal, Rápido, Lento, Rítmico, Arrítmico
paroCardiorespiratorio	Paro cardiorrespiratorio	checkbox	
piel	Piel	checkbox-grupo	Normal, Rojiza, Pálida, Cianótica, Ictericia
caracteristicasPiel	Características de la piel	checkbox-grupo	Húmeda, Seca, Normal
temperaturaComparativa	Temperatura comparativa	checkbox-grupo	Normal, Caliente, Fría

IX. Evaluación Secundaria → columna evaluacionSecundaria

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
exploracionFisica	Exploración física	checkbox-grupo	Deformaciones, Contusiones, Abrasiones, Penetraciones, Movimientos paradójicos, Crepitación, Heridas, Fracturas, Enfisema subcutáneo, Quemaduras, Laceraciones, Edema, Alteraciones de sensibilidad, Alteraciones de movilidad, Dolor, Amputación
zonasLesion	Zonas de lesión	checkbox-grupo	Cabeza, Cuello, Tórax anterior, Tórax posterior, Abdomen, Pelvis, Brazo izquierdo, Brazo derecho, Antebrazo izquierdo, Antebrazo derecho, Mano izquierda, Mano derecha, Muslo izquierdo, Muslo derecho, Pierna izquierda, Pierna derecha, Pie izquierdo, Pie derecho, Espalda
zonasLesionEspecifique	Zonas de lesión — especifique	texto	
pupilas	Pupilas	checkbox-grupo	Normales, Midriáticas, Mióticas, Anisocóricas, No reactivas

Tabla repetible: Signos vitales y monitoreo → columna signosVitales (arreglo, 1 objeto por fila)

Clave	Columna	Tipo	
hora	Hora	hora	
fr	FR	numero	
fc	FC	numero	
tas	TAS	numero	
tad	TAD	numero	
spo2	SpO2	numero	
temp	TEMP	numero	
gluc	GLUC	numero	
etco2	EtCO2	texto	
hallazgosEkg	Hallazgos EKG	texto	

X. Anamnesis → columna anamnesis

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
interrogatorio	Interrogatorio	radio-grupo	Directo, Indirecto
peso	Peso	numero	(unidad: kg)
talla	Talla	numero	(unidad: cm)
alergias	Alergias	textarea	
medicamentos	Medicamentos	textarea	
enfermedadesCirugiasPrevias	Enfermedades y/o cirugías previas	textarea	
horaUltimaComida	Hora de última comida	hora	
eventosPreviosRelacionados	Eventos previos relacionados	textarea	
condicionPaciente	Condición del paciente	radio-grupo	Crítico, No crítico
estabilidadPaciente	Estabilidad	radio-grupo	Inestable, Estable
prioridad	Prioridad	radio-grupo	Rojo, Amarillo, Verde, Negro
glasgowOcular	Glasgow — ocular	numero	
glasgowVerbal	Glasgow — verbal	numero	
glasgowMotora	Glasgow — motora	numero	
glasgowTotal	Glasgow — total	numero	
news2Total	NEWS2 — puntaje total	numero	

XI. Tratamiento → columna tratamiento

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
viaAereaTratamiento	Vía aérea	checkbox-grupo	Aspiración, Cánula orofaríngea, Cánula nasofaríngea, Intubación endotraqueal, Dispositivo supraglótico, Vía aérea quirúrgica
controlCervical	Control cervical	radio-grupo	Manual, Collarín rígido, Collarín blando
asistenciaVentilatoria	Asistencia ventilatoria	checkbox-grupo	Balón-válvula mascarilla, Ventilador automático
ventilacionMecanica	Ventilación mecánica	checkbox-grupo	AC/V, AC/P, CPAP/PS
frecuenciaVentilatoria	Frecuencia	numero	
volumenVentilatorio	Volumen	numero	
peep	PEEP	numero	
presionVentilatoria	PIP	numero	
relacionIE	I:E	texto	
fio2	FiO2	numero	(unidad: %)
ps	PS	numero	
modoVentilatorio	Modo ventilatorio	texto	
oxigenoterapia	Oxigenoterapia	checkbox-grupo	Puntas nasales, Mascarilla simple, Mascarilla con reservorio, Mascarilla para nebulizar

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
litrosPorMinuto	Lts x min	numero	
descompresionPleural	Descompresión pleural	checkbox-grupo	Hemitórax derecho, Hemitórax izquierdo
controlHemorragias	Control de hemorragias	checkbox-grupo	Presión directa, Vendaje compresivo, Torniquete, Empaquetamiento de herida
lineasIv	Líneas IV #	numero	
numeroCateter	Catéter #	texto	
sitioAplicacion	Sitio de aplicación	checkbox-grupo	Mano, Pliegue antecubital, Intraósea, Otra
sitioAplicacionEspecifique	Sitio de aplicación — especifique	texto	
tipoSoluciones	Tipo de soluciones	texto	
cantidadSolucion	Cantidad	texto	
infusion	Infusión	texto	
rcp	RCP	checkbox-grupo	RCP básica, RCP avanzada
inmovilizacion	Inmovilización	checkbox-grupo	Inmovilización de extremidad(es), Inmovilización de columna
curacion	Curación	checkbox	
vendaje	Vendaje	checkbox	

Tabla repetible: Manejo farmacológico y terapia eléctrica → columna manejoFarmacologico (arreglo, 1 objeto por fila)

Clave	Columna	Tipo	
hora	Hora	hora	
medicamento	Medicamento	texto	
dosis	Dosis	texto	
viaAdministracion	Vía de administración	texto	
terapiaElectrica	Terapia eléctrica	texto	

XII. Traslado → columna traslado

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
institucionTraslado	Institución a la que se traslada el paciente	texto	
condicionPacienteTraslado	Condición del paciente	radio-grupo	Crítico, No crítico
estabilidadPacienteTraslado	Estabilidad	radio-grupo	Inestable, Estable
prioridadTraslado	Prioridad de traslado	radio-grupo	Rojo, Amarillo, Verde, Negro
negativaAtencion	Negativa a recibir atención / ser trasladado (eximente de responsabilidad)	checkbox	
nombrePaciente	Nombre de quien firma por el paciente	texto	
firmaPaciente	Firma de quien firma por el paciente	firma	
nombreTestigo	Nombre del testigo	texto	
firmaTestigo	Firma del testigo	firma	

XIII. Observaciones → columna observaciones

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
observaciones	Observaciones	textarea	
observacionesAdicionales	Más observaciones	textarea	
ministerioPublicoNotificado	Ministerio Público notificado	checkbox	
selloMinisterioPublico	Sello / folio de Ministerio Público	texto	
nombreQuienRecibeMP	Nombre y firma de quien recibe	texto	

XV. Datos Legales → columna datosLegales

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
dependenciaAutoridad	Dependencia	texto	
numeroUnidades	Número de unidades	texto	
nombreNumeroOficiales	Nombre o número de los oficiales	texto	
pertenencias	Pertenencias	textarea	
recibioPertenenciasNombreFirmaCargo	Recibió las pertenencias (nombre, firma y cargo)	texto	

Tabla repetible: Vehículos involucrados → columna vehiculosInvolucrados (arreglo, 1 objeto por fila)

Clave	Columna	Tipo	
tipoMarca	Tipo y marca	texto	
placas	Placas	texto	

XVI. Hospital Receptor → columna hospitalReceptor

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
aceptacionHospitalReceptor	Aceptación de hospital receptor	textarea	
nombreQuienEntrega	Nombre de quien entrega al paciente	texto	
firmaQuienEntrega	Firma de quien entrega al paciente	firma	
nombreQuienRecibe	Nombre de persona que recibe al paciente	texto	
firmaQuienRecibe	Firma de persona que recibe al paciente	firma	
imagenesEkgRxLaboratorios	Imágenes de EKG, Rx y/o laboratorios	imagenes	

XVII. Consentimiento Informado → columna consentimientoInformado

Clave	Etiqueta en pantalla	Tipo	Opciones / unidad
nombrePacienteConsentimiento	Nombre y firma del paciente	texto	
firmaPacienteConsentimiento	Firma del paciente	firma	
nombreResponsableConsentimiento	Nombre y firma del familiar o responsable	texto	
firmaResponsableConsentimiento	Firma del familiar o responsable	firma	
nombreParamedicoConsentimiento	Nombre y firma del paramédico	texto	
firmaParamedicoConsentimiento	Firma del paramédico	firma	

4. API REST del backend

URL base: `http://localhost:3000/api`. Todas las respuestas y peticiones son JSON.

Método	Ruta	Qué hace	Respuestas
GET	<code>/registros</code>	Lista todos los registros, del más antiguo al más reciente. La usan el Historial y el formulario al abrir (toma el último como borrador en curso).	200 + arreglo
GET	<code>/registros/{id}</code>	Obtiene un registro.	200 + registro · 404 no existe · 400 id no es UUID
PUT	<code>/registros/{id}</code>	Crea el registro si no existe o lo actualiza si existe ("upsert"). Lo llama el autoguardado.	200 + registro guardado · 400 datos inválidos o id distinto
DELETE	<code>/registros/{id}</code>	Elimina el registro.	204 sin contenido

Validaciones que aplica el servidor

- `id` debe ser UUID y coincidir con el de la URL (evita sobrescribir un registro con los datos de otro).
- `fechaCreacion` obligatoria en formato ISO 8601.
- `folio`, `estado`, `ciudad`: texto. Cada sección: objeto. Cada tabla repetible: arreglo de objetos.
- Las propiedades desconocidas se descartan (*whitelist*), no llegan a la base de datos.
- Tamaño máximo del JSON: 50 MB (las firmas y fotos viajan en base64). Configurable con `BODY_LIMIT`.
- CORS: solo se aceptan peticiones del origen configurado en `CORS_ORIGIN` (por defecto `http://localhost:4200`).

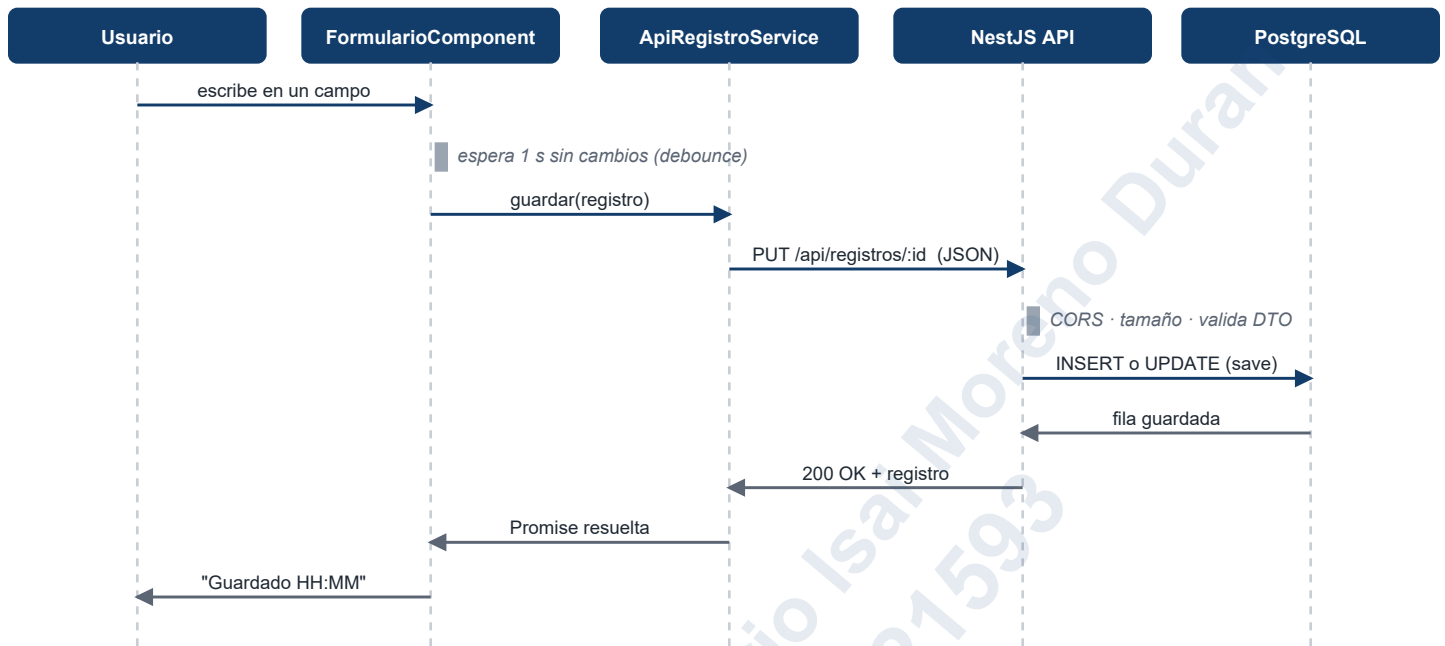
Ejemplo con curl

```
curl -X PUT http://localhost:3000/api/registros/6f1c2a9e-3b7d-4c1e-9a55-2f0d8e7b1c34 \
-H "Content-Type: application/json" \
-d '{"id":"6f1c2a9e-3b7d-4c1e-9a55-2f0d8e7b1c34",
    "fechaCreacion":"2026-10-08T21:26:39.635Z","folio":"F-001"}'
```

```
curl http://localhost:3000/api/registros
curl -X DELETE http://localhost:3000/api/registros/6f1c2a9e-3b7d-4c1e-9a55-2f0d8e7b1c34
```

5. Cómo funciona: flujos principales

5.1 Autoguardado (diagrama de secuencia)



El formulario escucha cada cambio (`valueChanges`) pero espera 1 segundo sin cambios (`debounceTime(1000)`) antes de guardar, para no mandar una petición por cada tecla. Si el servidor no responde, la pantalla muestra **"Error al guardar"** y el siguiente cambio vuelve a intentarlo.

5.2 Abrir la aplicación

- Se crea un borrador vacío con un UUID nuevo y se construye el formulario a partir de `secciones.data.ts`.
- Se piden todos los registros (GET). Si hay alguno y el usuario aún no escribió nada, se carga el más reciente para continuarlo.
- Se activa el autoguardado. "Nuevo registro" reinicia el formulario con otro UUID.

5.3 Historial

- Muestra todos los trámites, el más reciente primero, con folio, paciente y fecha.
- Abrir:** carga ese registro en el formulario para seguir editándolo (el autoguardado sigue usando su mismo id).
- PDF / Excel:** exporta el trámite tal como quedó guardado.
- Eliminar:** pide confirmación y envía DELETE al servidor.

5.4 Exportaciones

PDF: el componente `PdfVista` es una réplica en HTML/CSS de la hoja física oficial (2 páginas carta). Al exportar se toma una "foto" del formulario y se llama a `window.print()`; el usuario elige "Guardar como PDF". Un registro vacío imprime la hoja en blanco, igual que el papel.

Excel: `ExcelExportadorService` crea un libro con una hoja por sección (columnas Campo / Valor) y las tablas repetibles. Las firmas e imágenes no se vuelcan en celdas: se indica "Firmado — ver PDF" o el número de imágenes adjuntas.

6. Estructura del proyecto: qué hace cada archivo

```
uman/  
├── src/  
│   ├── app/  
│   │   ├── core/  
│   │   ├── features/  
│   │   └── shared/  
│   └── backend/  
│       ├── src/registros/  
│       ├── test/  
│       └── docker-compose.yml
```

Frontend Angular

Modelos, definición de secciones, formularios y servicios
Pantallas: formulario, historial, exportación PDF/Excel
Componentes reutilizables: campo, sección, tabla repetible
Backend NestJS
Entidad, DTO, servicio, controlador y módulo de registros
Pruebas de punta a punta de la API
Base de datos PostgreSQL

Frontend — raíz

Archivo	Responsabilidad
package.json	Dependencias del frontend (Angular, Material, ExcelJS) y scripts npm (start, build, test).
angular.json	Configuración del proyecto Angular: build, servidor de desarrollo, pruebas y presupuestos de tamaño.
tsconfig*.json	Opciones del compilador TypeScript para la app y las pruebas.
README.md	Instrucciones de uso, conexión con el backend, pruebas y build.

Frontend — src/app

Archivo	Responsabilidad
main.ts / index.html	Arranque de la aplicación: monta el componente raíz App en la página.
app.config.ts	Proveedores globales: HttpClient, qué implementación de RegistroService se usa (API) y el idioma es-MX de calendarios.
app.ts / app.html / app.scss	Componente raíz: barra superior y cambio entre las vistas Formulario e Historial.
core/models/campo-formulario.model.ts	Tipos que describen un campo (clave, etiqueta, tipo, opciones), una tabla repetible y una sección.
core/models/registro.model.ts	Interfaz RegistroAtencionPrehospitalaria (forma del registro completo) y crearRegistroVacio() que genera un borrador con UUID nuevo.
core/data/secciones.data.ts	Definición declarativa de las 16 secciones del formato y de sus campos. Es la "fuente de verdad": el formulario, el PDF y el Excel se construyen a partir de aquí.
core/forms/construir-formulario.ts	Convierte las secciones + un registro guardado en un FormGroup reactivo de Angular (con valores iniciales por tipo de campo).
core/services/registro.service.ts	Interfaz RegistroService y token REGISTRO_SERVICE: el contrato de persistencia que usan los componentes.
core/services/registro-api.service.ts	Implementación que guarda en el backend NestJS vía HTTP (la que está activa).
core/services/registro-local-storage.service.ts	Implementación alternativa que guarda solo en el navegador (localStorage), útil sin servidor.
features/formulario/formulario.component.ts	Wizard de captura paso a paso: carga el último borrador, autoguarda 1 s después de cada cambio, navega entre pasos y dispara exportaciones.
features/historial/historial.component.ts	Lista de trámites guardados (más reciente primero) con acciones abrir, exportar PDF/Excel y eliminar.
features/exportacion/pdf-vista/pdf-vista.component.ts	Réplica imprimible de la hoja física oficial UMAM (2 páginas carta); se imprime con window.print() → "Guardar como PDF".
features/exportacion/excel-exportador.service.ts	Genera un libro de Excel con una hoja por sección (Campo / Valor) y tablas repetibles.
shared/campo-formulario/campo-formulario.component.ts	Dibuja un campo según su tipo: texto, número, fecha/hora (Material), chips, radios, select, cámara/imágenes y firma con dedo o lápiz óptico.
shared/seccion-paso/seccion-paso.component.ts	Dibuja una sección completa (sus campos en cuadrícula y sus tablas repetibles).
shared/tabla-repetible/tabla-repetible.component.ts	Tabla con filas que se agregan/quitan (signos vitales, fármacos, vehículos).
shared/theme/umam-theme.scss / styles.scss	Tema visual (colores institucionales) y estilos globales.
*.spec.ts	Pruebas unitarias (Jasmine/Karma) de cada pieza: 113 pruebas.

Backend — backend/

Archivo	Responsabilidad
package.json	Dependencias del backend (NestJS, TypeORM, pg, class-validator) y scripts (start:dev, build, test:e2e).
docker-compose.yml	Levanta PostgreSQL 17 en un contenedor con volumen persistente.
.env.example	Plantilla de variables de entorno (puerto, CORS, conexión a la base de datos).
nest-cli.json / tsconfig*.json	Configuración del compilador de NestJS y TypeScript.
src/main.ts	Arranque del servidor: prefijo /api, CORS, límite de 50 MB para el JSON y validación global.
src/app.module.ts	Módulo raíz: lee el .env y abre la conexión a PostgreSQL con TypeORM.
src/registros/registros.module.ts	Agrupar entidad, servicio y controlador de registros.
src/registros/registro.entity.ts	Entidad Registro = tabla registros (columnas y tipos).
src/registros/dto/guardar-registro.dto.ts	Reglas de validación del JSON que llega en el PUT.
src/registros/registros.controller.ts	Rutas HTTP GET/PUT/DELETE de /api/registros.
src/registros/registros.service.ts	Operaciones contra la base de datos (listar, obtener, guardar, eliminar).
test/app.e2e-spec.ts	Pruebas de punta a punta de la API contra un PostgreSQL real.

7. Instalación, configuración y ejecución

Requisitos: Node.js 20 o superior, npm y Docker Desktop.

1. Base de datos y backend

```
cd backend
npm install
copy .env.example .env
docker compose up -d      # PostgreSQL en localhost:5432
npm run start:dev         # API en http://localhost:3000/api
```

2. Frontend

```
cd ..                      # raíz del proyecto
npm install
npx ng serve               # abrir http://localhost:4200
```

3. Pruebas

```
npx ng test --watch=false --browsers=ChromeHeadless  # frontend (113 pruebas)
cd backend && npm run test:e2e                        # API contra PostgreSQL (3 pruebas)
```

Variables de entorno (backend/.env)

Variable	Valor por defecto	Descripción
PORT	3000	Puerto de la API.
CORS_ORIGIN	http://localhost:4200	Origen del frontend autorizado.
BODY_LIMIT	50mb	Tamaño máximo del JSON de un registro.
DB_HOST / DB_PORT	localhost / 5432	Servidor de PostgreSQL.
DB_USER / DB_PASSWORD / DB_NAME	umam / umam / umam	Credenciales y base de datos (coinciden con docker-compose.yml).
DB_SYNCHRONIZE	true	Crea/actualiza la tabla automáticamente. En producción: false y usar migraciones.

Recomendaciones para producción

- Agregar autenticación (p. ej. JWT) antes de exponer la API: contiene datos médicos personales.
- Usar HTTPS, contraseñas seguras en la base de datos y DB_SYNCHRONIZE=false con migraciones.
- Configurar API_REGISTROS_URL (frontend) y CORS_ORIGIN (backend) con los dominios reales.
- Respallos periódicos del volumen de PostgreSQL (pg_dump).

```
1 # Editor configuration, see https://editorconfig.org
2 root = true
3
4 [*]
5 charset = utf-8
6 indent_style = space
7 indent_size = 2
8 insert_final_newline = true
9 trim_trailing_whitespace = true
10
11 [*.ts]
12 quote_type = single
13 ij_typescript_use_double_quotes = false
14
15 [*.md]
16 max_line_length = off
17 trim_trailing_whitespace = false
```

```
1 # Auto detect text files and perform LF normalization
2 * text=auto
```

Desarrollado Ingeniero Mario Isai Moreno Duran
Número 6564121593

```
1 # See https://docs.github.com/get-started/getting-started-with-git/ignoring-files for more about ignoring files.
2
3 # Compiled output
4 /dist
5 /tmp
6 /out-tsc
7 /bazel-out
8
9 # Node
10 /node_modules
11 npm-debug.log
12 yarn-error.log
13
14 # IDEs and editors
15 .idea/
16 .project
17 .classpath
18 .c9/
19 *.launch
20 .settings/
21 *.sublime-workspace
22
23 # Visual Studio Code
24 .vscode/*
25 !.vscode/settings.json
26 !.vscode/tasks.json
27 !.vscode/launch.json
28 !.vscode/extensions.json
29 .history/*
30
31 # Miscellaneous
32 /.angular/cache
33 .sass-cache/
34 /connect.lock
35 /coverage
36 /libpeerconnection.log
37 testem.log
38 /typings
39 __screenshots__/
40
41 # System files
42 .DS_Store
43 Thumbs.db
```

```
1 # Formato UMAM Digital
2
3 Formulario web (Angular 20) que reproduce el "Registro de Atención
4 Prehospitalaria" de UMAM, con exportación a PDF y Excel respetando el
5 diseño del documento. Ver `docs/superpowers/specs/2026-08-06-formato-umam-design.md`
6 para el diseño completo y `docs/superpowers/plans/2026-08-06-formato-umam-angular.md`
7 para el plan de implementación.
8
9 ## Uso
10
11 ```powershell
12 npm install
13 npx ng serve
14 ```
15
16 Abre `http://localhost:4200`. Los datos se guardan automáticamente en el
17 backend (PostgreSQL) mientras se llena el formulario, así que el backend
18 debe estar corriendo – ver `backend/README.md`.
19
20 ## Backend
21
22 Los componentes solo dependen de la interfaz `RegistroService`
23 (`src/app/core/services/registro.service.ts`), inyectada mediante el
24 token `REGISTRO_SERVICE`. En `src/app/app.config.ts` se usa
25 `ApiRegistroService` (habla con la API NestJS en `backend/`; la URL está en
26 `API_REGISTROS_URL`). Para volver a guardar solo en el navegador, cambia a
27 `useClass: LocalStorageRegistroService`.
28
29 ## Pruebas
30
31 ```powershell
32 npx ng test --watch=false --browsers=ChromeHeadless
33 ```
34
35 ## Build de producción
36
37 ```powershell
38 npx ng build
39 ```
40
41 El resultado se genera en `dist/app-tmp/`.
```

```
1 {
2   "$schema": "./node_modules/@angular/cli/lib/config/schema.json",
3   "version": 1,
4   "cli": {
5     "packageManager": "npm",
6     "analytics": false
7   },
8   "newProjectRoot": "projects",
9   "projects": {
10    "app-tmp": {
11      "projectType": "application",
12      "schematics": {
13        "@schematics/angular:component": {
14          "style": "scss"
15        }
16      },
17      "root": "",
18      "sourceRoot": "src",
19      "prefix": "app",
20      "architect": {
21        "build": {
22          "builder": "@angular/build:application",
23          "options": {
24            "browser": "src/main.ts",
25            "polyfills": [
26              "zone.js"
27            ],
28            "tsConfig": "tsconfig.app.json",
29            "inlineStyleLanguage": "scss",
30            "assets": [
31              {
32                "glob": "**/*",
33                "input": "public"
34              }
35            ],
36            "styles": [
37              "src/styles.scss"
38            ],
39            "allowedCommonJsDependencies": [
40              "exceljs",
41              "file-saver"
42            ]
43          },
44          "configurations": {
45            "production": {
46              "budgets": [
47                {
48                  "type": "initial",
49                  "maximumWarning": "2MB",
50                  "maximumError": "3MB"
51                },
52                {
53                  "type": "anyComponentStyle",
54                  "maximumWarning": "4kB",
55                  "maximumError": "8kB"
56                }
57              ],
58              "outputHashing": "all"
59            },
60            "development": {
61              "optimization": false,
62              "extractLicenses": false,
63              "sourceMap": true
64            }
65          },
66          "defaultConfiguration": "production"
67        },
68        "serve": {
69          "builder": "@angular/build:dev-server",
70          "configurations": {
71            "production": {
72              "buildTarget": "app-tmp:build:production"
73            },
74            "development": {
```



```
75         "buildTarget": "app-tmp:build:development"
76     }
77 },
78     "defaultConfiguration": "development"
79 },
80     "extract-i18n": {
81         "builder": "@angular/build:extract-i18n"
82     },
83     "test": {
84         "builder": "@angular/build:karma",
85         "options": {
86             "polyfills": [
87                 "zone.js",
88                 "zone.js/testing"
89             ],
90             "tsConfig": "tsconfig.spec.json",
91             "inlineStyleLanguage": "scss",
92             "assets": [
93                 {
94                     "glob": "**/*",
95                     "input": "public"
96                 }
97             ],
98             "styles": [
99                 "src/styles.scss"
100             ]
101         }
102     }
103 }
104 }
105 }
106 }
```

```
1 {
2   "name": "app-tmp",
3   "version": "0.0.0",
4   "scripts": {
5     "ng": "ng",
6     "start": "ng serve",
7     "build": "ng build",
8     "watch": "ng build --watch --configuration development",
9     "test": "ng test"
10  },
11  "prettier": {
12    "printWidth": 100,
13    "singleQuote": true,
14    "overrides": [
15      {
16        "files": "*.html",
17        "options": {
18          "parser": "angular"
19        }
20      }
21    ]
22  },
23  "private": true,
24  "dependencies": {
25    "@angular/cdk": "^20.2.14",
26    "@angular/common": "^20.3.0",
27    "@angular/compiler": "^20.3.0",
28    "@angular/core": "^20.3.0",
29    "@angular/forms": "^20.3.0",
30    "@angular/material": "^20.2.14",
31    "@angular/platform-browser": "^20.3.0",
32    "@angular/router": "^20.3.0",
33    "exceljs": "^4.4.0",
34    "file-saver": "^2.0.5",
35    "rxjs": "~7.8.0",
36    "tslib": "^2.3.0",
37    "zone.js": "~0.15.0"
38  },
39  "devDependencies": {
40    "@angular/build": "^20.3.3",
41    "@angular/cli": "^20.3.3",
42    "@angular/compiler-cli": "^20.3.0",
43    "@types/file-saver": "^2.0.7",
44    "@types/jasmine": "~5.1.0",
45    "jasmine-core": "~5.9.0",
46    "karma": "~6.4.0",
47    "karma-chrome-launcher": "~3.2.0",
48    "karma-coverage": "~2.2.0",
49    "karma-jasmine": "~5.1.0",
50    "karma-jasmine-html-reporter": "~2.1.0",
51    "typescript": "~5.9.2"
52  }
53 }
```

```
1  /* To learn more about Typescript configuration file: https://www.typescriptlang.org/docs/handbook/tsconfig-json.html. */
2  /* To learn more about Angular compiler options: https://angular.dev/reference/configs/angular-compiler-options. */
3  {
4    "extends": "./tsconfig.json",
5    "compilerOptions": {
6      "outDir": "./out-tsc/app",
7      "types": []
8    },
9    "include": [
10     "src/**/*.ts"
11   ],
12   "exclude": [
13     "src/**/*.spec.ts"
14   ]
15 }
```

```
1  /* To learn more about Typescript configuration file: https://www.typescriptlang.org/docs/handbook/tsconfig-json.html. */
2  /* To learn more about Angular compiler options: https://angular.dev/reference/configs/angular-compiler-options. */
3  {
4    "compileOnSave": false,
5    "compilerOptions": {
6      "strict": true,
7      "noImplicitOverride": true,
8      "noPropertyAccessFromIndexSignature": true,
9      "noImplicitReturns": true,
10     "noFallthroughCasesInSwitch": true,
11     "skipLibCheck": true,
12     "isolatedModules": true,
13     "experimentalDecorators": true,
14     "importHelpers": true,
15     "target": "ES2022",
16     "module": "preserve"
17   },
18   "angularCompilerOptions": {
19     "enableI18nLegacyMessageIdFormat": false,
20     "strictInjectionParameters": true,
21     "strictInputAccessModifiers": true,
22     "typeCheckHostBindings": true,
23     "strictTemplates": true
24   },
25   "files": [],
26   "references": [
27     {
28       "path": "./tsconfig.app.json"
29     },
30     {
31       "path": "./tsconfig.spec.json"
32     }
33   ]
34 }
```

```
1  /* To learn more about Typescript configuration file: https://www.typescriptlang.org/docs/handbook/tsconfig-json.html. */
2  /* To learn more about Angular compiler options: https://angular.dev/reference/configs/angular-compiler-options. */
3  {
4    "extends": "./tsconfig.json",
5    "compilerOptions": {
6      "outDir": "./out-tsc/spec",
7      "types": [
8        "jasmine"
9      ]
10   },
11   "include": [
12     "src/**/*.ts"
13   ]
14 }
```

```
1 import { ApplicationConfig, provideBrowserGlobalErrorListeners, provideZoneChangeDetection } from '@angular/core';
2 import { MAT_DATE_LOCALE, provideNativeDateAdapter } from '@angular/material/core';
3 import { provideHttpClient } from '@angular/common/http';
4 import { REGISTRO_SERVICE } from '../core/services/registro.service';
5 import { ApiRegistroService } from '../core/services/registro-api.service';
6
7 export const appConfig: ApplicationConfig = {
8   providers: [
9     provideBrowserGlobalErrorListeners(),
10    provideZoneChangeDetection({ eventCoalescing: true }),
11    // Habilita HttpClient en toda la app; ApiRegistroService lo usa para
12    // hablar con el backend.
13    provideHttpClient(),
14    // Decide dónde se guardan los registros: ApiRegistroService los manda al
15    // backend NestJS + PostgreSQL (carpeta backend/, debe estar corriendo).
16    // Para volver a guardar solo en el navegador: useClass: LocalStorageRegistroService
17    // (importándolo de '../core/services/registro-local-storage.service').
18    { provide: REGISTRO_SERVICE, useClass: ApiRegistroService },
19    // Fecha del calendario (mat-datepicker) y del selector de hora (mat-timepicker)
20    // en español; los campos siguen guardando texto plano 'YYYY-MM-DD'/'HH:mm'
21    // (ver CampoFormularioComponent), este adaptador solo alimenta la UI.
22    provideNativeDateAdapter(),
23    { provide: MAT_DATE_LOCALE, useValue: 'es-MX' },
24  ]
25 };
```

```
1 <nav class="navegacion-vistas">
2   <button type="button" class="pestana" [class.activa]="vista === 'formulario'" (click)="irAFormulario()">Formulario</button>
3   >
4   <button type="button" class="pestana" [class.activa]="vista === 'historial'" (click)="irAHistorial()">Historial</button>
5 </nav>
6 <app-formulario
7   *ngIf="vista === 'formulario'"
8   [registroInicial]="registroACargar"
9   (enviarAlHistorial)="irAHistorial()"
10 ></app-formulario>
11 <app-historial *ngIf="vista === 'historial'" (abrir)="abrirDesdeHistorial($event)"></app-historial>
```

```
1 .navegacion-vistas {
2   display: flex;
3   gap: 4px;
4   padding: 6px 10px 0;
5   background: #fff;
6   border-bottom: 1px solid var(--umam-section-border, #b9def2);
7 }
8 .pestana {
9   min-height: 40px;
10  padding: 8px 16px;
11  border: none;
12  border-bottom: 3px solid transparent;
13  background: transparent;
14  color: #555;
15  font-size: 0.95rem;
16  font-weight: 600;
17  font-family: inherit;
18  cursor: pointer;
19  -webkit-tap-highlight-color: transparent;
20 }
21 .pestana.activa {
22   color: var(--umam-header-bg, #1892d3);
23   border-bottom-color: var(--umam-header-bg, #1892d3);
24 }
25 @media print {
26   .navegacion-vistas { display: none !important; }
27 }
```



```

1 import { TestBed } from '@angular/core/testing';
2 import { By } from '@angular/platform-browser';
3 import { provideNativeDateAdapter } from '@angular/material/core';
4 import { App } from './app';
5 import { HistorialComponent } from './features/historial/historial.component';
6 import { FormularioComponent } from './features/formulario/formulario.component';
7 import { REGISTRO_SERVICE } from './core/services/registro.service';
8 import { LocalStorageRegistroService } from './core/services/registro-local-storage.service';
9 import { crearRegistroVacio } from './core/models/registro.model';
10
11 describe('App', () => {
12   beforeEach(async () => {
13     localStorage.clear();
14     await TestBed.configureTestingModule({
15       imports: [App],
16       providers: [{ provide: REGISTRO_SERVICE, useClass: LocalStorageRegistroService }, provideNativeDateAdapter()],
17     }).compileComponents();
18   });
19
20   it('should create the app', () => {
21     const fixture = TestBed.createComponent(App);
22     const app = fixture.componentInstance;
23     expect(app).toBeTruthy();
24   });
25
26   it('should render app-formulario as the root content', () => {
27     const fixture = TestBed.createComponent(App);
28     fixture.detectChanges();
29     const compiled = fixture.nativeElement as HTMLElement;
30     expect(compiled.querySelector('app-formulario')).toBeTruthy();
31   });
32
33   it('la pestaña "Historial" muestra app-historial y oculta app-formulario; "Formulario" regresa', () => {
34     const fixture = TestBed.createComponent(App);
35     fixture.detectChanges();
36     const compiled = fixture.nativeElement as HTMLElement;
37
38     const botonHistorial = compiled.querySelectorAll<HTMLButtonElement>('button.pestana')[1];
39     botonHistorial.click();
40     fixture.detectChanges();
41     expect(compiled.querySelector('app-historial')).toBeTruthy();
42     expect(compiled.querySelector('app-formulario')).toBeFalsy();
43
44     const botonFormulario = compiled.querySelectorAll<HTMLButtonElement>('button.pestana')[0];
45     botonFormulario.click();
46     fixture.detectChanges();
47     expect(compiled.querySelector('app-formulario')).toBeTruthy();
48     expect(compiled.querySelector('app-historial')).toBeFalsy();
49   });
50
51   it('al "abrir" un trámite desde el Historial, vuelve a la vista de Formulario con ese registro', () => {
52     const fixture = TestBed.createComponent(App);
53     fixture.detectChanges();
54     fixture.componentInstance.irAHistorial();
55     fixture.detectChanges();
56
57     const registro = { ...crearRegistroVacio(), folio: 'F-HIST' };
58     const historial = fixture.debugElement.query(By.directive(HistorialComponent)).componentInstance as HistorialComponent;
59     historial.abrir.emit(registro);
60     fixture.detectChanges();
61
62     expect(fixture.componentInstance.vista).toBe('formulario');
63     expect(fixture.componentInstance.registroACargar).toBe(registro);
64     expect((fixture.nativeElement as HTMLElement).querySelector('app-formulario')).toBeTruthy();
65   });
66
67   it('al terminar "Enviar al Historial" en el formulario, cambia a la vista de Historial', () => {
68     const fixture = TestBed.createComponent(App);
69     fixture.detectChanges();
70
71     const formulario = fixture.debugElement.query(By.directive(FormularioComponent)).componentInstance as FormularioComponent;
72     formulario.enviarAlHistorial.emit();
73     fixture.detectChanges();

```

```
74
75     expect(fixture.componentInstance.vista).toBe('historial');
76     expect((fixture.nativeElement as HTMLElement).querySelector('app-historial')).toBeTruthy();
77   });
78 };
```

```
1 import { Component } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { FormularioComponent } from '../features/formulario/formulario.component';
4 import { HistorialComponent } from '../features/historial/historial.component';
5 import { RegistroAtencionPrehospitalaria } from '../core/models/registro.model';
6
7 @Component({
8   selector: 'app-root',
9   imports: [CommonModule, FormularioComponent, HistorialComponent],
10  templateUrl: './app.html',
11  styleUrls: ['./app.scss']
12 })
13 export class App {
14   vista: 'formulario' | 'historial' = 'formulario';
15   registroACargar?: RegistroAtencionPrehospitalaria;
16
17   irAFormulario(): void {
18     this.vista = 'formulario';
19   }
20
21   irAHistorial(): void {
22     this.vista = 'historial';
23   }
24
25   /** El Historial pide abrir un trámite puntual: se lo pasamos al formulario y volvemos a esa vista. */
26   abrirDesdeHistorial(registro: RegistroAtencionPrehospitalaria): void {
27     this.registroACargar = registro;
28     this.vista = 'formulario';
29   }
30 }
```

```

1 import { SECCIONES } from './secciones.data';
2
3 describe('SECCIONES', () => {
4   it('define las 16 secciones del formulario (excluye material utilizado, incluye consentimiento informado)', () => {
5     expect(SECCIONES.length).toBe(16);
6     const claves = SECCIONES.map(s => s.clave);
7     expect(claves).toEqual([
8       'datosGenerales', 'datosServicio', 'control', 'datosPaciente', 'causaTraumatica',
9       'causaClinica', 'parto', 'evaluacionInicial', 'evaluacionSecundaria',
10      'anamnesis', 'tratamiento', 'traslado', 'observaciones',
11      'datosLegales', 'hospitalReceptor', 'consentimientoInformado',
12    ]);
13   });
14
15   it('datosGenerales es la primera sección y trae los campos folio/estado/ciudad', () => {
16     const seccion = SECCIONES[0];
17     expect(seccion.clave).toBe('datosGenerales');
18     expect(seccion.campos.map(c => c.clave)).toEqual(['folio', 'estado', 'ciudad']);
19   });
20
21   it('cada sección tiene título y al menos un campo o una tabla', () => {
22     for (const seccion of SECCIONES) {
23       expect(seccion.titulo).toBeTruthy();
24       const tieneCampos = seccion.campos.length > 0;
25       const tieneTablas = (seccion.tablas ?? []).length > 0;
26       expect(tieneCampos || tieneTablas).toBeTrue();
27     }
28   });
29
30   it('evaluacionSecundaria trae la tabla de signos vitales con sus columnas (EtCO2 y hallazgos EKG en vez de EKG/examen neur
31   » ológico)', () => {
32     const seccion = SECCIONES.find(s => s.clave === 'evaluacionSecundaria')!;
33     const tabla = seccion.tablas!.find(t => t.clave === 'signosVitales')!;
34     expect(tabla.columnas.map(c => c.clave)).toEqual([
35       'hora', 'fr', 'fc', 'tas', 'tad', 'spo2', 'temp', 'gluc', 'etco2', 'hallazgosEkg',
36     ]);
37   });
38
39   it('tratamiento trae la tabla de manejo farmacológico', () => {
40     const seccion = SECCIONES.find(s => s.clave === 'tratamiento')!;
41     const tabla = seccion.tablas!.find(t => t.clave === 'manejoFarmacologico')!;
42     expect(tabla.columnas.map(c => c.clave)).toEqual([
43       'hora', 'medicamento', 'dosis', 'viaAdministracion', 'terapiaElectrica',
44     ]);
45   });
46
47   it('datosLegales trae la tabla de vehículos involucrados y ya no tiene posición/orientación ni compañía de seguro de autom
48   » óvil', () => {
49     const seccion = SECCIONES.find(s => s.clave === 'datosLegales')!;
50     const tabla = seccion.tablas!.find(t => t.clave === 'vehiculosInvolucrados')!;
51     expect(tabla.columnas.map(c => c.clave)).toEqual(['tipoMarca', 'placas']);
52     const claves = seccion.campos.map(c => c.clave);
53     expect(claves).not.toContain('posicionOrientacionPaciente');
54     expect(claves).not.toContain('companiaSeguroAutomovil');
55   });
56
57   it('hospitalReceptor trae el campo de imágenes de EKG/Rx/laboratorios', () => {
58     const seccion = SECCIONES.find(s => s.clave === 'hospitalReceptor')!;
59     const campo = seccion.campos.find(c => c.clave === 'imagenesEkgRxLaboratorios')!;
60     expect(campo).toBeTruthy();
61     expect(campo.tipo).toBe('imagenes');
62   });
63
64   it('hospitalReceptor trae firma dibujada (pluma digital / dedo) además del nombre de quien entrega y quien recibe', () => {
65     » {
66       const seccion = SECCIONES.find(s => s.clave === 'hospitalReceptor')!;
67       const firmaEntrega = seccion.campos.find(c => c.clave === 'firmaQuienEntrega')!;
68       const firmaRecibe = seccion.campos.find(c => c.clave === 'firmaQuienRecibe')!;
69       expect(firmaEntrega.tipo).toBe('firma');
70       expect(firmaRecibe.tipo).toBe('firma');
71       expect(seccion.campos.find(c => c.clave === 'nombreQuienEntrega')!.tipo).toBe('texto');
72       expect(seccion.campos.find(c => c.clave === 'nombreQuienRecibe')!.tipo).toBe('texto');
73     }
74   });
75

```

```

72 it('consentimientoInformado trae los campos de firma', () => {
73     const seccion = SECCIONES.find(s => s.clave === 'consentimientoInformado');
74     expect(seccion.campos.map(c => c.clave)).toEqual([
75         'nombrePacienteConsentimiento', 'firmaPacienteConsentimiento',
76         'nombreResponsableConsentimiento', 'firmaResponsableConsentimiento',
77         'nombreParamedicoConsentimiento', 'firmaParamedicoConsentimiento',
78     ]);
79     expect(seccion.campos.find(c => c.clave === 'firmaPacienteConsentimiento')!.tipo).toBe('firma');
80     expect(seccion.campos.find(c => c.clave === 'firmaResponsableConsentimiento')!.tipo).toBe('firma');
81     expect(seccion.campos.find(c => c.clave === 'firmaParamedicoConsentimiento')!.tipo).toBe('firma');
82 });
83
84 it('traslado trae firma dibujada para quien firma por el paciente y para el testigo', () => {
85     const seccion = SECCIONES.find(s => s.clave === 'traslado');
86     expect(seccion.campos.find(c => c.clave === 'firmaPaciente')!.tipo).toBe('firma');
87     expect(seccion.campos.find(c => c.clave === 'firmaTestigo')!.tipo).toBe('firma');
88 });
89
90 it('datosServicio ya no tiene hora de llamada, calle ni entre calles, y sí responsable de entrega', () => {
91     const seccion = SECCIONES.find(s => s.clave === 'datosServicio');
92     const claves = seccion.campos.map(c => c.clave);
93     expect(claves).not.toContain('horaLlamada');
94     expect(claves).not.toContain('calle');
95     expect(claves).not.toContain('entreCalle1');
96     expect(claves).not.toContain('entreCalle2');
97     expect(claves).toContain('responsableEntregaPaciente');
98 });
99
100 it('control ya no tiene iniciales de ambulancia', () => {
101     const seccion = SECCIONES.find(s => s.clave === 'control');
102     expect(seccion.campos.map(c => c.clave)).not.toContain('ambulanciaIniciales');
103 });
104
105 it('datosPaciente ya no tiene lugar de nacimiento y renombró derechohabiente/compañía de seguro', () => {
106     const seccion = SECCIONES.find(s => s.clave === 'datosPaciente');
107     const claves = seccion.campos.map(c => c.clave);
108     expect(claves).not.toContain('lugarNacimiento');
109     expect(claves).not.toContain('derechohabienteA');
110     expect(claves).not.toContain('companiaSeguroGastosMedicos');
111     expect(claves).toContain('numeroEmpleado');
112     expect(claves).toContain('tipoServicioMedico');
113 });
114
115 it('anamnesis ya no tiene trauma score y sí NEWS2', () => {
116     const seccion = SECCIONES.find(s => s.clave === 'anamnesis');
117     const claves = seccion.campos.map(c => c.clave);
118     expect(claves).not.toContain('traumaScoreTas');
119     expect(claves).not.toContain('traumaScoreFr');
120     expect(claves).not.toContain('traumaScoreTotal');
121     expect(claves).toContain('news2Total');
122 });
123
124 it('tratamiento agrega ventilación mecánica y parámetros del ventilador, y quita opciones retiradas', () => {
125     const seccion = SECCIONES.find(s => s.clave === 'tratamiento');
126     const claves = seccion.campos.map(c => c.clave);
127     expect(claves).toContain('ventilacionMecanica');
128     expect(claves).toContain('relacionIE');
129     expect(claves).toContain('fio2');
130     expect(claves).toContain('ps');
131
132     const viaAerea = seccion.campos.find(c => c.clave === 'viaAereaTratamiento');
133     expect(viaAerea.opciones).not.toContain('Intubación nasotraqueal');
134
135     const asistencia = seccion.campos.find(c => c.clave === 'asistenciaVentilatoria');
136     expect(asistencia.opciones).not.toContain('Válvula de demanda');
137
138     const oxigenoterapia = seccion.campos.find(c => c.clave === 'oxigenoterapia');
139     expect(oxigenoterapia.opciones).toContain('Mascarilla para nebulizar');
140     expect(oxigenoterapia.opciones).not.toContain('Mascarilla venturi');
141
142     const hemorragias = seccion.campos.find(c => c.clave === 'controlHemorragias');
143     expect(hemorragias.opciones).not.toContain('Pinzamiento de vaso');
144 });
145 });

```

```

1 import { SeccionFormulario } from '../models/campo-formulario.model';
2
3 export const SECCIONES: SeccionFormulario[] = [
4   {
5     clave: 'datosGenerales',
6     titulo: 'I. Datos Generales',
7     campos: [
8       { clave: 'folio', etiqueta: 'Folio', tipo: 'texto' },
9       { clave: 'estado', etiqueta: 'Estado', tipo: 'texto' },
10      { clave: 'ciudad', etiqueta: 'Ciudad', tipo: 'texto' },
11    ],
12  },
13  {
14    clave: 'datosServicio',
15    titulo: 'II. Datos del Servicio',
16    campos: [
17      { clave: 'fecha', etiqueta: 'Fecha', tipo: 'fecha' },
18      { clave: 'diaSemana', etiqueta: 'Día de la semana', tipo: 'texto' },
19      { clave: 'horaSalida', etiqueta: 'Hora de salida', tipo: 'hora' },
20      { clave: 'horaLlegada', etiqueta: 'Hora de llegada', tipo: 'hora' },
21      { clave: 'horaTraslado', etiqueta: 'Hora de traslado', tipo: 'hora' },
22      { clave: 'horaHospital', etiqueta: 'Hora de hospital', tipo: 'hora' },
23      { clave: 'horaLiberacion', etiqueta: 'Hora de liberación', tipo: 'hora' },
24      { clave: 'motivoAtencion', etiqueta: 'Motivo de la atención', tipo: 'radio-grupo', opciones: ['Enfermedad', 'Traumatismo', 'Gineco-Obstétrico'] },
25      { clave: 'colonia', etiqueta: 'Colonia / Comunidad', tipo: 'texto' },
26      { clave: 'delegacionMunicipio', etiqueta: 'Delegación política / Municipio', tipo: 'texto' },
27      { clave: 'lugarOcurriencia', etiqueta: 'Lugar de la ocurrencia', tipo: 'checkbox-grupo', opciones: ['Hogar', 'Vía pública', 'Trabajo', 'Escuela', 'Deporte y recreación', 'Transporte público', 'Otro'] },
28      { clave: 'lugarOcurrienciaOtro', etiqueta: 'Otro (especifique)', tipo: 'texto' },
29      { clave: 'responsableEntregaPaciente', etiqueta: 'Responsable de entrega del paciente', tipo: 'texto' },
30    ],
31  },
32  {
33    clave: 'control',
34    titulo: 'III. Control',
35    campos: [
36      { clave: 'ambulanciaNumero', etiqueta: 'Número de ambulancia', tipo: 'texto' },
37      { clave: 'operador', etiqueta: 'Operador', tipo: 'texto' },
38      { clave: 'prestadoresServicio', etiqueta: 'Prestadores del servicio', tipo: 'texto' },
39    ],
40  },
41  {
42    clave: 'datosPaciente',
43    titulo: 'IV. Datos del Paciente',
44    campos: [
45      { clave: 'nombreOMediaFiliacion', etiqueta: 'Nombre o media filiación', tipo: 'texto' },
46      { clave: 'sexo', etiqueta: 'Sexo', tipo: 'select', opciones: ['Masculino', 'Femenino'] },
47      { clave: 'edadAnios', etiqueta: 'Edad (años)', tipo: 'numero' },
48      { clave: 'edadMeses', etiqueta: 'Menores de 1 año (meses)', tipo: 'numero' },
49      { clave: 'fechaNacimiento', etiqueta: 'Fecha de nacimiento', tipo: 'fecha' },
50      { clave: 'colonia', etiqueta: 'Colonia / Comunidad', tipo: 'texto' },
51      { clave: 'delegacionMunicipio', etiqueta: 'Delegación política / Municipio', tipo: 'texto' },
52      { clave: 'telefono', etiqueta: 'Teléfono', tipo: 'texto' },
53      { clave: 'ocupacion', etiqueta: 'Ocupación', tipo: 'texto' },
54      { clave: 'numeroEmpleado', etiqueta: 'Número de empleado', tipo: 'texto' },
55      { clave: 'tipoServicioMedico', etiqueta: 'Tipo de servicio médico', tipo: 'texto' },
56    ],
57  },
58  {
59    clave: 'causaTraumatica',
60    titulo: 'V. Causa Traumática',
61    campos: [
62      { clave: 'agenteCausal', etiqueta: 'Agente causal', tipo: 'checkbox-grupo', opciones: ['Automotor', 'Bicicleta', 'Maquinaria', 'Herramienta', 'Fuego', 'Sustancia caliente', 'Sustancia tóxica', 'Electricidad', 'Explosión', 'Producto biológico', 'Arma', 'Juguete', 'Ser humano', 'Animal', 'Otro'] },
63      { clave: 'agenteCausalEspecifique', etiqueta: 'Especifique', tipo: 'texto' },
64      { clave: 'lesionesCausadasPor', etiqueta: 'Lesiones causadas por', tipo: 'texto' },
65      { clave: 'accidenteAutomovilistico', etiqueta: 'Accidente automovilístico', tipo: 'checkbox-grupo', opciones: ['Colisión', 'Volcadura', 'Automotor', 'Motocicleta', 'Bicicleta', 'Maquinaria'] },
66      { clave: 'contraObjeto', etiqueta: 'Contra objeto', tipo: 'radio-grupo', opciones: ['Fijo', 'En movimiento'] },
67      { clave: 'impacto', etiqueta: 'Impacto', tipo: 'radio-grupo', opciones: ['Frontal', 'Lateral', 'Posterior'] },
68      { clave: 'hundimientoCms', etiqueta: 'Hundimiento', tipo: 'numero', sufijo: 'cm' },
69      { clave: 'parabrisas', etiqueta: 'Parabrisas', tipo: 'radio-grupo', opciones: ['Íntegro', 'Roto/Doblado'] },

```

```

70     { clave: 'volante', etiqueta: 'Volante', tipo: 'radio-grupo', opciones: ['Íntegro', 'Doblado'] },
71     { clave: 'bolsasAire', etiqueta: 'Bolsas de aire', tipo: 'radio-grupo', opciones: ['Sí', 'No'] },
72     { clave: 'cinturonSeguridad', etiqueta: 'Cinturón de seguridad', tipo: 'radio-grupo', opciones: ['Colocado', 'No coloc
» ado'] },
73     { clave: 'dentroVehiculo', etiqueta: 'Dentro del vehículo', tipo: 'radio-grupo', opciones: ['Sí', 'No'] },
74     { clave: 'eyectado', etiqueta: 'Eyectado', tipo: 'checkbox' },
75     { clave: 'atropellado', etiqueta: 'Atropellado por', tipo: 'checkbox-grupo', opciones: ['Automotor', 'Motocicleta', 'B
» icicleta', 'Maquinaria'] },
76   ],
77 },
78 {
79   clave: 'causaClinica',
80   titulo: 'VI. Causa Clínica',
81   campos: [
82     { clave: 'origenProbable', etiqueta: 'Origen probable', tipo: 'checkbox-grupo', opciones: ['Metabólico', 'Respiratorio
» ', 'Cardiovascular', 'Neurológico', 'Urogenital', 'Gineco-obstétrico', 'Digestivo', 'Psico-emotivo', 'Oncológico', 'Infeccio
» so', 'Músculo esquelético', 'Otro'] },
83     { clave: 'origenProbableEspecifique', etiqueta: 'Especifique', tipo: 'texto' },
84     { clave: 'primeraVez', etiqueta: '1era vez', tipo: 'radio-grupo', opciones: ['Sí', 'No'] },
85     { clave: 'subsecuente', etiqueta: 'Subsecuente', tipo: 'texto' },
86   ],
87 },
88 {
89   clave: 'parto',
90   titulo: 'VII. Parto',
91   campos: [
92     { clave: 'gesta', etiqueta: 'Gesta', tipo: 'numero' },
93     { clave: 'cesarea', etiqueta: 'Cesárea', tipo: 'numero' },
94     { clave: 'para', etiqueta: 'Para', tipo: 'numero' },
95     { clave: 'abortos', etiqueta: 'Abortos', tipo: 'numero' },
96     { clave: 'semanaGestacion', etiqueta: 'Semana de gestación', tipo: 'numero' },
97     { clave: 'fechaProbableParto', etiqueta: 'Fecha probable de parto', tipo: 'fecha' },
98     { clave: 'membranas', etiqueta: 'Membranas', tipo: 'texto' },
99     { clave: 'horaInicioContracciones', etiqueta: 'Hora de inicio de contracciones', tipo: 'hora' },
100    { clave: 'frecuenciaContracciones', etiqueta: 'Frecuencia', tipo: 'texto' },
101    { clave: 'duracionContracciones', etiqueta: 'Duración', tipo: 'texto' },
102    { clave: 'horaNacimiento', etiqueta: 'Hora de nacimiento', tipo: 'hora' },
103    { clave: 'lugarNacimientoParto', etiqueta: 'Lugar', tipo: 'texto' },
104    { clave: 'placentaExpulsada', etiqueta: 'Placenta expulsada', tipo: 'radio-grupo', opciones: ['Sí', 'No'] },
105    { clave: 'productoVivoMuerto', etiqueta: 'Producto', tipo: 'radio-grupo', opciones: ['Vivo', 'Muerto'] },
106    { clave: 'sexoRecienNacido', etiqueta: 'Sexo del recién nacido', tipo: 'radio-grupo', opciones: ['Masculino', 'Femenin
» o'] },
107    { clave: 'apgar1min', etiqueta: 'APGAR 1 min', tipo: 'numero' },
108    { clave: 'apgar5min', etiqueta: 'APGAR 5 min', tipo: 'numero' },
109    { clave: 'apgar10min', etiqueta: 'APGAR 10 min', tipo: 'numero' },
110    { clave: 'silverman1min', etiqueta: 'Silverman 1 min', tipo: 'numero' },
111    { clave: 'silverman5min', etiqueta: 'Silverman 5 min', tipo: 'numero' },
112  ],
113 },
114 {
115   clave: 'evaluacionInicial',
116   titulo: 'VIII. Evaluación Inicial',
117   campos: [
118     { clave: 'nivelConciencia', etiqueta: 'Nivel de conciencia', tipo: 'radio-grupo', opciones: ['Consciente', 'Respuesta
» a estímulo verbal', 'Respuesta a estímulo doloroso', 'Inconsciente'] },
119     { clave: 'viaAerea', etiqueta: 'Vía aérea', tipo: 'radio-grupo', opciones: ['Permeable', 'Comprometida'] },
120     { clave: 'reflejoDeglucion', etiqueta: 'Reflejo de deglución', tipo: 'radio-grupo', opciones: ['Presente', 'Ausente']
» },
121     { clave: 'ventilacionObservacion', etiqueta: 'Ventilación – observación', tipo: 'checkbox-grupo', opciones: ['Automati
» smo regular', 'Automatismo irregular', 'Ventilación rápida', 'Ventilación lenta', 'Ventilación profunda', 'Ventilación super
» ficial', 'Apnea'] },
122     { clave: 'ventilacionAuscultacion', etiqueta: 'Ventilación – auscultación', tipo: 'checkbox-grupo', opciones: ['Ruidos
» respiratorios normales', 'Ruidos respiratorios disminuidos', 'Ruidos respiratorios anormales', 'Ruidos respiratorios ausent
» es'] },
123     { clave: 'ventilacionAuscultacionEspecifique', etiqueta: 'Auscultación – especifique', tipo: 'texto' },
124     { clave: 'alternacionesLocalizadas', etiqueta: 'Alternaciones localizadas en', tipo: 'checkbox-grupo', opciones: ['Hem
» itórax derecho', 'Hemitórax izquierdo', 'Ápices', 'Bases'] },
125     { clave: 'presenciaPulsos', etiqueta: 'Presencia de pulsos', tipo: 'checkbox-grupo', opciones: ['Carotídeo', 'Radial']
» },
126     { clave: 'calidadPulso', etiqueta: 'Calidad del pulso', tipo: 'checkbox-grupo', opciones: ['Normal', 'Rápido', 'Lento'
» , 'Rítmico', 'Arrítmico'] },
127     { clave: 'paroCardiorespiratorio', etiqueta: 'Paro cardiorespiratorio', tipo: 'checkbox' },
128     { clave: 'piel', etiqueta: 'Piel', tipo: 'checkbox-grupo', opciones: ['Normal', 'Rojiza', 'Pálida', 'Cianótica', 'Icte
» ricia'] },

```



```

129     { clave: 'caracteristicasPiel', etiqueta: 'Características de la piel', tipo: 'checkbox-grupo', opciones: ['Húmeda', '
    » Seca', 'Normal'] },
130     { clave: 'temperaturaComparativa', etiqueta: 'Temperatura comparativa', tipo: 'checkbox-grupo', opciones: ['Normal', '
    » Caliente', 'Fría'] },
131   ],
132 },
133 {
134   clave: 'evaluacionSecundaria',
135   titulo: 'IX. Evaluación Secundaria',
136   campos: [
137     { clave: 'exploracionFisica', etiqueta: 'Exploración física', tipo: 'checkbox-grupo', opciones: ['Deformaciones', 'Con
    » tusiones', 'Abrasiones', 'Penetraciones', 'Movimientos paradójicos', 'Crepitación', 'Heridas', 'Fracturas', 'Enfisema subcut
    » áneo', 'Quemaduras', 'Laceraciones', 'Edema', 'Alteraciones de sensibilidad', 'Alteraciones de movilidad', 'Dolor', 'Amputac
    » ión'] },
138     { clave: 'zonasLesion', etiqueta: 'Zonas de lesión', tipo: 'checkbox-grupo', opciones: ['Cabeza', 'Cuello', 'Tórax ant
    » erior', 'Tórax posterior', 'Abdomen', 'Pelvis', 'Brazo izquierdo', 'Brazo derecho', 'Antebrazo izquierdo', 'Antebrazo derech
    » o', 'Mano izquierda', 'Mano derecha', 'Muslo izquierdo', 'Muslo derecho', 'Pierna izquierda', 'Pierna derecha', 'Pie izquier
    » do', 'Pie derecho', 'Espalda'] },
139     { clave: 'zonasLesionEspecifique', etiqueta: 'Zonas de lesión – especifique', tipo: 'texto' },
140     { clave: 'pupilas', etiqueta: 'Pupilas', tipo: 'checkbox-grupo', opciones: ['Normales', 'Midriáticas', 'Mióticas', 'An
    » isocóricas', 'No reactivas'] },
141   ],
142   tablas: [
143     {
144       clave: 'signosVitales',
145       titulo: 'Signos vitales y monitoreo',
146       columnas: [
147         { clave: 'hora', etiqueta: 'Hora', tipo: 'hora' },
148         { clave: 'fr', etiqueta: 'FR', tipo: 'numero' },
149         { clave: 'fc', etiqueta: 'FC', tipo: 'numero' },
150         { clave: 'tas', etiqueta: 'TAS', tipo: 'numero' },
151         { clave: 'tad', etiqueta: 'TAD', tipo: 'numero' },
152         { clave: 'spo2', etiqueta: 'SpO2', tipo: 'numero' },
153         { clave: 'temp', etiqueta: 'TEMP', tipo: 'numero' },
154         { clave: 'gluc', etiqueta: 'GLUC', tipo: 'numero' },
155         { clave: 'etco2', etiqueta: 'EtCO2', tipo: 'texto' },
156         { clave: 'hallazgosEkg', etiqueta: 'Hallazgos EKG', tipo: 'texto' },
157       ],
158     },
159   ],
160 },
161 {
162   clave: 'anamnesis',
163   titulo: 'X. Anamnesis',
164   campos: [
165     { clave: 'interrogatorio', etiqueta: 'Interrogatorio', tipo: 'radio-grupo', opciones: ['Directo', 'Indirecto'] },
166     { clave: 'peso', etiqueta: 'Peso', tipo: 'numero', sufijo: 'kg' },
167     { clave: 'talla', etiqueta: 'Talla', tipo: 'numero', sufijo: 'cm' },
168     { clave: 'alergias', etiqueta: 'Alergias', tipo: 'textarea' },
169     { clave: 'medicamentos', etiqueta: 'Medicamentos', tipo: 'textarea' },
170     { clave: 'enfermedadesCirugiasPrevias', etiqueta: 'Enfermedades y/o cirugías previas', tipo: 'textarea' },
171     { clave: 'horaUltimaComida', etiqueta: 'Hora de última comida', tipo: 'hora' },
172     { clave: 'eventosPreviosRelacionados', etiqueta: 'Eventos previos relacionados', tipo: 'textarea' },
173     { clave: 'condicionPaciente', etiqueta: 'Condición del paciente', tipo: 'radio-grupo', opciones: ['Crítico', 'No críti
    » co'] },
174     { clave: 'estabilidadPaciente', etiqueta: 'Estabilidad', tipo: 'radio-grupo', opciones: ['Inestable', 'Estable'] },
175     { clave: 'prioridad', etiqueta: 'Prioridad', tipo: 'radio-grupo', opciones: ['Rojo', 'Amarillo', 'Verde', 'Negro'] },
176     { clave: 'glasgowOcular', etiqueta: 'Glasgow – ocular', tipo: 'numero' },
177     { clave: 'glasgowVerbal', etiqueta: 'Glasgow – verbal', tipo: 'numero' },
178     { clave: 'glasgowMotora', etiqueta: 'Glasgow – motora', tipo: 'numero' },
179     { clave: 'glasgowTotal', etiqueta: 'Glasgow – total', tipo: 'numero' },
180     { clave: 'news2Total', etiqueta: 'NEWS2 – puntaje total', tipo: 'numero' },
181   ],
182 },
183 {
184   clave: 'tratamiento',
185   titulo: 'XI. Tratamiento',
186   campos: [
187     { clave: 'viaAereaTratamiento', etiqueta: 'Vía aérea', tipo: 'checkbox-grupo', opciones: ['Aspiración', 'Cánula orofar
    » íngea', 'Cánula nasofaríngea', 'Intubación endotraqueal', 'Dispositivo supraglótico', 'Vía aérea quirúrgica'] },
188     { clave: 'controlCervical', etiqueta: 'Control cervical', tipo: 'radio-grupo', opciones: ['Manual', 'Collarín rígido',
    » 'Collarín blando'] },
189     { clave: 'asistenciaVentilatoria', etiqueta: 'Asistencia ventilatoria', tipo: 'checkbox-grupo', opciones: ['Balón-válv
    » ula mascarilla', 'Ventilador automático'] },

```



```

190     { clave: 'ventilacionMecanica', etiqueta: 'Ventilación mecánica', tipo: 'checkbox-grupo', opciones: ['AC/V', 'AC/P', '
    » CPAP/PS'] },
191     { clave: 'frecuenciaVentilatoria', etiqueta: 'Frecuencia', tipo: 'numero' },
192     { clave: 'volumenVentilatorio', etiqueta: 'Volumen', tipo: 'numero' },
193     { clave: 'peep', etiqueta: 'PEEP', tipo: 'numero' },
194     { clave: 'presionVentilatoria', etiqueta: 'PIP', tipo: 'numero' },
195     { clave: 'relacionIE', etiqueta: 'I:E', tipo: 'texto' },
196     { clave: 'fio2', etiqueta: 'FiO2', tipo: 'numero', sufijo: '%' },
197     { clave: 'ps', etiqueta: 'PS', tipo: 'numero' },
198     { clave: 'modoVentilatorio', etiqueta: 'Modo ventilatorio', tipo: 'texto' },
199     { clave: 'oxigenoterapia', etiqueta: 'Oxigenoterapia', tipo: 'checkbox-grupo', opciones: ['Puntas nasales', 'Mascarill
    » a simple', 'Mascarilla con reservorio', 'Mascarilla para nebulizar'] },
200     { clave: 'litrosPorMinuto', etiqueta: 'Lts x min', tipo: 'numero' },
201     { clave: 'descompresionPleural', etiqueta: 'Descompresión pleural', tipo: 'checkbox-grupo', opciones: ['Hemitórax dere
    » cho', 'Hemitórax izquierdo'] },
202     { clave: 'controlHemorragias', etiqueta: 'Control de hemorragias', tipo: 'checkbox-grupo', opciones: ['Presión directa
    » ', 'Vendaje compresivo', 'Torniquete', 'Empaquetamiento de herida'] },
203     { clave: 'lineasIV', etiqueta: 'Líneas IV #', tipo: 'numero' },
204     { clave: 'numeroCateter', etiqueta: 'Catéter #', tipo: 'texto' },
205     { clave: 'sitioAplicacion', etiqueta: 'Sitio de aplicación', tipo: 'checkbox-grupo', opciones: ['Mano', 'Pliegue antec
    » ubital', 'Intraósea', 'Otra'] },
206     { clave: 'sitioAplicacionEspecifique', etiqueta: 'Sitio de aplicación – especifique', tipo: 'texto' },
207     { clave: 'tipoSoluciones', etiqueta: 'Tipo de soluciones', tipo: 'texto' },
208     { clave: 'cantidadSolucion', etiqueta: 'Cantidad', tipo: 'texto' },
209     { clave: 'infusion', etiqueta: 'Infusión', tipo: 'texto' },
210     { clave: 'rcp', etiqueta: 'RCP', tipo: 'checkbox-grupo', opciones: ['RCP básica', 'RCP avanzada'] },
211     { clave: 'inmovilizacion', etiqueta: 'Inmovilización', tipo: 'checkbox-grupo', opciones: ['Inmovilización de extremida
    » d(es)', 'Inmovilización de columna'] },
212     { clave: 'curacion', etiqueta: 'Curación', tipo: 'checkbox' },
213     { clave: 'vendaje', etiqueta: 'Vendaje', tipo: 'checkbox' },
214   ],
215   tablas: [
216     {
217       clave: 'manejoFarmacologico',
218       titulo: 'Manejo farmacológico y terapia eléctrica',
219       columnas: [
220         { clave: 'hora', etiqueta: 'Hora', tipo: 'hora' },
221         { clave: 'medicamento', etiqueta: 'Medicamento', tipo: 'texto' },
222         { clave: 'dosis', etiqueta: 'Dosis', tipo: 'texto' },
223         { clave: 'viaAdministracion', etiqueta: 'Vía de administración', tipo: 'texto' },
224         { clave: 'terapiaElectrica', etiqueta: 'Terapia eléctrica', tipo: 'texto' },
225       ],
226     },
227   ],
228 },
229 {
230   clave: 'traslado',
231   titulo: 'XII. Traslado',
232   campos: [
233     { clave: 'institucionTraslado', etiqueta: 'Institución a la que se traslada el paciente', tipo: 'texto' },
234     { clave: 'condicionPacienteTraslado', etiqueta: 'Condición del paciente', tipo: 'radio-grupo', opciones: ['Crítico', '
    » No crítico'] },
235     { clave: 'estabilidadPacienteTraslado', etiqueta: 'Estabilidad', tipo: 'radio-grupo', opciones: ['Inestable', 'Estable
    » ' ] },
236     { clave: 'prioridadTraslado', etiqueta: 'Prioridad de traslado', tipo: 'radio-grupo', opciones: ['Rojo', 'Amarillo', '
    » Verde', 'Negro'] },
237     { clave: 'negativaAtencion', etiqueta: 'Negativa a recibir atención / ser trasladado (eximente de responsabilidad)', t
    » ipo: 'checkbox' },
238     { clave: 'nombrePaciente', etiqueta: 'Nombre de quien firma por el paciente', tipo: 'texto' },
239     { clave: 'firmaPaciente', etiqueta: 'Firma de quien firma por el paciente', tipo: 'firma' },
240     { clave: 'nombreTestigo', etiqueta: 'Nombre del testigo', tipo: 'texto' },
241     { clave: 'firmaTestigo', etiqueta: 'Firma del testigo', tipo: 'firma' },
242   ],
243 },
244 {
245   clave: 'observaciones',
246   titulo: 'XIII. Observaciones',
247   campos: [
248     { clave: 'observaciones', etiqueta: 'Observaciones', tipo: 'textarea' },
249     { clave: 'observacionesAdicionales', etiqueta: 'Más observaciones', tipo: 'textarea' },
250     { clave: 'ministerioPublicoNotificado', etiqueta: 'Ministerio Público notificado', tipo: 'checkbox' },
251     { clave: 'selloMinisterioPublico', etiqueta: 'Sello / folio de Ministerio Público', tipo: 'texto' },
252     { clave: 'nombreQuienRecibeMP', etiqueta: 'Nombre y firma de quien recibe', tipo: 'texto' },
253   ],

```

```
254   },
255   {
256     clave: 'datosLegales',
257     titulo: 'XV. Datos Legales',
258     campos: [
259       { clave: 'dependenciaAutoridad', etiqueta: 'Dependencia', tipo: 'texto' },
260       { clave: 'numeroUnidades', etiqueta: 'Número de unidades', tipo: 'texto' },
261       { clave: 'nombreNumeroOficiales', etiqueta: 'Nombre o número de los oficiales', tipo: 'texto' },
262       { clave: 'pertenencias', etiqueta: 'Pertenencias', tipo: 'textarea' },
263       { clave: 'recibioPertenenciasNombreFirmaCargo', etiqueta: 'Recibió las pertenencias (nombre, firma y cargo)', tipo: 't
    » exto' },
264     ],
265     tablas: [
266       {
267         clave: 'vehiculosInvolucrados',
268         titulo: 'Vehículos involucrados',
269         columnas: [
270           { clave: 'tipoMarca', etiqueta: 'Tipo y marca', tipo: 'texto' },
271           { clave: 'placas', etiqueta: 'Placas', tipo: 'texto' },
272         ],
273       },
274     ],
275   },
276   {
277     clave: 'hospitalReceptor',
278     titulo: 'XVI. Hospital Receptor',
279     campos: [
280       { clave: 'aceptacionHospitalReceptor', etiqueta: 'Aceptación de hospital receptor', tipo: 'textarea' },
281       { clave: 'nombreQuienEntrega', etiqueta: 'Nombre de quien entrega al paciente', tipo: 'texto' },
282       { clave: 'firmaQuienEntrega', etiqueta: 'Firma de quien entrega al paciente', tipo: 'firma' },
283       { clave: 'nombreQuienRecibe', etiqueta: 'Nombre de persona que recibe al paciente', tipo: 'texto' },
284       { clave: 'firmaQuienRecibe', etiqueta: 'Firma de persona que recibe al paciente', tipo: 'firma' },
285       { clave: 'imagenesEkgRxLaboratorios', etiqueta: 'Imágenes de EKG, Rx y/o laboratorios', tipo: 'imagenes' },
286     ],
287   },
288   {
289     clave: 'consentimientoInformado',
290     titulo: 'XVII. Consentimiento Informado',
291     campos: [
292       { clave: 'nombrePacienteConsentimiento', etiqueta: 'Nombre y firma del paciente', tipo: 'texto' },
293       { clave: 'firmaPacienteConsentimiento', etiqueta: 'Firma del paciente', tipo: 'firma' },
294       { clave: 'nombreResponsableConsentimiento', etiqueta: 'Nombre y firma del familiar o responsable', tipo: 'texto' },
295       { clave: 'firmaResponsableConsentimiento', etiqueta: 'Firma del familiar o responsable', tipo: 'firma' },
296       { clave: 'nombreParamedicoConsentimiento', etiqueta: 'Nombre y firma del paramédico', tipo: 'texto' },
297       { clave: 'firmaParamedicoConsentimiento', etiqueta: 'Firma del paramédico', tipo: 'firma' },
298     ],
299   },
300 ];
```

```

1 import { FormBuilder, FormGroup } from '@angular/forms';
2 import { construirFormularioRegistro } from './construir-formulario';
3 import { crearRegistroVacio } from '../models/registro.model';
4 import { SECCIONES } from '../data/secciones.data';
5
6 describe('construirFormularioRegistro', () => {
7   const fb = new FormBuilder();
8
9   it('crea un FormGroup por cada sección, incluyendo datosGenerales con folio/estado/ciudad', () => {
10     const form = construirFormularioRegistro(fb, crearRegistroVacio());
11
12     const datosGenerales = form.get('datosGenerales') as FormGroup;
13     expect(datosGenerales.contains('folio')).toBeTrue();
14     expect(datosGenerales.contains('estado')).toBeTrue();
15     expect(datosGenerales.contains('ciudad')).toBeTrue();
16
17     for (const seccion of SECCIONES) {
18       expect(form.contains(seccion.clave)).toBeTrue();
19       const grupoSeccion = form.get(seccion.clave)!;
20       for (const campo of seccion.campos) {
21         expect(grupoSeccion.get(campo.clave)).withContext(`${seccion.clave}.${campo.clave}`).toBeTruthy();
22       }
23     }
24   });
25
26   it('siembra datosGenerales.folio/estado/ciudad desde los campos planos del registro', () => {
27     const registro = { ...crearRegistroVacio(), folio: 'F-100', estado: 'Chihuahua', ciudad: 'Cd. Juárez' };
28     const form = construirFormularioRegistro(fb, registro);
29     expect(form.get('datosGenerales')!.get('folio')!.value).toBe('F-100');
30     expect(form.get('datosGenerales')!.get('estado')!.value).toBe('Chihuahua');
31     expect(form.get('datosGenerales')!.get('ciudad')!.value).toBe('Cd. Juárez');
32   });
33
34   it('crea un FormArray vacío por cada tabla repetible', () => {
35     const form = construirFormularioRegistro(fb, crearRegistroVacio());
36     expect(form.get('signosVitales')).toBeTruthy();
37     expect((form.get('signosVitales') as any).length).toBe(0);
38     expect(form.get('manejoFarmacologico')).toBeTruthy();
39     expect(form.get('vehiculosInvolucrados')).toBeTruthy();
40   });
41
42   it('un campo tipo "imagenes" tiene valor por defecto [] (arreglo de imágenes)', () => {
43     const form = construirFormularioRegistro(fb, crearRegistroVacio());
44     const control = form.get('hospitalReceptor')!.get('imagenesEkgRxLaboratorios')!;
45     expect(control.value).toEqual([]);
46   });
47
48   it('un campo tipo "checkbox" (booleano simple) tiene valor por defecto false, no []', () => {
49     const form = construirFormularioRegistro(fb, crearRegistroVacio());
50     // causaTraumatica.eyectado es un campo tipo 'checkbox' (booleano simple, no checkbox-grupo).
51     const control = form.get('causaTraumatica')!.get('eyectado')!;
52     expect(control.value).toBe(false);
53     expect(Array.isArray(control.value)).toBeFalse();
54   });
55
56   it('un campo tipo "checkbox-grupo" sigue teniendo valor por defecto []', () => {
57     const form = construirFormularioRegistro(fb, crearRegistroVacio());
58     const control = form.get('causaTraumatica')!.get('agenteCausal')!;
59     expect(control.value).toEqual([]);
60   });
61
62   it('agregar una fila a signosVitales crea los controles de columna definidos', () => {
63     const form = construirFormularioRegistro(fb, crearRegistroVacio());
64     const tablaCfg = SECCIONES.find(s => s.clave === 'evaluacionSecundaria')!.tablas!.find(t => t.clave === 'signosVitales')
65     » !;
66     const fila = fb.group(Object.fromEntries(tablaCfg.columnas.map(c => [c.clave, ['']])));
67     (form.get('signosVitales') as any).push(fila);
68     expect((form.get('signosVitales') as any).length).toBe(1);
69     expect(fila.get('fc')).toBeTruthy();
70   });

```

```
1 import { FormArray, FormBuilder, FormGroup } from '@angular/forms';
2 import { SECCIONES } from '../data/secciones.data';
3 import { RegistroAtencionPrehospitalaria } from '../models/registro.model';
4
5 export function construirFormularioRegistro(
6   fb: FormBuilder,
7   registro: RegistroAtencionPrehospitalaria,
8 ): FormGroup {
9   const grupo: Record<string, unknown> = {};
10
11   for (const seccion of SECCIONES) {
12     // 'datosGenerales' es una sección normal en SECCIONES (para que el acordeón y
13     // ambos exportadores la recorran igual que cualquier otra), pero sus valores
14     // guardados viven como campos planos en la raíz del modelo (registro.folio,
15     // registro.estado, registro.ciudad) en vez de bajo registro.datosGenerales
16     // – así que sembramos el grupo desde ahí en vez de leer registro[seccion.clave].
17     const valoresSeccion = seccion.clave === 'datosGenerales'
18       ? { folio: registro.folio, estado: registro.estado, ciudad: registro.ciudad }
19       : (registro as unknown as Record<string, unknown>)[seccion.clave] as Record<string, unknown> | undefined;
20     const controlesSeccion: Record<string, unknown> = {};
21     for (const campo of seccion.campos) {
22       const valorPorDefecto = campo.tipo === 'checkbox'
23         ? false
24         : campo.tipo === 'checkbox-grupo' || campo.tipo === 'imagenes'
25           ? []
26           : '';
27       controlesSeccion[campo.clave] = [valoresSeccion?.[campo.clave] ?? valorPorDefecto];
28     }
29     grupo[seccion.clave] = fb.group(controlesSeccion);
30
31     for (const tabla of seccion.tablas ?? []) {
32       const filasGuardadas = (registro as unknown as Record<string, unknown>)[tabla.clave] as Record<string, unknown>[] | un
33       » defined ?? [];
34       const filas = filasGuardadas.map(fila =>
35         fb.group(Object.fromEntries(tabla.columnas.map(c => [c.clave, [fila[c.clave] ?? '']]))),
36       );
37       grupo[tabla.clave] = fb.array(filas);
38     }
39   }
40   return fb.group(grupo);
41 }
42
43 export function nuevaFilaTabla(fb: FormBuilder, columnas: { clave: string }[]): FormGroup {
44   return fb.group(Object.fromEntries(columnas.map(c => [c.clave, ['']]));
45 }
```

```
1  export type TipoCampo =
2    | 'texto'
3    | 'numero'
4    | 'fecha'
5    | 'hora'
6    | 'textarea'
7    | 'checkbox'
8    | 'checkbox-grupo'
9    | 'radio-grupo'
10   | 'select'
11   | 'imagenes'
12   | 'firma';
13
14  export interface CampoFormulario {
15    clave: string;
16    etiqueta: string;
17    tipo: TipoCampo;
18    opciones?: string[];
19    sufijo?: string;
20  }
21
22  export interface TablaRepetible {
23    clave: string;
24    titulo: string;
25    columnas: CampoFormulario[];
26  }
27
28  export interface SeccionFormulario {
29    clave: string;
30    titulo: string;
31    campos: CampoFormulario[];
32    tablas?: TablaRepetible[];
33  }
```

```
1 import { crearRegistroVacio } from './registro.model';
2
3 describe('crearRegistroVacio', () => {
4   it('crea un registro con id, fechaCreacion y todas las secciones top-level presentes', () => {
5     const registro = crearRegistroVacio();
6
7     expect(registro.id).toBeTruthy();
8     expect(registro.fechaCreacion).toBeTruthy();
9     expect(registro.folio).toBe('');
10    expect(registro.consentimientoInformado).toEqual({});
11    expect(registro.signosVitales).toEqual([]);
12    expect(registro.manejoFarmacologico).toEqual([]);
13    expect(registro.vehiculosInvolucrados).toEqual([]);
14  });
15
16  it('genera un id distinto en cada llamada', () => {
17    const a = crearRegistroVacio();
18    const b = crearRegistroVacio();
19    expect(a.id).not.toBe(b.id);
20  });
21 });
```

```
1 export interface RegistroAtencionPrehospitalaria {
2   id: string;
3   fechaCreacion: string;
4   estado: string;
5   ciudad: string;
6   folio: string;
7
8   // Secciones cuyos campos vienen de secciones.data.ts (Record<clave, valor>)
9   datosServicio: Record<string, unknown>;
10  control: Record<string, unknown>;
11  datosPaciente: Record<string, unknown>;
12  causaTraumatica: Record<string, unknown>;
13  causaClinica: Record<string, unknown>;
14  parto: Record<string, unknown>;
15  evaluacionInicial: Record<string, unknown>;
16  evaluacionSecundaria: Record<string, unknown>;
17  anamnesis: Record<string, unknown>;
18  tratamiento: Record<string, unknown>;
19  traslado: Record<string, unknown>;
20  observaciones: Record<string, unknown>;
21  datosLegales: Record<string, unknown>;
22  hospitalReceptor: Record<string, unknown>;
23  consentimientoInformado: Record<string, unknown>;
24
25  // Tablas repetibles (una fila = un Record<claveColumna, valor>)
26  signosVitales: Record<string, unknown>[];
27  manejoFarmacologico: Record<string, unknown>[];
28  vehiculosInvolucrados: Record<string, unknown>[];
29 }
30
31 export function crearRegistroVacio(): RegistroAtencionPrehospitalaria {
32   return {
33     id: crypto.randomUUID(),
34     fechaCreacion: new Date().toISOString(),
35     estado: '',
36     ciudad: '',
37     folio: '',
38     datosServicio: {},
39     control: {},
40     datosPaciente: {},
41     causaTraumatica: {},
42     causaClinica: {},
43     parto: {},
44     evaluacionInicial: {},
45     evaluacionSecundaria: {},
46     anamnesis: {},
47     tratamiento: {},
48     traslado: {},
49     observaciones: {},
50     datosLegales: {},
51     hospitalReceptor: {},
52     consentimientoInformado: {},
53     signosVitales: [],
54     manejoFarmacologico: [],
55     vehiculosInvolucrados: [],
56   };
57 }
```

```
1 import { TestBed } from '@angular/core/testing';
2 import { provideHttpClient } from '@angular/common/http';
3 import { HttpTestingController, provideHttpClientTesting } from '@angular/common/http/testing';
4 import { API_REGISTROS_URL, ApiRegistroService } from './registro-api.service';
5 import { crearRegistroVacio } from '../models/registro.model';
6
7 describe('ApiRegistroService', () => {
8   let servicio: ApiRegistroService;
9   let http: HttpTestingController;
10
11   beforeEach(() => {
12     TestBed.configureTestingModule({
13       providers: [ApiRegistroService, provideHttpClient(), provideHttpClientTesting()],
14     });
15     servicio = TestBed.inject(ApiRegistroService);
16     http = TestBed.inject(HttpTestingController);
17   });
18
19   afterEach(() => http.verify());
20
21   it('guardar() hace PUT al id del registro', async () => {
22     const registro = crearRegistroVacio();
23     const promesa = servicio.guardar(registro);
24     const req = http.expectOne(`${API_REGISTROS_URL}/${registro.id}`);
25     expect(req.request.method).toBe('PUT');
26     expect(req.request.body).toEqual(registro);
27     req.flush(registro);
28     expect(await promesa).toEqual(registro);
29   });
30
31   it('obtener() devuelve undefined si el backend responde 404', async () => {
32     const promesa = servicio.obtener('no-existe');
33     http.expectOne(`${API_REGISTROS_URL}/no-existe`).flush(null, { status: 404, statusText: 'Not Found' });
34     expect(await promesa).toBeUndefined();
35   });
36
37   it('listar() hace GET a la colección', async () => {
38     const promesa = servicio.listar();
39     http.expectOne(API_REGISTROS_URL).flush([]);
40     expect(await promesa).toEqual([]);
41   });
42
43   it('eliminar() hace DELETE al id', async () => {
44     const promesa = servicio.eliminar('abc');
45     const req = http.expectOne(`${API_REGISTROS_URL}/abc`);
46     expect(req.request.method).toBe('DELETE');
47     req.flush(null, { status: 204, statusText: 'No Content' });
48     await promesa;
49   });
50 });
```



```

1 import { HttpClient, HttpResponse } from '@angular/common/http';
2 import { Injectable, inject } from '@angular/core';
3 import { firstValueFrom } from 'rxjs';
4 import { RegistroService } from '../registro.service';
5 import { RegistroAtencionPrehospitalaria } from '../models/registro.model';
6
7 /**
8  * URL del backend NestJS (carpeta backend/). Si el backend se despliega en
9  * otro servidor, este es el único lugar que hay que cambiar.
10 */
11 export const API_REGISTROS_URL = 'http://localhost:3000/api/registros';
12
13 /**
14  * Implementación de RegistroService que guarda los registros en el backend
15  * (PostgreSQL) en lugar de localStorage. Los componentes no saben cuál de las
16  * dos se usa: solo piden REGISTRO_SERVICE y app.config.ts decide.
17  *
18  * HttpClient devuelve Observables; firstValueFrom() los convierte en Promesas
19  * porque la interfaz RegistroService trabaja con async/await.
20 */
21 @Injectable()
22 export class ApiRegistroService implements RegistroService {
23   private http = inject(HttpClient);
24
25   /**
26    * PUT /api/registros/:id - crea el registro si es nuevo o lo actualiza si
27    * ya existe. Lo llama el autoguardado del formulario cada vez que cambia algo.
28    */
29   guardar(registro: RegistroAtencionPrehospitalaria): Promise<RegistroAtencionPrehospitalaria> {
30     return firstValueFrom(
31       this.http.put<RegistroAtencionPrehospitalaria>(`${API_REGISTROS_URL}/${registro.id}`, registro),
32     );
33   }
34
35   /**
36    * GET /api/registros/:id - el backend responde 404 cuando no existe; aquí
37    * se traduce a `undefined` para cumplir el contrato de RegistroService
38    * (igual que hace LocalStorageRegistroService). Cualquier otro error
39    * (sin conexión, 500, etc.) se deja propagar.
40    */
41   async obtener(id: string): Promise<RegistroAtencionPrehospitalaria | undefined> {
42     try {
43       return await firstValueFrom(this.http.get<RegistroAtencionPrehospitalaria>(`${API_REGISTROS_URL}/${id}`));
44     } catch (error) {
45       if (error instanceof HttpResponse && error.status === 404) return undefined;
46       throw error;
47     }
48   }
49
50   /**
51    * GET /api/registros - todos los registros, del más antiguo al más reciente
52    * (el backend ya los devuelve ordenados). Lo usan el Historial y el
53    * formulario al iniciar para recuperar el último borrador.
54    */
55   listar(): Promise<RegistroAtencionPrehospitalaria[]> {
56     return firstValueFrom(this.http.get<RegistroAtencionPrehospitalaria[]>(API_REGISTROS_URL));
57   }
58
59   /** DELETE /api/registros/:id - elimina el registro (botón "Eliminar" del Historial). */
60   async eliminar(id: string): Promise<void> {
61     await firstValueFrom(this.http.delete<void>(`${API_REGISTROS_URL}/${id}`));
62   }
63 }

```

```
1 import { LocalStorageRegistroService } from './registro-local-storage.service';
2 import { crearRegistroVacio } from '../models/registro.model';
3
4 describe('LocalStorageRegistroService', () => {
5   let servicio: LocalStorageRegistroService;
6
7   beforeEach(() => {
8     localStorage.clear();
9     servicio = new LocalStorageRegistroService();
10  });
11
12  it('guardar() persiste el registro y obtener() lo recupera por id', async () => {
13    const registro = crearRegistroVacio();
14    registro.folio = '56222';
15
16    await servicio.guardar(registro);
17    const recuperado = await servicio.obtener(registro.id);
18
19    expect(recuperado?.folio).toBe('56222');
20  });
21
22  it('listar() devuelve todos los registros guardados', async () => {
23    const uno = crearRegistroVacio();
24    const dos = crearRegistroVacio();
25    await servicio.guardar(uno);
26    await servicio.guardar(dos);
27
28    const todos = await servicio.listar();
29    expect(todos.map((r) => r.id).sort()).toEqual([uno.id, dos.id].sort());
30  });
31
32  it('eliminar() quita el registro y obtener() devuelve undefined después', async () => {
33    const registro = crearRegistroVacio();
34    await servicio.guardar(registro);
35
36    await servicio.eliminar(registro.id);
37    const recuperado = await servicio.obtener(registro.id);
38
39    expect(recuperado).toBeUndefined();
40  });
41
42  it('guardar() sobre un id existente actualiza en vez de duplicar', async () => {
43    const registro = crearRegistroVacio();
44    await servicio.guardar(registro);
45    registro.folio = 'actualizado';
46    await servicio.guardar(registro);
47
48    const todos = await servicio.listar();
49    expect(todos.length).toBe(1);
50    expect(todos[0].folio).toBe('actualizado');
51  });
52
53  it('listar() devuelve [] (no lanza) cuando localStorage contiene JSON inválido', async () => {
54    localStorage.setItem('umam.registros', '{esto no es JSON valido}');
55
56    const todos = await servicio.listar();
57
58    expect(todos).toEqual([]);
59  });
60  });
```

```
1 import { RegistroService } from './registro.service';
2 import { RegistroAtencionPrehospitalaria } from '../models/registro.model';
3
4 const CLAVE_ALMACENAMIENTO = 'umam.registros';
5
6 export class LocalStorageRegistroService implements RegistroService {
7   async guardar(registro: RegistroAtencionPrehospitalaria): Promise<RegistroAtencionPrehospitalaria> {
8     const registros = await this.leerTodos();
9     const indice = registros.findIndex(r => r.id === registro.id);
10    if (indice >= 0) {
11      registros[indice] = registro;
12    } else {
13      registros.push(registro);
14    }
15    localStorage.setItem(CLAVE_ALMACENAMIENTO, JSON.stringify(registros));
16    return registro;
17  }
18
19  async obtener(id: string): Promise<RegistroAtencionPrehospitalaria | undefined> {
20    const registros = await this.leerTodos();
21    return registros.find(r => r.id === id);
22  }
23
24  async listar(): Promise<RegistroAtencionPrehospitalaria[]> {
25    return this.leerTodos();
26  }
27
28  async eliminar(id: string): Promise<void> {
29    const registros = await this.leerTodos();
30    const restantes = registros.filter(r => r.id !== id);
31    localStorage.setItem(CLAVE_ALMACENAMIENTO, JSON.stringify(restantes));
32  }
33
34  private async leerTodos(): Promise<RegistroAtencionPrehospitalaria[]> {
35    const bruto = localStorage.getItem(CLAVE_ALMACENAMIENTO);
36    if (!bruto) return [];
37    try {
38      return JSON.parse(bruto);
39    } catch {
40      // localStorage corrupto (JSON inválido): lo tratamos como "sin registros
41      // guardados" en vez de dejar que listar() rechace la promesa – si no,
42      // ngOnInit() nunca llega a suscribir el autosave y el formulario deja de
43      // guardar en silencio para el resto de la sesión.
44      return [];
45    }
46  }
47 }
```

```
1 import { InjectionToken } from '@angular/core';
2 import { RegistroAtencionPrehospitalaria } from '../models/registro.model';
3
4 export interface RegistroService {
5   guardar(registro: RegistroAtencionPrehospitalaria): Promise<RegistroAtencionPrehospitalaria>;
6   obtener(id: string): Promise<RegistroAtencionPrehospitalaria | undefined>;
7   listar(): Promise<RegistroAtencionPrehospitalaria[]>;
8   eliminar(id: string): Promise<void>;
9 }
10
11 export const REGISTRO_SERVICE = new InjectionToken<RegistroService>('REGISTRO_SERVICE');
```

```

1 import { ExcelExportadorService } from './excel-exportador.service';
2 import { crearRegistroVacio } from '../../../core/models/registro.model';
3 import { SECCIONES } from '../../../core/data/secciones.data';
4
5 describe('ExcelExportadorService', () => {
6   const servicio = new ExcelExportadorService();
7
8   it('construirLibro() crea una hoja por cada sección', async () => {
9     const registro = crearRegistroVacio();
10    const libro = await servicio.construirLibro(registro as any);
11
12    const nombresHojas = libro.worksheets.map(h => h.name);
13    for (const seccion of SECCIONES) {
14      expect(nombresHojas).withContext(seccion.clave).toContain(seccion.titulo.slice(0, 31));
15    }
16  });
17
18  it('la hoja de una sección tiene el valor del campo en la fila correspondiente', async () => {
19    const registro = crearRegistroVacio();
20    (registro as any).control.operador = 'Juan Pérez';
21    const libro = await servicio.construirLibro(registro as any);
22
23    const hoja = libro.getWorksheet('III. Control');
24    const filaOperador = hoja!.getRows(1, hoja!.rowCount)!.find(f => f.getCell(1).text === 'Operador');
25    expect(filaOperador!.getCell(2).text).toBe('Juan Pérez');
26  });
27
28  it('el encabezado de cada hoja tiene relleno de color (mismo diseño que el documento)', async () => {
29    const registro = crearRegistroVacio();
30    const libro = await servicio.construirLibro(registro as any);
31    const hoja = libro.getWorksheet('III. Control');
32    const encabezado = hoja!.getRow(1);
33    expect(encabezado.getCell(1).fill).toBeTruthy();
34  });
35
36  it('un campo tipo checkbox (booleano) se imprime como "Sí"/"No", no como "true"/"false"', async () => {
37    const registro = crearRegistroVacio();
38    (registro as any).causaTraumatica.eyectado = true;
39    (registro as any).observaciones.ministerioPublicoNotificado = false;
40    const libro = await servicio.construirLibro(registro as any);
41
42    const hojaCausaTraumatica = libro.getWorksheet('V. Causa Traumática');
43    const filaEyectado = hojaCausaTraumatica!.getRows(1, hojaCausaTraumatica!.rowCount)!.find(f => f.getCell(1).text === 'Ey
    » ectado');
44    expect(filaEyectado!.getCell(2).text).toBe('Sí');
45
46    const hojaObservaciones = libro.getWorksheet('XIII. Observaciones');
47    const filaMp = hojaObservaciones!.getRows(1, hojaObservaciones!.rowCount)!.find(f => f.getCell(1).text === 'Ministerio P
    » úblico notificado');
48    expect(filaMp!.getCell(2).text).toBe('No');
49  });
50
51  it('un campo tipo "imagenes" se resume como cantidad adjunta en vez de volcar el base64', async () => {
52    const registro = crearRegistroVacio();
53    (registro as any).hospitalReceptor.imagenesEkgRxLaboratorios = ['data:image/png;base64,AAA', 'data:image/png;base64,BBB'
    » ];
54    const libro = await servicio.construirLibro(registro as any);
55
56    const hoja = libro.getWorksheet('XVI. Hospital Receptor');
57    const fila = hoja!.getRows(1, hoja!.rowCount)!.find(f => f.getCell(1).text === 'Imágenes de EKG, Rx y/o laboratorios');
58    expect(fila!.getCell(2).text).toContain('2 imagen(es) adjunta(s)');
59  });
60
61  it('un campo tipo "firma" se resume como "Firmado"/"Sin firmar" en vez de volcar el base64', async () => {
62    const registro = crearRegistroVacio();
63    (registro as any).hospitalReceptor.firmaQuienEntrega = 'data:image/png;base64,FIRMAENTREGA';
64    const libro = await servicio.construirLibro(registro as any);
65
66    const hoja = libro.getWorksheet('XVI. Hospital Receptor');
67    const filas = hoja!.getRows(1, hoja!.rowCount)!;
68    const filaFirmada = filas.find(f => f.getCell(1).text === 'Firma de quien entrega al paciente');
69    const filaSinFirmar = filas.find(f => f.getCell(1).text === 'Firma de persona que recibe al paciente');
70    expect(filaFirmada!.getCell(2).text).toBe('Firmado - ver PDF');
71    expect(filaSinFirmar!.getCell(2).text).toBe('Sin firmar');

```

```
72     });  
73   });
```

Desarrollado Ingeniero Mario Isai Moreno Duran
Número 6564121593

```

1 import { Injectable } from '@angular/core';
2 import * as ExcelJS from 'exceljs';
3 import { saveAs } from 'file-saver';
4 import { SECCIONES } from '../../../core/data/secciones.data';
5 import { RegistroAtencionPrehospitalaria } from '../../../core/models/registro.model';
6
7 const RELLENO_ENCABEZADO: ExcelJS.Fill = {
8   type: 'pattern',
9   pattern: 'solid',
10  fgColor: { argb: 'FF1892D3' },
11 };
12 const FUENTE_ENCABEZADO: Partial<ExcelJS.Font> = { color: { argb: 'FFFFFF' }, bold: true };
13
14 @Injectable({ providedIn: 'root' })
15 export class ExcelExportadorService {
16   async construirLibro(registro: RegistroAtencionPrehospitalaria): Promise<ExcelJS.Workbook> {
17     const libro = new ExcelJS.Workbook();
18     libro.creator = 'UMAM';
19     libro.created = new Date();
20
21     for (const seccion of SECCIONES) {
22       const hoja = libro.addWorksheet(seccion.titulo.slice(0, 31));
23       hoja.columns = [{ header: 'Campo', key: 'campo', width: 40 }, { header: 'Valor', key: 'valor', width: 40 }];
24       this.estilizarFilaEncabezado(hoja.getRow(1));
25
26       const valoresSeccion = (registro as unknown as Record<string, unknown>)[seccion.clave] as Record<string, unknown> | un
27       » defined;
28       for (const campo of seccion.campos) {
29         const valorCrudo = valoresSeccion?.[campo.clave];
30         // Las imágenes y las firmas dibujadas se guardan como data URIs base64:
31         // no tiene sentido volcarlas en una celda, así que solo dejamos
32         // constancia de si hay algo adjunto/firmado (el contenido se ve en el PDF).
33         const valor = campo.tipo === 'imagenes'
34         » ? (Array.isArray(valorCrudo) && valorCrudo.length > 0 ? `${valorCrudo.length} imagen(es) adjunta(s) – ver PDF` : '
35         » ')
36         : campo.tipo === 'firma'
37         ? (typeof valorCrudo === 'string' && valorCrudo ? 'Firmado – ver PDF' : 'Sin firmar')
38         : typeof valorCrudo === 'boolean'
39         ? (valorCrudo ? 'Sí' : 'No')
40         : Array.isArray(valorCrudo) ? valorCrudo.join(', ') : (valorCrudo ?? '');
41         hoja.addRow({ campo: campo.etiqueta, valor });
42       }
43
44       for (const tabla of seccion.tablas ?? []) {
45         hoja.addRow([]);
46         const filaTitulo = hoja.addRow([tabla.titulo]);
47         filaTitulo.font = { bold: true };
48         const filaColumnas = hoja.addRow(tabla.columnas.map(c => c.etiqueta));
49         this.estilizarFilaEncabezado(filaColumnas);
50         const filas = (registro as unknown as Record<string, unknown>)[tabla.clave] as Record<string, unknown>[] | undefined
51         » ?? [];
52         for (const fila of filas) {
53           hoja.addRow(tabla.columnas.map(c => fila[c.clave] ?? ''));
54         }
55       }
56     }
57     return libro;
58   }
59   async exportar(registro: RegistroAtencionPrehospitalaria): Promise<void> {
60     const libro = await this.construirLibro(registro);
61     const buffer = await libro.xlsx.writeBuffer();
62     const nombreArchivo = `umam-registro-${registro.folio || registro.id}.xlsx`;
63     saveAs(new Blob([buffer], { type: 'application/octet-stream' }), nombreArchivo);
64   }
65   private estilizarFilaEncabezado(fila: ExcelJS.Row): void {
66     fila.eachCell(celda => {
67       celda.fill = RELLENO_ENCABEZADO;
68       celda.font = FUENTE_ENCABEZADO;
69     });
70   }
71 }

```

```

1 import { ComponentFixture, TestBed } from '@angular/core/testing';
2 import { PdfVistaComponent } from './pdf-vista.component';
3
4 describe('PdfVistaComponent (réplica de la hoja física)', () => {
5   let fixture: ComponentFixture<PdfVistaComponent>;
6
7   function render(registro: Record<string, any> | undefined): HTMLElement {
8     // setInput dispara ngOnChanges (asignar la propiedad directamente no lo hace)
9     fixture.componentRef.setInput('registro', registro);
10    fixture.detectChanges();
11    return fixture.nativeElement as HTMLElement;
12  }
13
14  beforeEach(async () => {
15    await TestBed.configureTestingModule({
16      imports: [PdfVistaComponent],
17    }).compileComponents();
18    fixture = TestBed.createComponent(PdfVistaComponent);
19  });
20
21  it('renderiza el encabezado oficial con teléfonos, correo y el folio del registro', () => {
22    const el = render({ datosGenerales: { folio: '56222', estado: 'Chihuahua', ciudad: 'Cd. Juárez' } });
23    expect(el.textContent).toContain('REGISTRO DE ATENCIÓN PREHOSPITALARIA');
24    expect(el.textContent).toContain('(656) 625-9472');
25    expect(el.textContent).toContain('(656) 625-9473');
26    expect(el.textContent).toContain('ambulanciasumam@yahoo.com');
27    expect(el.querySelector('.folio-valor')!.textContent).toContain('56222');
28    expect(el.textContent).toContain('Chihuahua');
29    expect(el.textContent).toContain('Cd. Juárez');
30  });
31
32  it('renderiza las pestañas laterales de sección de la página 1 (frente)', () => {
33    const el = render({});
34    const pestanas = Array.from(el.querySelectorAll('.pagina-1 .tab-seccion')).map(p => p.textContent!.trim());
35    for (const esperada of [
36      'II DATOS DEL SERVICIO', 'III CONTROL', 'IV DATOS DEL PACIENTE', 'V CAUSA TRAUMÁTICA',
37      'VI CAUSA CLÍNICA', 'VII PARTO', 'VIII EVALUACIÓN INICIAL', 'IX EVALUACIÓN SECUNDARIA',
38      'X ANAMNESIS',
39    ]) {
40      expect(pestanas).toContain(esperada);
41    }
42    // XI Tratamiento se movió a la página 2 (reverso) para aprovechar el espacio
43    // que dejó libre Material Utilizado y no desbordar la página 1 a una hoja aparte.
44    expect(pestanas).not.toContain('XI TRATAMIENTO');
45  });
46
47  it('renderiza la tabla de cronometría con las 5 horas (sin hora de llamada)', () => {
48    const el = render({ datosServicio: { horaSalida: '10:15' } });
49    const cronometria = el.querySelector('.cronometria')!;
50    for (const columna of ['SALIDA', 'LLEGADA', 'TRASLADO', 'HOSPITAL', 'LIBERACIÓN']) {
51      expect(cronometria.textContent).toContain(columna);
52    }
53    expect(cronometria.textContent).not.toContain('LLAMADA');
54    expect(cronometria.textContent).toContain('10:15');
55  });
56
57  it('marca la casilla de una opción seleccionada y deja vacías las demás', () => {
58    const el = render({ datosServicio: { motivoAtencion: 'Enfermedad' } });
59    const casillas = Array.from(el.querySelectorAll('.motivo-atencion .opcion')) as HTMLElement[];
60    const enfermedad = casillas.find(c => c.textContent!.includes('ENFERMEDAD'))!;
61    const traumatismo = casillas.find(c => c.textContent!.includes('TRAUMATISMO'))!;
62    expect(enfermedad.querySelector('.casilla')!.classList).toContain('marcada');
63    expect(trumatismo.querySelector('.casilla')!.classList).not.toContain('marcada');
64  });
65
66  it('marca opciones de un checkbox-grupo (arreglo de valores)', () => {
67    const el = render({ evaluacionInicial: { presenciaPulsos: ['Radial'] } });
68    const opciones = Array.from(el.querySelectorAll('.presencia-pulsos .opcion')) as HTMLElement[];
69    const radial = opciones.find(c => c.textContent!.includes('RADIAL'))!;
70    const carotideo = opciones.find(c => c.textContent!.includes('CAROTÍDEO'))!;
71    expect(radial.querySelector('.casilla')!.classList).toContain('marcada');
72    expect(carotideo.querySelector('.casilla')!.classList).not.toContain('marcada');
73  });
74

```



```

75     it('la tabla de signos vitales siempre muestra al menos 3 filas (vacías si no hay datos)', () => {
76         const el = render({});
77         const filas = el.querySelectorAll('.signos-vitales tbody tr');
78         expect(filas.length).toBeGreaterThanOrEqual(3);
79     });
80
81     it('la tabla de signos vitales muestra los valores capturados (EtCO2 y hallazgos EKG)', () => {
82         const el = render({ signosVitales: [{ hora: '10:30', fr: '18', fc: '80', tas: '120', tad: '80', spo2: '97', temp: '36.5'
83         » , gluc: '90', etco2: '35', hallazgosEkg: 'Sinusal' }] });
84         const primeraFila = el.querySelector('.signos-vitales tbody tr')!;
85         expect(primeraFila.textContent).toContain('10:30');
86         expect(primeraFila.textContent).toContain('120');
87         expect(primeraFila.textContent).toContain('Sinusal');
88     });
89
90     it('renderiza sin errores cuando registro es undefined (hoja en blanco imprimible)', () => {
91         const el = render(undefined);
92         expect(el.querySelector('.pagina-1')).toBeTruthy();
93         expect(el.querySelector('.pagina-2')).toBeTruthy();
94         expect(el.querySelectorAll('.pagina').length).toBe(2);
95         expect(el.querySelectorAll('.casilla.marcada').length).toBe(0);
96     });
97
98     describe('página 2 (reverso)', () => {
99         it('renderiza el texto legal de la negativa y las firmas del paciente y testigo', () => {
100             const el = render({ traslado: { nombrePaciente: 'Juan Pérez', nombreTestigo: 'Ana López' } });
101             expect(el.textContent).toContain('NEGATIVA A RECIBIR ATENCIÓN / SER TRASLADADO');
102             expect(el.textContent).toContain('me niego a aceptar el (tratamiento) / (traslado)');
103             expect(el.textContent).toContain('Nombre y firma del paciente');
104             expect(el.textContent).toContain('Nombre y firma del testigo');
105             expect(el.textContent).toContain('Juan Pérez');
106             expect(el.textContent).toContain('Ana López');
107         });
108
109         it('renderiza la firma dibujada del paciente y del testigo en la negativa de traslado', () => {
110             const el = render({
111                 traslado: {
112                     nombrePaciente: 'Juan Pérez', firmaPaciente: 'data:image/png;base64,FIRMAPACIENTE',
113                     nombreTestigo: 'Ana López',
114                 },
115             });
116             const firmas = el.querySelectorAll('.firma .imagen-firma');
117             expect(firmas.length).toBe(1);
118             expect((firmas[0] as HTMLImageElement).src).toContain('data:image/png;base64,FIRMAPACIENTE');
119         });
120
121         it('la tabla de vehículos involucrados siempre tiene 4 filas', () => {
122             const el = render({ vehiculosInvolucrados: [{ tipoMarca: 'Nissan Tsuru', placas: 'ABC-123' }] });
123             const filas = el.querySelectorAll('.vehiculos tbody tr');
124             expect(filas.length).toBe(4);
125             expect(filas[0].textContent).toContain('Nissan Tsuru');
126             expect(filas[0].textContent).toContain('ABC-123');
127             expect(el.querySelector('.vehiculos')!.textContent).toContain('TIPO Y MARCA');
128             expect(el.querySelector('.vehiculos')!.textContent).toContain('PLACAS');
129         });
130
131         it('renderiza las imágenes adjuntas de EKG/Rx/laboratorios en hospital receptor', () => {
132             const el = render({ hospitalReceptor: { imagenesEkgRxLaboratorios: ['data:image/png;base64,AAA', 'data:image/png;base6
133             » 4,BBB'] } });
134             const imagenes = el.querySelectorAll('.imagenes-adjuntas img');
135             expect(imagenes.length).toBe(2);
136             expect((imagenes[0] as HTMLImageElement).src).toContain('data:image/png;base64,AAA');
137         });
138
139         it('renderiza la firma dibujada de quien entrega/recibe cuando existe, y no un <img> vacío cuando no se firmó', () => {
140             const el = render({
141                 hospitalReceptor: {
142                     nombreQuienEntrega: 'TUM Pedro', firmaQuienEntrega: 'data:image/png;base64,FIRMAENTREGA',
143                     nombreQuienRecibe: 'Enf. Ana',
144                 },
145             });
146             const firmas = el.querySelectorAll('.firma .imagen-firma');
147             expect(firmas.length).toBe(1);
148             expect((firmas[0] as HTMLImageElement).src).toContain('data:image/png;base64,FIRMAENTREGA');

```

```

147     });
148
149     it('renderiza el pie con la dirección y los teléfonos', () => {
150         const el = render({});
151         const pie = el.querySelector('.pie')!;
152         expect(pie.textContent).toContain('Calle Quinta Amalia # 107');
153         expect(pie.textContent).toContain('Fracc. Las Quintas');
154         expect(pie.textContent).toContain('C. P. 32401');
155         expect(pie.textContent).toContain('(656)625-9472');
156     });
157
158     it('renderiza las secciones XI a XVII (tratamiento se movió aquí, y consentimiento informado)', () => {
159         const el = render({ hospitalReceptor: { nombreQuienEntrega: 'TUM Pedro' } });
160         const pestanas = Array.from(el.querySelectorAll('.pagina-2 .tab-seccion')).map(p => p.textContent!.trim());
161         for (const esperada of [
162             'XI TRATAMIENTO', 'XII TRASLADO', 'XIII OBSERVACIONES', 'XV DATOS LEGALES',
163             'XVI HOSPITAL RECEPTOR', 'XVII CONSENTIMIENTO INFORMADO',
164         ]) {
165             expect(pestanas).toContain(esperada);
166         }
167         expect(el.textContent).toContain('ACEPTACIÓN DE HOSPITAL RECEPTOR');
168         expect(el.textContent).toContain('TUM Pedro');
169     });
170
171     it('renderiza el texto de consentimiento informado y las tres firmas', () => {
172         const el = render({
173             consentimientoInformado: {
174                 nombrePacienteConsentimiento: 'Juan Pérez',
175                 nombreResponsableConsentimiento: 'María Pérez',
176                 nombreParamedicoConsentimiento: 'TUM Pedro',
177             },
178         });
179         const pagina2 = el.querySelector('.pagina-2')!;
180         expect(pagina2.textContent).toContain('CONSENTIMIENTO INFORMADO');
181         expect(pagina2.textContent).toContain('otorgo mi consentimiento');
182         expect(pagina2.textContent).toContain('Juan Pérez');
183         expect(pagina2.textContent).toContain('María Pérez');
184         expect(pagina2.textContent).toContain('TUM Pedro');
185         expect(pagina2.textContent).toContain('NOMBRE Y FIRMA DEL PACIENTE');
186         expect(pagina2.textContent).toContain('NOMBRE Y FIRMA DEL FAMILIAR O RESPONSABLE');
187         expect(pagina2.textContent).toContain('NOMBRE Y FIRMA DEL PARAMÉDICO');
188     });
189
190     it('renderiza las firmas dibujadas del paciente, responsable y paramédico en consentimiento informado', () => {
191         const el = render({
192             consentimientoInformado: {
193                 firmaPacienteConsentimiento: 'data:image/png;base64,FIRMAPACIENTE',
194                 firmaResponsableConsentimiento: 'data:image/png;base64,FIRMARESPONSABLE',
195                 firmaParamedicoConsentimiento: 'data:image/png;base64,FIRMAPARAMEDICO',
196             },
197         });
198         const pagina2 = el.querySelector('.pagina-2')!;
199         const firmas = Array.from(pagina2.querySelectorAll('.firma .imagen-firma')) as HTMLImageElement[];
200         expect(firmas.length).toBe(3);
201         expect(firmas.map(f => f.src)).toEqual([
202             jasmine.stringContaining('FIRMAPACIENTE'),
203             jasmine.stringContaining('FIRMARESPONSABLE'),
204             jasmine.stringContaining('FIRMAPARAMEDICO'),
205         ]);
206     });
207 });
208 });

```

```

1 import { Component, Input, OnChanges } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { SECCIONES } from '../../../core/data/secciones.data';
4
5 /**
6  * Réplica imprimible de la hoja física oficial de UMAM (Registro de Atención
7  * Prehospitalaria, 2 páginas carta). El template está pensado para que un
8  * registro vacío imprima la hoja en blanco, igual que el papel.
9  */
10 @Component({
11   selector: 'app-pdf-vista',
12   standalone: true,
13   imports: [CommonModule],
14   template: `
15 <div class="hoja">
16
17   <!-- ===== PÁGINA 1 (FRENTE) ===== -->
18   <div class="pagina pagina-1">
19
20     <header class="encabezado">
21       <div class="logo">
22         <svg viewBox="0 0 40 40" class="logo-svg" aria-hidden="true">
23           <circle cx="20" cy="20" r="18" fill="none" stroke="#0d3a5c" stroke-width="3"/>
24           <path d="M20 6 L23 15 L32 12 L26 20 L32 28 L23 25 L20 34 L17 25 L8 28 L14 20 L8 12 L17 15 Z" fill="#0d3a5c"/>
25           <path d="M20 11 L20 29 M15 14 C 25 17, 15 23, 25 26" stroke="#fff" stroke-width="1.6" fill="none"/>
26         </svg>
27         <div class="logo-texto">
28           <span class="logo-umam">UMAM</span>
29           <span class="logo-sub">UNIDAD MÓVIL DE<br />ATENCIÓN MÉDICA</span>
30         </div>
31       </div>
32       <div class="titulo-principal">
33         <h1>REGISTRO DE ATENCIÓN PREHOSPITALARIA</h1>
34         <div class="telefonos">(656) 625-9472</div>
35         <div class="telefonos">(656) 625-9473</div>
36         <div class="telefonos">ambulanciasumam&#64;yahoo.com</div>
37       </div>
38       <div class="folio">
39         <span class="folio-etiqueta">FOLIO:</span>
40         <span class="folio-valor">{{ v('datosGenerales', 'folio') }}</span>
41       </div>
42     </header>
43
44     <div class="renglon-estado">
45       <span class="titulo-campo">ESTADO:</span>
46       <span class="linea">{{ v('datosGenerales', 'estado') }}</span>
47       <span class="titulo-campo">CIUDAD:</span>
48       <span class="linea">{{ v('datosGenerales', 'ciudad') }}</span>
49     </div>
50
51     <div class="columnas">
52       <!-- ===== COLUMNA IZQUIERDA ===== -->
53       <div class="columna">
54
55         <!-- II. DATOS DEL SERVICIO -->
56         <div class="seccion">
57           <div class="tab-seccion">II DATOS DEL SERVICIO</div>
58           <div class="cuerpo-seccion">
59             <div class="fila-lineas">
60               <span class="titulo-campo">FECHA:</span><span class="linea">{{ v('datosServicio', 'fecha') }}</span>
61               <span class="titulo-campo">DÍA DE LA SEMANA:</span><span class="linea">{{ v('datosServicio', 'diaSemana') }}</span>
62             </div>
63             <table class="tabla-mini cronometria">
64               <caption>CRONOMETRÍA</caption>
65               <thead>
66                 <tr>
67                   <th>HORA DE SALIDA</th><th>HORA DE LLEGADA</th>
68                   <th>HORA DE TRASLADO</th><th>HORA DE HOSPITAL</th><th>HORA DE LIBERACIÓN</th>
69                 </tr>
70               </thead>
71               <tbody>
72                 <tr>
73                   <td>{{ v('datosServicio', 'horaSalida') }}</td>

```

```

74         <td>{{ v('datosServicio', 'horaLlegada') }}</td>
75         <td>{{ v('datosServicio', 'horaTraslado') }}</td>
76         <td>{{ v('datosServicio', 'horaHospital') }}</td>
77         <td>{{ v('datosServicio', 'horaLiberacion') }}</td>
78     </tr>
79 </tbody>
80 </table>
81 <div class="grupo-opciones motivo-atencion">
82     <span class="titulo-campo">MOTIVO DE LA ATENCIÓN:</span>
83     <span class="opcion" *ngFor="let o of ops('datosServicio', 'motivoAtencion')">
84         <span class="casilla" [class.marcada]="marcado('datosServicio', 'motivoAtencion', o)"></span>{{ o | uppercas
85     e }}
86     </span>
87 </div>
88 <div class="fila-lineas"><span class="titulo-campo">COLONIA / COMUNIDAD:</span><span class="linea">{{ v('datosSe
89     rvicio', 'colonia') }}</span></div>
90 <div class="fila-lineas"><span class="titulo-campo">DELEGACIÓN POLÍTICA / MUNICIPIO:</span><span class="linea">{
91     { v('datosServicio', 'delegacionMunicipio') }}</span></div>
92 <div class="grupo-opciones">
93     <span class="titulo-campo">LUGAR DE LA OCURRENCIA:</span>
94     <span class="opcion" *ngFor="let o of ops('datosServicio', 'lugarOcurriencia')">
95         <span class="casilla" [class.marcada]="marcado('datosServicio', 'lugarOcurriencia', o)"></span>{{ o | upperca
96     se }}
97     </span>
98     <span class="linea">{{ v('datosServicio', 'lugarOcurrienciaOtro') }}</span>
99 </div>
100 <div class="fila-lineas"><span class="titulo-campo">RESPONSABLE DE ENTREGA DEL PACIENTE:</span><span class="line
101     a">{{ v('datosServicio', 'responsableEntregaPaciente') }}</span></div>
102 </div>
103 </div>
104 <!-- III. CONTROL -->
105 <div class="seccion">
106     <div class="tab-seccion">III CONTROL</div>
107     <div class="cuerpo-seccion">
108         <div class="fila-lineas">
109             <span class="titulo-campo">NÚMERO DE AMBULANCIA:</span><span class="linea">{{ v('control', 'ambulanciaNumero')
110         }}</span>
111         </div>
112         <div class="fila-lineas"><span class="titulo-campo">OPERADOR:</span><span class="linea">{{ v('control', 'operado
113         r') }}</span></div>
114         <div class="fila-lineas"><span class="titulo-campo">PRESTADORES DEL SERVICIO:</span><span class="linea">{{ v('co
115         ntrol', 'prestadoresServicio') }}</span></div>
116     </div>
117 </div>
118 <!-- IV. DATOS DEL PACIENTE -->
119 <div class="seccion">
120     <div class="tab-seccion">IV DATOS DEL PACIENTE</div>
121     <div class="cuerpo-seccion">
122         <div class="fila-lineas"><span class="titulo-campo">NOMBRE O MEDIA FILIACIÓN:</span><span class="linea">{{ v('da
123         tosPaciente', 'nombreOMediaFiliacion') }}</span></div>
124         <div class="fila-lineas">
125             <span class="titulo-campo">SEXO:</span><span class="linea">{{ v('datosPaciente', 'sexo') }}</span>
126             <span class="titulo-campo">EDAD:</span><span class="linea corta">{{ v('datosPaciente', 'edadAnios') }}</span><
127             span>AÑOS</span>
128             <span class="titulo-campo">MENORES DE 1 AÑO:</span><span class="linea corta">{{ v('datosPaciente', 'edadMeses'
129             ) }}</span><span>MESES</span>
130         </div>
131         <div class="fila-lineas">
132             <span class="titulo-campo">FECHA DE NACIMIENTO:</span><span class="linea">{{ v('datosPaciente', 'fechaNacimien
133             to') }}</span>
134         </div>
135         <div class="fila-lineas"><span class="titulo-campo">COLONIA / COMUNIDAD:</span><span class="linea">{{ v('datosPa
136             ciente', 'colonia') }}</span></div>
137         <div class="fila-lineas"><span class="titulo-campo">DELEGACIÓN POLÍTICA / MUNICIPIO:</span><span class="linea">{
138             { v('datosPaciente', 'delegacionMunicipio') }}</span></div>
139         <div class="fila-lineas">
140             <span class="titulo-campo">TELÉFONO:</span><span class="linea">{{ v('datosPaciente', 'telefono') }}</span>
141             <span class="titulo-campo">OCUPACIÓN:</span><span class="linea">{{ v('datosPaciente', 'ocupacion') }}</span>
142         </div>
143         <div class="fila-lineas"><span class="titulo-campo">NÚMERO DE EMPLEADO:</span><span class="linea">{{ v('datosPac
144             iente', 'numeroEmpleado') }}</span></div>
145         <div class="fila-lineas"><span class="titulo-campo">TIPO DE SERVICIO MÉDICO:</span><span class="linea">{{ v('dat

```

```

    » osPaciente', 'tipoServicioMedico') }}</span></div>
133     </div>
134   </div>
135
136   <!-- V. CAUSA TRAUMÁTICA -->
137   <div class="seccion">
138     <div class="tab-seccion">V CAUSA TRAUMÁTICA</div>
139     <div class="cuerpo-seccion">
140       <div class="titulo-campo">AGENTE CAUSAL</div>
141       <div class="rejilla-opciones tres">
142         <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'agenteCausal')">
143           <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'agenteCausal', o)"></span>{{ o | uppercase
144         e }}
145       </span>
146     </div>
147     <div class="fila-lineas"><span class="titulo-campo">ESPECIFIQUE:</span><span class="linea">{{ v('causaTraumatica
148     ', 'agenteCausalEspecifique') }}</span></div>
149     <div class="fila-lineas"><span class="titulo-campo">LESIONES CAUSADAS POR:</span><span class="linea">{{ v('causa
150     Traumatica', 'lesionesCausadasPor') }}</span></div>
151
152     <div class="subtitulo">ACCIDENTE AUTOMOVILÍSTICO</div>
153     <div class="rejilla-opciones tres">
154       <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'accidenteAutomovilistico')">
155         <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'accidenteAutomovilistico', o)"></span>{{
156         o | uppercase }}
157       </span>
158     </div>
159
160     <div class="subtitulo">SOBRE LA COLISIÓN</div>
161     <div class="grupo-opciones">
162       <span class="titulo-campo">CONTRA OBJETO:</span>
163       <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'contraObjeto')">
164         <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'contraObjeto', o)"></span>{{ o | uppercase
165       e }}
166     </span>
167     <span class="titulo-campo">IMPACTO:</span>
168     <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'impacto')">
169       <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'impacto', o)"></span>{{ o | uppercase }}
170     </span>
171   </div>
172   <div class="grupo-opciones">
173     <span class="titulo-campo">HUNDIMIENTO:</span><span class="linea corta">{{ v('causaTraumatica', 'hundimientoCm
174     s') }}</span><span>CMS</span>
175     <span class="titulo-campo">PARABRISAS:</span>
176     <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'parabrisas')">
177       <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'parabrisas', o)"></span>{{ o | uppercase
178     }}
179   </span>
180   <span class="titulo-campo">VOLANTE:</span>
181   <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'volante')">
182     <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'volante', o)"></span>{{ o | uppercase }}
183   </span>
184 </div>
185 <div class="grupo-opciones">
186   <span class="titulo-campo">BOLSAS DE AIRE:</span>
187   <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'bolsasAire')">
188     <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'bolsasAire', o)"></span>{{ o | uppercase
189   }}
190 </span>
191 <span class="titulo-campo">CINTURÓN DE SEGURIDAD:</span>
192 <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'cinturonSeguridad')">
193   <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'cinturonSeguridad', o)"></span>{{ o | upp
194   ercase }}
195 </span>
196 </div>
197 <div class="grupo-opciones">
198   <span class="titulo-campo">DENTRO DEL VEHÍCULO:</span>
199   <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'dentroVehiculo')">
200     <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'dentroVehiculo', o)"></span>{{ o | upperc
201   ase }}
202 </span>
203 <span class="opcion">
204   <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'eyectado')"></span>EYECTADO
205 </span>

```

```

196         </div>
197
198         <div class="subtitulo">ATROPELLADO</div>
199         <div class="rejilla-opciones cuatro">
200             <span class="opcion" *ngFor="let o of ops('causaTraumatica', 'atropellado')">
201                 <span class="casilla" [class.marcada]="marcado('causaTraumatica', 'atropellado', o)"></span>{{ o | uppercase
202             }}
203         </span>
204         </div>
205     </div>
206
207     <!-- VI. CAUSA CLÍNICA -->
208     <div class="seccion">
209         <div class="tab-seccion">VI CAUSA CLÍNICA</div>
210         <div class="cuerpo-seccion">
211             <div class="titulo-campo">ORIGEN PROBABLE:</div>
212             <div class="rejilla-opciones tres">
213                 <span class="opcion" *ngFor="let o of ops('causaClinica', 'origenProbable')">
214                     <span class="casilla" [class.marcada]="marcado('causaClinica', 'origenProbable', o)"></span>{{ o | uppercase
215                 }}
216             </span>
217             </div>
218             <div class="fila-lineas"><span class="titulo-campo">ESPECIFIQUE:</span><span class="linea">{{ v('causaClinica',
219             'origenProbableEspecifique') }}</span></div>
220             <div class="grupo-opciones">
221                 <span class="titulo-campo">1ERA VEZ:</span>
222                 <span class="opcion" *ngFor="let o of ops('causaClinica', 'primeraVez')">
223                     <span class="casilla" [class.marcada]="marcado('causaClinica', 'primeraVez', o)"></span>{{ o | uppercase }}
224                 </span>
225                 <span class="titulo-campo">SUBSECUENTE:</span>
226                 <span class="linea">{{ v('causaClinica', 'subsecuente') }}</span>
227             </div>
228         </div>
229     </div>
230
231     <!-- VII. PARTO -->
232     <div class="seccion">
233         <div class="tab-seccion">VII PARTO</div>
234         <div class="cuerpo-seccion">
235             <div class="subtitulo">DATOS DE LA MADRE</div>
236             <div class="fila-lineas">
237                 <span class="titulo-campo">GESTA:</span><span class="linea corta">{{ v('parto', 'gesta') }}</span>
238                 <span class="titulo-campo">CESÁREA:</span><span class="linea corta">{{ v('parto', 'cesarea') }}</span>
239                 <span class="titulo-campo">PARA:</span><span class="linea corta">{{ v('parto', 'para') }}</span>
240                 <span class="titulo-campo">ABORTOS:</span><span class="linea corta">{{ v('parto', 'abortos') }}</span>
241                 <span class="titulo-campo">SEMANA DE GESTACIÓN:</span><span class="linea corta">{{ v('parto', 'semanaGestacion
242             ')}</span>
243             </div>
244             <div class="fila-lineas">
245                 <span class="titulo-campo">FECHA PROBABLE DE PARTO:</span><span class="linea">{{ v('parto', 'fechaProbablePart
246             o') }}</span>
247                 <span class="titulo-campo">MEMBRANAS:</span><span class="linea">{{ v('parto', 'membranas') }}</span>
248             </div>
249             <div class="fila-lineas">
250                 <span class="titulo-campo">HORA DE INICIO DE CONTRACCIONES:</span><span class="linea corta">{{ v('parto', 'hor
251             aInicioContracciones') }}</span>
252                 <span class="titulo-campo">FRECUENCIA:</span><span class="linea corta">{{ v('parto', 'frecuenciaContracciones'
253             ) }}</span>
254                 <span class="titulo-campo">DURACIÓN:</span><span class="linea corta">{{ v('parto', 'duracionContracciones') }}
255             </span>
256             </div>
257             <div class="subtitulo">DATOS POST-PARTO</div>
258             <div class="grupo-opciones">
259                 <span class="titulo-campo">HORA DE NACIMIENTO:</span><span class="linea corta">{{ v('parto', 'horaNacimiento')
260             }}</span>
261                 <span class="titulo-campo">LUGAR:</span><span class="linea">{{ v('parto', 'lugarNacimientoParto') }}</span>
262                 <span class="titulo-campo">PLACENTA EXPULSADA:</span>
263                 <span class="opcion" *ngFor="let o of ops('parto', 'placentaExpulsada')">
264                     <span class="casilla" [class.marcada]="marcado('parto', 'placentaExpulsada', o)"></span>{{ o | uppercase }}
265                 </span>
266             </div>
267             <div class="subtitulo">DATOS DEL RECIÉN NACIDO</div>
268             <div class="grupo-opciones">

```



```

261         <span class="titulo-campo">PRODUCTO:</span>
262         <span class="opcion" *ngFor="let o of ops('parto', 'productoVivoMuerto')">
263             <span class="casilla" [class.marcada]="marcado('parto', 'productoVivoMuerto', o)"></span>{{ o | uppercase }}
264         </span>
265         <span class="opcion" *ngFor="let o of ops('parto', 'sexoRecienNacido')">
266             <span class="casilla" [class.marcada]="marcado('parto', 'sexoRecienNacido', o)"></span>{{ o === 'Masculino'
267         ? 'MASC' : 'FEM' }}
268     </span>
269 </div>
270 <div class="fila-lineas">
271     <span class="titulo-campo">APGAR – 1 MIN:</span><span class="linea corta">{{ v('parto', 'apgar1min') }}</span>
272     <span class="titulo-campo">5 MIN:</span><span class="linea corta">{{ v('parto', 'apgar5min') }}</span>
273     <span class="titulo-campo">10 MIN:</span><span class="linea corta">{{ v('parto', 'apgar10min') }}</span>
274     <span class="titulo-campo">SILVERMAN – 1 MIN:</span><span class="linea corta">{{ v('parto', 'silverman1min') }}
275 </span>
276     <span class="titulo-campo">5 MIN:</span><span class="linea corta">{{ v('parto', 'silverman5min') }}</span>
277 </div>
278 </div>
279
280 <!-- ===== COLUMNA DERECHA ===== -->
281 <div class="columna">
282
283     <!-- VIII. EVALUACIÓN INICIAL -->
284     <div class="seccion">
285         <div class="tab-seccion">VIII EVALUACIÓN INICIAL</div>
286         <div class="cuerpo-seccion">
287             <div class="rejilla-bloques tres">
288                 <div class="bloque">
289                     <div class="titulo-campo">NIVEL DE CONCIENCIA:</div>
290                     <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'nivelConciencia')">
291                         <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'nivelConciencia', o)"></span>{{ o | u
292 ppercase }}
293                     </div>
294                 </div>
295                 <div class="bloque">
296                     <div class="titulo-campo">VÍA AÉREA:</div>
297                     <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'viaAerea')">
298                         <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'viaAerea', o)"></span>{{ o | uppercas
299 e }}
300                     </div>
301                 </div>
302                 <div class="bloque">
303                     <div class="titulo-campo">REFLEJO DE DEGLUCIÓN:</div>
304                     <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'reflejoDeglucion')">
305                         <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'reflejoDeglucion', o)"></span>{{ o |
306 uppercase }}
307                     </div>
308                 </div>
309             </div>
310             <div class="titulo-campo banda">VENTILACIÓN:</div>
311             <div class="rejilla-bloques tres">
312                 <div class="bloque">
313                     <div class="titulo-campo">OBSERVACIÓN</div>
314                     <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'ventilacionObservacion')">
315                         <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'ventilacionObservacion', o)"></span>{
316 { o | uppercase }}
317                     </div>
318                 </div>
319                 <div class="bloque">
320                     <div class="titulo-campo">AUSCULTACIÓN</div>
321                     <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'ventilacionAuscultacion')">
322                         <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'ventilacionAuscultacion', o)"></span>
323 { o | uppercase }}
324                     </div>
325                 </div>
326             </div>
327             <div class="fila-lineas"><span class="titulo-campo">ESPECIFIQUE:</span><span class="linea">{{ v('evaluacionI
328 nicial', 'ventilacionAuscultacionEspecifique') }}</span></div>
329             <div class="bloque">
330                 <div class="titulo-campo">ALTERNACIONES LOCALIZADAS EN:</div>
331                 <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'alternacionesLocalizadas')">
332                     <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'alternacionesLocalizadas', o)"></span>

```

```

    » >{{ o | uppercase }}
327         </div>
328     </div>
329 </div>
330
331     <div class="titulo-campo banda">CIRCULACIÓN:</div>
332     <div class="rejilla-bloques cuatro">
333         <div class="bloque presencia-pulsos">
334             <div class="titulo-campo">PRESENCIA DE PULSOS</div>
335             <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'presenciaPulsos')">
336                 <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'presenciaPulsos', o)"></span>{{ o | u
    » ppercase }}
337             </div>
338             <div class="opcion">
339                 <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'paroCardiorespiratorio')"></span>PARO
    » CARDIORESPIRATORIO
340             </div>
341         </div>
342         <div class="bloque">
343             <div class="titulo-campo">CALIDAD</div>
344             <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'calidadPulso')">
345                 <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'calidadPulso', o)"></span>{{ o | uppe
    » rcase }}
346             </div>
347         </div>
348         <div class="bloque">
349             <div class="titulo-campo">PIEL</div>
350             <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'piel')">
351                 <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'piel', o)"></span>{{ o | uppercase }}
352             </div>
353         </div>
354         <div class="bloque">
355             <div class="titulo-campo">CARACTERÍSTICAS</div>
356             <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'caracteristicasPiel')">
357                 <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'caracteristicasPiel', o)"></span>{{ o
    » | uppercase }}
358             </div>
359             <div class="titulo-campo">TEMPERATURA COMPARATIVA</div>
360             <div class="opcion" *ngFor="let o of ops('evaluacionInicial', 'temperaturaComparativa')">
361                 <span class="casilla" [class.marcada]="marcado('evaluacionInicial', 'temperaturaComparativa', o)"></span>{
    » { o | uppercase }}
362             </div>
363         </div>
364     </div>
365 </div>
366 </div>
367
368 <!-- IX. EVALUACIÓN SECUNDARIA -->
369 <div class="seccion">
370     <div class="tab-seccion">IX EVALUACIÓN SECUNDARIA</div>
371     <div class="cuerpo-seccion">
372         <div class="rejilla-bloques exploracion">
373             <div class="bloque">
374                 <div class="titulo-campo">EXPLORACIÓN FÍSICA</div>
375                 <table class="tabla-exploracion">
376                     <tr *ngFor="let item of exploracionFisica">
377                         <td class="codigo">{{ item.codigo }}</td>
378                         <td>{{ item.nombre | uppercase }}</td>
379                         <td><span class="casilla" [class.marcada]="marcado('evaluacionSecundaria', 'exploracionFisica', item.nom
    » bre)"></span></td>
380                     </tr>
381                 </table>
382             </div>
383             <div class="bloque bloque-silueta">
384                 <div class="titulo-campo">ZONAS DE LESIÓN</div>
385                 <div class="siluetas">
386                     <svg viewBox="0 0 60 130" class="silueta" aria-hidden="true">
387                         <circle cx="30" cy="10" r="8" />
388                         <path d="M30 18 C 22 20, 20 24, 19 32 L 15 62 L 20 63 L 24 40 L 24 66 L 21 122 L 28 122 L 30 80 L 32 122
    » L 39 122 L 36 66 L 36 40 L 40 63 L 45 62 L 41 32 C 40 24, 38 20, 30 18 Z" />
389                     </svg>
390                     <svg viewBox="0 0 60 130" class="silueta" aria-hidden="true">
391                         <circle cx="30" cy="10" r="8" />
392                         <path d="M30 18 C 22 20, 20 24, 19 32 L 15 62 L 20 63 L 24 40 L 24 66 L 21 122 L 28 122 L 30 80 L 32 122

```



```

    » L 39 122 L 36 66 L 36 40 L 40 63 L 45 62 L 41 32 C 40 24, 38 20, 30 18 Z" />
393     </svg>
394   </div>
395   <div class="rejilla-opciones dos zonas-lesion">
396     <span class="opcion" *ngFor="let o of ops('evaluacionSecundaria', 'zonasLesion')">
397       <span class="casilla" [class.marcada]="marcado('evaluacionSecundaria', 'zonasLesion', o)"></span>{{ o |
    » uppercase }}
398     </span>
399   </div>
400   <div class="fila-lineas"><span class="titulo-campo">ESPECIFIQUE:</span><span class="linea">{{ v('evaluacionS
    » ecundaria', 'zonasLesionEspecifique') }}</span></div>
401   <div class="titulo-campo">PUPILAS</div>
402   <div class="pupilas">
403     <span class="opcion" *ngFor="let o of ops('evaluacionSecundaria', 'pupilas')">
404       <span class="ojo" aria-hidden="true"></span>
405       <span class="casilla" [class.marcada]="marcado('evaluacionSecundaria', 'pupilas', o)"></span>{{ o | uppe
    » rcase }}
406     </span>
407   </div>
408 </div>
409 </div>
410
411 <table class="tabla-mini signos-vitales">
412   <caption>SIGNOS VITALES Y MONITOREO</caption>
413   <thead>
414     <tr>
415       <th>HORA</th><th>FR</th><th>FC</th><th>TAS</th><th>TAD</th><th>SpO2</th>
416       <th>TEMP</th><th>GLUC</th><th>ETCO2</th><th>HALLAZGOS EKG</th>
417     </tr>
418   </thead>
419   <tbody>
420     <tr *ngFor="let fila of filasSignos">
421       <td>{{ fila['hora'] }}</td><td>{{ fila['fr'] }}</td><td>{{ fila['fc'] }}</td>
422       <td>{{ fila['tas'] }}</td><td>{{ fila['tad'] }}</td><td>{{ fila['spo2'] }}</td>
423       <td>{{ fila['temp'] }}</td><td>{{ fila['gluc'] }}</td><td>{{ fila['etco2'] }}</td>
424       <td>{{ fila['hallazgosEkg'] }}</td>
425     </tr>
426   </tbody>
427 </table>
428 </div>
429 </div>
430
431 <!-- X. ANAMNESIS -->
432 <div class="seccion">
433   <div class="tab-seccion">X ANAMNESIS</div>
434   <div class="cuerpo-seccion">
435     <div class="grupo-opciones">
436       <span class="titulo-campo">INTERROGATORIO:</span>
437       <span class="opcion" *ngFor="let o of ops('anamnesis', 'interrogatorio')">
438         <span class="casilla" [class.marcada]="marcado('anamnesis', 'interrogatorio', o)"></span>{{ o | uppercase }}
439       </span>
440       <span class="titulo-campo">PESO:</span><span class="linea corta">{{ v('anamnesis', 'peso') }}</span>
441       <span class="titulo-campo">TALLA:</span><span class="linea corta">{{ v('anamnesis', 'talla') }}</span>
442     </div>
443     <div class="fila-lineas"><span class="titulo-campo">ALERGIAS:</span><span class="linea">{{ v('anamnesis', 'alerg
    » ias') }}</span></div>
444     <div class="fila-lineas"><span class="titulo-campo">MEDICAMENTOS:</span><span class="linea">{{ v('anamnesis', 'm
    » edicamentos') }}</span></div>
445     <div class="fila-lineas"><span class="titulo-campo">ENFERMEDADES Y/O CIRUGÍAS PREVIAS:</span><span class="linea"
    » >{{ v('anamnesis', 'enfermedadesCirugiasPrevias') }}</span></div>
446     <div class="fila-lineas">
447       <span class="titulo-campo">HORA DE ÚLTIMA COMIDA:</span><span class="linea corta">{{ v('anamnesis', 'horaUltim
    » aComida') }}</span>
448       <span class="titulo-campo">EVENTOS PREVIOS RELACIONADOS:</span><span class="linea">{{ v('anamnesis', 'eventosP
    » reviosRelacionados') }}</span>
449     </div>
450     <div class="rejilla-bloques cuatro">
451       <div class="bloque">
452         <div class="titulo-campo">CONDICIÓN DEL PACIENTE</div>
453         <div class="opcion" *ngFor="let o of ops('anamnesis', 'condicionPaciente')">
454           <span class="casilla" [class.marcada]="marcado('anamnesis', 'condicionPaciente', o)"></span>{{ o | upperca
    » se }}
455         </div>
456         <div class="opcion" *ngFor="let o of ops('anamnesis', 'estabilidadPaciente')">

```

```

457         <span class="casilla" [class.marcada]="marcado('anamnesis', 'estabilidadPaciente', o)"></span>{{ o | upper
» case }}
458     </div>
459 </div>
460 <div class="bloque">
461     <div class="titulo-campo">PRIORIDAD</div>
462     <div class="opcion" *ngFor="let o of ops('anamnesis', 'prioridad')">
463         <span class="casilla" [class.marcada]="marcado('anamnesis', 'prioridad', o)"></span>{{ o | uppercase }}
464     </div>
465 </div>
466 <div class="bloque">
467     <div class="titulo-campo">GLASGOW</div>
468     <div class="fila-lineas"><span class="titulo-campo">OCULAR:</span><span class="linea corta">{{ v('anamnesis'
» , 'glasgowOcular') }}</span></div>
469     <div class="fila-lineas"><span class="titulo-campo">VERBAL:</span><span class="linea corta">{{ v('anamnesis'
» , 'glasgowVerbal') }}</span></div>
470     <div class="fila-lineas"><span class="titulo-campo">MOTORA:</span><span class="linea corta">{{ v('anamnesis'
» , 'glasgowMotora') }}</span></div>
471     <div class="fila-lineas"><span class="titulo-campo">TOTAL:</span><span class="linea corta">{{ v('anamnesis',
» 'glasgowTotal') }}</span></div>
472 </div>
473 <div class="bloque">
474     <div class="titulo-campo">NEWS2</div>
475     <div class="fila-lineas"><span class="titulo-campo">PUNTAJE TOTAL:</span><span class="linea corta">{{ v('ana
» mnesis', 'news2Total') }}</span></div>
476 </div>
477 </div>
478 </div>
479 </div>
480 </div>
481 </div>
482 </div>
483
484 <div class="salto"></div>
485
486 <!-- ===== PÁGINA 2 (REVERSO) ===== -->
487 <div class="pagina pagina-2">
488     <div class="columnas">
489         <!-- ===== COLUMNA IZQUIERDA ===== -->
490         <div class="columna">
491
492             <!-- XI. TRATAMIENTO -->
493             <div class="seccion">
494                 <div class="tab-seccion">XI TRATAMIENTO</div>
495                 <div class="cuerpo-seccion">
496                     <div class="rejilla-bloques cuatro">
497                         <div class="bloque">
498                             <div class="titulo-campo">VÍA AÉREA</div>
499                             <div class="opcion" *ngFor="let o of ops('tratamiento', 'viaAereaTratamiento')">
500                                 <span class="casilla" [class.marcada]="marcado('tratamiento', 'viaAereaTratamiento', o)"></span>{{ o | upp
» ercase }}
501                             </div>
502                         </div>
503                         <div class="bloque">
504                             <div class="titulo-campo">CONTROL CERVICAL:</div>
505                             <div class="opcion" *ngFor="let o of ops('tratamiento', 'controlCervical')">
506                                 <span class="casilla" [class.marcada]="marcado('tratamiento', 'controlCervical', o)"></span>{{ o | upperca
» se }}
507                             </div>
508                         </div>
509                         <div class="bloque">
510                             <div class="titulo-campo">ASISTENCIA VENTILATORIA:</div>
511                             <div class="opcion" *ngFor="let o of ops('tratamiento', 'asistenciaVentilatoria')">
512                                 <span class="casilla" [class.marcada]="marcado('tratamiento', 'asistenciaVentilatoria', o)"></span>{{ o |
» uppercase }}
513                             </div>
514                             <div class="titulo-campo">VENTILACIÓN MECÁNICA:</div>
515                             <div class="opcion" *ngFor="let o of ops('tratamiento', 'ventilacionMecanica')">
516                                 <span class="casilla" [class.marcada]="marcado('tratamiento', 'ventilacionMecanica', o)"></span>{{ o | upp
» ercase }}
517                             </div>
518                             <div class="fila-lineas">
519                                 <span class="titulo-campo">FREC:</span><span class="linea corta">{{ v('tratamiento', 'frecuenciaVentilator
» ia') }}</span>

```

```

520         <span class="titulo-campo">VOL:</span><span class="linea corta">{{ v('tratamiento', 'volumenVentilatorio')
521     }}</span>
522     </div>
523     <div class="fila-lineas">
524         <span class="titulo-campo">PEEP:</span><span class="linea corta">{{ v('tratamiento', 'peep') }}</span>
525         <span class="titulo-campo">PIP:</span><span class="linea corta">{{ v('tratamiento', 'presionVentilatoria')
526     }}</span>
527     </div>
528     <div class="fila-lineas">
529         <span class="titulo-campo">I:E:</span><span class="linea corta">{{ v('tratamiento', 'relacionIE') }}</span>
530     </div>
531     <div class="fila-lineas"><span class="titulo-campo">FiO2:</span><span class="linea corta">{{ v('tratamiento', 'fio2') }}</span>
532     <span class="titulo-campo">PS:</span><span class="linea corta">{{ v('tratamiento', 'ps') }}</span>
533     </div>
534     <div class="fila-lineas"><span class="titulo-campo">MODO VENTILATORIO:</span><span class="linea corta">{{ v(
535     'tratamiento', 'modoVentilatorio') }}</span></div>
536     </div>
537     <div class="bloque">
538         <div class="titulo-campo">OXIGENOTERAPIA:</div>
539         <div class="opcion" *ngFor="let o of ops('tratamiento', 'oxigenoterapia')">
540             <span class="casilla" [class.marcada]="marcado('tratamiento', 'oxigenoterapia', o)"></span>{{ o | uppercas
541         e }}
542     </div>
543     <div class="fila-lineas"><span class="titulo-campo">LTS X MIN:</span><span class="linea corta">{{ v('tratami
544     ento', 'litrosPorMinuto') }}</span></div>
545     <div class="titulo-campo">DESCOMPRESIÓN PLEURAL</div>
546     <div class="opcion" *ngFor="let o of ops('tratamiento', 'descompresionPleural')">
547         <span class="casilla" [class.marcada]="marcado('tratamiento', 'descompresionPleural', o)"></span>{{ o | up
548     percase }}
549     </div>
550     </div>
551     <div class="rejilla-bloques cuatro">
552         <div class="bloque">
553             <div class="titulo-campo">CONTROL DE HEMORRAGIAS:</div>
554             <div class="opcion" *ngFor="let o of ops('tratamiento', 'controlHemorragias')">
555                 <span class="casilla" [class.marcada]="marcado('tratamiento', 'controlHemorragias', o)"></span>{{ o | uppe
556             rcase }}
557         </div>
558     </div>
559     <div class="bloque">
560         <div class="titulo-campo">VÍAS VENOSAS:</div>
561         <div class="fila-lineas"><span class="titulo-campo">LÍNEAS IV #:</span><span class="linea corta">{{ v('trata
562         miento', 'lineasIv') }}</span></div>
563         <div class="fila-lineas"><span class="titulo-campo">CATÉTER #:</span><span class="linea corta">{{ v('tratami
564         ento', 'numeroCateter') }}</span></div>
565     </div>
566     <div class="bloque">
567         <div class="titulo-campo">SITIO DE APLICACIÓN:</div>
568         <div class="opcion" *ngFor="let o of ops('tratamiento', 'sitioAplicacion')">
569             <span class="casilla" [class.marcada]="marcado('tratamiento', 'sitioAplicacion', o)"></span>{{ o | upperca
570         se }}
571     </div>
572     <div class="fila-lineas"><span class="titulo-campo">ESPECIFIQUE:</span><span class="linea">{{ v('tratamiento
573     ', 'sitioAplicacionEspecifique') }}</span></div>
574     </div>
575     <div class="bloque">
576         <div class="titulo-campo">TIPO DE SOLUCIONES:</div>
577         <div class="fila-lineas"><span class="linea">{{ v('tratamiento', 'tipoSoluciones') }}</span></div>
578         <div class="fila-lineas"><span class="titulo-campo">CANTIDAD:</span><span class="linea corta">{{ v('tratamie
579         nto', 'cantidadSolucion') }}</span></div>
580         <div class="fila-lineas"><span class="titulo-campo">INFUSIÓN:</span><span class="linea corta">{{ v('tratamie
581         nto', 'infusion') }}</span></div>
582     </div>
583     </div>
584     <table class="tabla-mini farmacologico">
585         <caption>MANEJO FARMACOLÓGICO Y TERAPIA ELÉCTRICA</caption>
586         <thead>
587             <tr><th>HORA</th><th>MEDICAMENTO</th><th>DOSIS</th><th>VÍA DE ADMINISTRACIÓN</th><th>TERAPIA ELÉCTRICA</th><
588         /tr>
589         </thead>
590         <tbody>

```

```

579         <tr *ngFor="let fila of filasFarmaco">
580             <td>{{ fila['hora'] }}</td><td>{{ fila['medicamento'] }}</td><td>{{ fila['dosis'] }}</td>
581             <td>{{ fila['viaAdministracion'] }}</td><td>{{ fila['terapiaElectrica'] }}</td>
582         </tr>
583     </tbody>
584 </table>
585
586     <div class="grupo-opciones">
587         <span class="opcion" *ngFor="let o of ops('tratamiento', 'rcp')">
588             <span class="casilla" [class.marcada]="marcado('tratamiento', 'rcp', o)"></span>{{ o | uppercase }}
589         </span>
590         <span class="opcion" *ngFor="let o of ops('tratamiento', 'inmovilizacion')">
591             <span class="casilla" [class.marcada]="marcado('tratamiento', 'inmovilizacion', o)"></span>{{ o | uppercase
592     }}
593     </span>
594     <span class="opcion"><span class="casilla" [class.marcada]="marcado('tratamiento', 'curacion')"></span>CURACIÓ
595     </span>
596     <span class="opcion"><span class="casilla" [class.marcada]="marcado('tratamiento', 'vendaaje')"></span>VENDAJE<
597     </span>
598 </div>
599 </div>
600 </div>
601 <!-- XII. TRASLADO -->
602 <div class="seccion">
603     <div class="tab-seccion">XII TRASLADO</div>
604     <div class="cuerpo-seccion">
605         <div class="fila-lineas">
606             <span class="titulo-campo">INSTITUCIÓN A LA QUE SE TRASLADA EL PACIENTE:</span>
607             <span class="linea">{{ v('traslado', 'institucionTraslado') }}</span>
608         </div>
609         <div class="rejilla-bloques dos">
610             <div class="bloque">
611                 <div class="titulo-campo">CONDICIÓN DEL PACIENTE</div>
612                 <div class="opcion" *ngFor="let o of ops('traslado', 'condicionPacienteTraslado')">
613                     <span class="casilla" [class.marcada]="marcado('traslado', 'condicionPacienteTraslado', o)"></span>{{ o |
614                 </div>
615                 <div class="opcion" *ngFor="let o of ops('traslado', 'estabilidadPacienteTraslado')">
616                     <span class="casilla" [class.marcada]="marcado('traslado', 'estabilidadPacienteTraslado', o)"></span>{{ o
617                 </div>
618                 <div class="bloque">
619                     <div class="titulo-campo">PRIORIDAD DE TRASLADO</div>
620                     <div class="opcion" *ngFor="let o of ops('traslado', 'prioridadTraslado')">
621                         <span class="casilla" [class.marcada]="marcado('traslado', 'prioridadTraslado', o)"></span>{{ o | uppercas
622                     </div>
623                 </div>
624                 <div class="bloque negativa">
625                     <div class="titulo-negativa">
626                         <span class="casilla" [class.marcada]="marcado('traslado', 'negativaAtencion')"></span>
627                         NEGATIVA A RECIBIR ATENCIÓN / SER TRASLADADO<br />EXIMIENTE DE RESPONSABILIDAD
628                     </div>
629                     <p class="texto-legal">
630                         Mediante la presente declaré que me niego a aceptar el (tratamiento) / (traslado) a un hospital y
631                         reconozco que el personal de la ambulancia UMAM, así como el médico de los mismos me
632                         recomendaron lo anterior, por lo que eximo a UMAM y a dicho personal de toda
633                         responsabilidad que pudiera derivar al haber respetado y cumplir mis deseos.
634                     </p>
635                     <div class="firmas">
636                         <div class="firma">
637                             <img *ngIf="v('traslado', 'firmaPaciente')" [src]="v('traslado', 'firmaPaciente')" class="imagen-firma" alt=
638                             <span class="linea">{{ v('traslado', 'nombrePaciente') }}</span>
639                             <span class="pie-firma">Nombre y firma del paciente</span>
640                         </div>
641                         <div class="firma">
642                             <img *ngIf="v('traslado', 'firmaTestigo')" [src]="v('traslado', 'firmaTestigo')" class="imagen-firma" alt=
643                             <span class="linea">{{ v('traslado', 'nombreTestigo') }}</span>
644                             <span class="pie-firma">Nombre y firma del testigo</span>

```

```

645         </div>
646     </div>
647 </div>
648 </div>
649 </div>
650
651 <!-- XIII. OBSERVACIONES -->
652 <div class="seccion">
653     <div class="tab-seccion">XIII OBSERVACIONES</div>
654     <div class="cuerpo-seccion">
655         <div class="renglones-observaciones">
656             <div class="renglon-observacion">{{ v('observaciones', 'observaciones') }}</div>
657             <div class="renglon-observacion">{{ v('observaciones', 'observacionesAdicionales') }}</div>
658             <div class="renglon-observacion"></div>
659             <div class="renglon-observacion"></div>
660         </div>
661     </div>
662 </div>
663
664 <!-- XIV. SELLO DE MINISTERIO PÚBLICO -->
665 <div class="seccion">
666     <div class="tab-seccion">XIV SELLO DE MINISTERIO PÚBLICO</div>
667     <div class="cuerpo-seccion">
668         <div class="grupo-opciones">
669             <span class="opcion">
670                 <span class="casilla" [class.marcada]="marcado('observaciones', 'ministerioPublicoNotificado')"></span>MINIS
» TERIO PÚBLICO NOTIFICADO
671             </span>
672             <span class="linea">{{ v('observaciones', 'selloMinisterioPublico') }}</span>
673         </div>
674         <div class="caja-sello"></div>
675         <div class="firma centrada">
676             <span class="linea">{{ v('observaciones', 'nombreQuienRecibeMP') }}</span>
677             <span class="pie-firma">NOMBRE Y FIRMA DE QUIEN RECIBE</span>
678         </div>
679     </div>
680 </div>
681 </div>
682
683 <!-- ===== COLUMNA DERECHA ===== -->
684 <div class="columna">
685
686     <!-- XV. DATOS LEGALES -->
687     <div class="seccion">
688         <div class="tab-seccion">XV DATOS LEGALES</div>
689         <div class="cuerpo-seccion">
690             <div class="titulo-campo">AUTORIDAD O AUTORIDADES QUE TOMARON CONOCIMIENTO</div>
691             <div class="fila-lineas"><span class="titulo-campo">DEPENDENCIA:</span><span class="linea">{{ v('datosLegales',
» 'dependenciaAutoridad') }}</span></div>
692             <div class="fila-lineas"><span class="titulo-campo">NÚMERO DE UNIDADES:</span><span class="linea">{{ v('datosLeg
» ales', 'numeroUnidades') }}</span></div>
693             <div class="fila-lineas"><span class="titulo-campo">NOMBRE O NÚMERO DE LOS OFICIALES:</span><span class="linea">
» {{ v('datosLegales', 'nombreNumeroOficiales') }}</span></div>
694
695             <table class="tabla-mini vehiculos">
696                 <caption>VEHÍCULOS INVOLUCRADOS</caption>
697                 <thead>
698                     <tr><th class="col-num"></th><th>TIPO Y MARCA</th><th>PLACAS</th></tr>
699                 </thead>
700                 <tbody>
701                     <tr *ngFor="let fila of filasVehiculos; let i = index">
702                         <td class="col-num">{{ i + 1 }}</td>
703                         <td>{{ fila['tipoMarca'] }}</td>
704                         <td>{{ fila['placas'] }}</td>
705                     </tr>
706                 </tbody>
707             </table>
708
709             <div class="fila-lineas"><span class="titulo-campo">PERTENENCIAS:</span><span class="linea">{{ v('datosLegales',
» 'pertenencias') }}</span></div>
710             <div class="fila-lineas">
711                 <span class="titulo-campo">RECIBIÓ LAS PERTENENCIAS:</span>
712                 <span class="linea">{{ v('datosLegales', 'recibioPertenenciasNombreFirmaCargo') }}</span>
713                 <span class="pie-firma">NOMBRE, FIRMA Y CARGO</span>

```

```

714         </div>
715     </div>
716 </div>
717
718 <!-- XVI. HOSPITAL RECEPTOR -->
719 <div class="seccion">
720     <div class="tab-seccion">XVI HOSPITAL RECEPTOR</div>
721     <div class="cuerpo-seccion">
722         <div class="titulo-campo centrado">ACEPTACIÓN DE HOSPITAL RECEPTOR</div>
723         <div class="caja-sello">{{ v('hospitalReceptor', 'aceptacionHospitalReceptor') }}</div>
724         <div class="firmas">
725             <div class="firma">
726                 <img *ngIf="v('hospitalReceptor', 'firmaQuienEntrega')" [src]="v('hospitalReceptor', 'firmaQuienEntrega')" c
727                 lass="imagen-firma" alt="" />
728                 <span class="linea">{{ v('hospitalReceptor', 'nombreQuienEntrega') }}</span>
729                 <span class="pie-firma">NOMBRE Y FIRMA DE QUIEN ENTREGA EL PACIENTE</span>
730             </div>
731             <div class="firma">
732                 <img *ngIf="v('hospitalReceptor', 'firmaQuienRecibe')" [src]="v('hospitalReceptor', 'firmaQuienRecibe')" cla
733                 ss="imagen-firma" alt="" />
734                 <span class="linea">{{ v('hospitalReceptor', 'nombreQuienRecibe') }}</span>
735                 <span class="pie-firma">NOMBRE Y FIRMA DE PERSONA QUE RECIBE EL PACIENTE</span>
736             </div>
737         </div>
738         <div class="titulo-campo">IMÁGENES DE EKG, RX Y/O LABORATORIOS:</div>
739         <div class="imagenes-adjuntas">
740             <img *ngFor="let imagen of imagenesHospitalReceptor" [src]="imagen" alt="" />
741         </div>
742     </div>
743 </div>
744
745 <!-- XVII. CONSENTIMIENTO INFORMADO -->
746 <div class="seccion">
747     <div class="tab-seccion">XVII CONSENTIMIENTO INFORMADO</div>
748     <div class="cuerpo-seccion">
749         <p class="texto-legal">
750             Declaro que el personal de la ambulancia UMAM me ha explicado, en términos que
751             comprendo, la valoración, el tratamiento y/o el traslado que se me propone realizar,
752             así como sus riesgos, beneficios y alternativas. He tenido oportunidad de hacer
753             preguntas y de manera libre e informada otorgo mi consentimiento para que se lleve a
754             cabo la atención prehospitolaria descrita en este registro.
755         </p>
756         <div class="firmas">
757             <div class="firma">
758                 <img *ngIf="v('consentimientoInformado', 'firmaPacienteConsentimiento')" [src]="v('consentimientoInformado',
759                 'firmaPacienteConsentimiento')" class="imagen-firma" alt="" />
760                 <span class="linea">{{ v('consentimientoInformado', 'nombrePacienteConsentimiento') }}</span>
761                 <span class="pie-firma">NOMBRE Y FIRMA DEL PACIENTE</span>
762             </div>
763             <div class="firma">
764                 <img *ngIf="v('consentimientoInformado', 'firmaResponsableConsentimiento')" [src]="v('consentimientoInformad
765                 o', 'firmaResponsableConsentimiento')" class="imagen-firma" alt="" />
766                 <span class="linea">{{ v('consentimientoInformado', 'nombreResponsableConsentimiento') }}</span>
767                 <span class="pie-firma">NOMBRE Y FIRMA DEL FAMILIAR O RESPONSABLE</span>
768             </div>
769             <div class="firma">
770                 <img *ngIf="v('consentimientoInformado', 'firmaParamedicoConsentimiento')" [src]="v('consentimientoInformado
771                 ', 'firmaParamedicoConsentimiento')" class="imagen-firma" alt="" />
772                 <span class="linea">{{ v('consentimientoInformado', 'nombreParamedicoConsentimiento') }}</span>
773                 <span class="pie-firma">NOMBRE Y FIRMA DEL PARAMÉDICO</span>
774             </div>
775         </div>
776     </div>
777 </div>
778 </div>
779
780 <div class="pie">
781     <div class="pie-linea"><strong>TELS. (656)625-9472</strong></div>
782     <div class="pie-linea"><strong>(656)625-9473</strong></div>
783     <div class="pie-linea">Calle Quinta Amalia # 107 • Fracc. Las Quintas</div>
784     <div class="pie-linea">Cd. Juárez, Chih. • C. P. 32401</div>
785     <div class="pie-linea">E-mail: <strong>ambulanciasumam&#64;yahoo.com</strong></div>
786 </div>

```



```
783     </div>
784 </div>
785 `,
786   styles: [`
787 // Réplica de la hoja física UMAM. Todos los fondos llevan print-color-adjust
788 // para que el azul salga en el PDF.
789 $azul: #1892d3;
790 $azul-oscuro: #0d3a5c;
791 $celeste: #dbeef9;
792 $borde: #b9def2;
793 $rojo: #c0392b;
794 $tinta: #1a1a1a;
795
796 .hoja {
797   width: 8in;
798   margin: 0 auto;
799   font: 6.2px/1.22 Arial, Helvetica, sans-serif;
800   color: $tinta;
801   background: #fff;
802 }
803
804 .pagina {
805   padding: 8px;
806   box-sizing: border-box;
807 }
808
809 // ----- Encabezado -----
810 .encabezado {
811   display: flex;
812   align-items: flex-start;
813   gap: 8px;
814   margin-bottom: 4px;
815 }
816 .logo { display: flex; align-items: center; gap: 4px; min-width: 130px; }
817 .logo-svg { width: 34px; height: 34px; }
818 .logo-texto { display: flex; flex-direction: column; line-height: 1.1; }
819 .logo-umam { font-size: 17px; font-weight: 800; letter-spacing: 1px; color: $azul-oscuro; }
820 .logo-sub { font-size: 6px; font-weight: 700; color: $azul-oscuro; }
821 .titulo-principal {
822   flex: 1;
823   text-align: center;
824   h1 { margin: 0; font-size: 11px; letter-spacing: 0.3px; }
825   .telefonos { font-size: 8.5px; }
826 }
827 .folio {
828   min-width: 90px;
829   text-align: right;
830   .folio-etiqueta { font-size: 9px; font-weight: 700; }
831   .folio-valor { font-size: 13px; font-weight: 700; color: $rojo; letter-spacing: 1px; min-width: 40px; display: inline-block; }
832 }
833 .renglon-estado {
834   display: flex;
835   gap: 4px;
836   align-items: flex-end;
837   margin-bottom: 5px;
838   font-size: 8px;
839   .titulo-campo { font-size: 8px; }
840   .linea { min-height: 10px; }
841 }
842
843 // ----- Estructura de secciones -----
844 .columnas {
845   display: grid;
846   grid-template-columns: 1fr 1fr;
847   gap: 6px;
848   align-items: start;
849 }
850 .columna { display: flex; flex-direction: column; gap: 3px; min-width: 0; }
851
852 .seccion {
853   display: flex;
854   align-items: stretch;
855   border: 1px solid $azul;
```

```
856     break-inside: avoid;
857   }
858   .tab-seccion {
859     writing-mode: vertical-rl;
860     transform: rotate(180deg);
861     text-align: center;
862     background: $celeste;
863     color: $azul-oscuro;
864     font-weight: 700;
865     font-size: 6.5px;
866     letter-spacing: 0.5px;
867     padding: 4px 1px;
868     border-left: 1px solid $azul;
869     print-color-adjust: exact;
870     -webkit-print-color-adjust: exact;
871     flex: 0 0 auto;
872   }
873   .cuerpo-seccion { flex: 1; min-width: 0; padding: 2px 3px; }
874
875   // ----- Primitivas -----
876   .titulo-campo { font-weight: 700; font-size: 6px; }
877   .subtitulo {
878     font-weight: 700;
879     font-size: 6.5px;
880     background: $celeste;
881     padding: 1px 3px;
882     margin: 3px 0 2px;
883     print-color-adjust: exact;
884     -webkit-print-color-adjust: exact;
885   }
886   .banda {
887     background: $celeste;
888     padding: 1px 3px;
889     margin: 3px 0 2px;
890     print-color-adjust: exact;
891     -webkit-print-color-adjust: exact;
892   }
893   .linea {
894     flex: 1;
895     min-width: 30px;
896     min-height: 8px;
897     border-bottom: 1px solid $tinta;
898     padding: 0 2px;
899     &.corta { flex: 0 1 auto; min-width: 22px; }
900   }
901   .fila-lineas {
902     display: flex;
903     flex-wrap: wrap;
904     align-items: flex-end;
905     gap: 2px;
906     margin-bottom: 1.5px;
907   }
908   .casilla {
909     display: inline-block;
910     width: 7px;
911     height: 7px;
912     border: 1px solid $tinta;
913     margin-right: 2px;
914     position: relative;
915     vertical-align: middle;
916     flex: 0 0 auto;
917     box-sizing: border-box;
918     &.marcada:after {
919       content: ' ';
920       position: absolute;
921       inset: -2px 0 0 0;
922       font-size: 7px;
923       font-weight: 700;
924       text-align: center;
925       line-height: 8px;
926     }
927   }
928   .opcion {
929     display: inline-flex;
```



```

930     align-items: center;
931     gap: 1px;
932     margin: 0 4px 0 0;
933     font-size: 6px;
934 }
935 .grupo-opciones {
936     display: flex;
937     flex-wrap: wrap;
938     align-items: center;
939     gap: 3px;
940     margin: 2px 0;
941 }
942 .rejilla-opciones {
943     display: grid;
944     gap: 0 4px;
945     margin: 1px 0 2px;
946     &.dos { grid-template-columns: repeat(2, 1fr); }
947     &.tres { grid-template-columns: repeat(3, 1fr); }
948     &.cuatro { grid-template-columns: repeat(4, 1fr); }
949 }
950 .rejilla-bloques {
951     display: grid;
952     gap: 3px;
953     margin: 2px 0;
954     align-items: stretch;
955     &.tres { grid-template-columns: repeat(3, 1fr); }
956     &.cuatro { grid-template-columns: repeat(2, 1fr); }
957     &.exploracion { grid-template-columns: 1.1fr 1fr; }
958 }
959 .bloque {
960     border: 1px solid $borde;
961     padding: 1px 2px;
962     .opcion { display: flex; }
963 }
964
965 // ----- Tablas -----
966 .tabla-mini {
967     width: 100%;
968     border-collapse: collapse;
969     margin: 3px 0;
970     caption {
971         background: $azul;
972         color: #fff;
973         font-weight: 700;
974         font-size: 7px;
975         padding: 1.5px 0;
976         print-color-adjust: exact;
977         -webkit-print-color-adjust: exact;
978     }
979     th, td {
980         border: 1px solid $azul;
981         font-size: 5.8px;
982         padding: 1px;
983         text-align: center;
984     }
985     th {
986         background: $celeste;
987         color: $azul-oscuro;
988         print-color-adjust: exact;
989         -webkit-print-color-adjust: exact;
990     }
991     td { height: 8px; }
992 }
993
994 // ----- IX: exploración física, siluetas y pupilas -----
995 .tabla-exploracion {
996     width: 100%;
997     border-collapse: collapse;
998     td { border: 1px solid $borde; font-size: 6px; padding: 0.5px 2px; }
999     .codigo { font-weight: 700; width: 12px; text-align: center; background: $celeste; print-color-adjust: exact; -webkit-print-color-adjust: exact; }
1000     td:last-child { width: 11px; text-align: center; }
1001 }
1002 .siluetas {

```

```
1003     display: flex;
1004     justify-content: center;
1005     gap: 10px;
1006     margin: 2px 0;
1007   }
1008   .silueta {
1009     width: 34px;
1010     height: 72px;
1011     circle, path { fill: none; stroke: #666; stroke-width: 1.5; }
1012   }
1013   .zonas-lesion { font-size: 5.5px; .opcion { font-size: 5.5px; } }
1014   .pupilas {
1015     display: flex;
1016     flex-direction: column;
1017     .opcion { display: flex; }
1018     .ojo {
1019       display: inline-block;
1020       width: 8px;
1021       height: 8px;
1022       border-radius: 50%;
1023       border: 1px solid $tinta;
1024       background: radial-gradient(circle, $tinta 0 35%, transparent 36%);
1025       margin-right: 2px;
1026       print-color-adjust: exact;
1027       -webkit-print-color-adjust: exact;
1028     }
1029   }
1030
1031   // ----- Página 2 -----
1032   .rejilla-bloques.dos { grid-template-columns: repeat(2, 1fr); }
1033   .centrado { text-align: center; }
1034   .negativa {
1035     margin-top: 3px;
1036     .titulo-negativa { font-weight: 700; font-size: 6.5px; text-align: center; margin: 2px 0; }
1037   }
1038   .texto-legal {
1039     font-size: 6px;
1040     text-align: justify;
1041     margin: 3px 4px;
1042     line-height: 1.45;
1043   }
1044   .firmas { display: flex; gap: 10px; margin-top: 12px; }
1045   .firma {
1046     flex: 1;
1047     display: flex;
1048     flex-direction: column;
1049     align-items: center;
1050     .imagen-firma { max-width: 100%; max-height: 22px; object-fit: contain; }
1051     .linea { width: 100%; min-height: 10px; text-align: center; }
1052     .pie-firma { font-size: 5.5px; margin-top: 1px; }
1053     &.centrada { margin-top: 8px; }
1054   }
1055   .pie-firma { font-size: 5.5px; }
1056   .renglones-observaciones { display: flex; flex-direction: column; gap: 4px; padding: 2px 0; }
1057   .renglon-observacion {
1058     min-height: 11px;
1059     background: $celeste;
1060     border-bottom: 1px solid $borde;
1061     padding: 1px 3px;
1062     print-color-adjust: exact;
1063     -webkit-print-color-adjust: exact;
1064   }
1065   .caja-sello {
1066     min-height: 60px;
1067     border: 1px solid $borde;
1068     margin: 4px 0;
1069     padding: 2px 3px;
1070   }
1071   .vehiculos {
1072     .col-num { width: 10px; font-weight: 700; }
1073     td { text-align: left; }
1074   }
1075   .imagenes-adjuntas {
1076     display: flex;
```

```

1077 flex-wrap: wrap;
1078 gap: 4px;
1079 margin: 4px 0;
1080 img { width: 60px; height: 60px; object-fit: cover; border: 1px solid $borde; }
1081 }
1082 .pagina-1 { zoom: 0.9; }
1083 // La página 2 (reverso) le sobra espacio abajo tras quitar Material
1084 // Utilizado: se agranda un poco para aprovecharlo, igual que la página 1 se
1085 // achica para caber – mismo mecanismo, sentido contrario.
1086 .pagina-2 { zoom: 1.18; }
1087 .pie {
1088   margin-top: 8px;
1089   background: #111;
1090   color: #fff;
1091   text-align: right;
1092   padding: 6px 10px;
1093   font-size: 8px;
1094   print-color-adjust: exact;
1095   -webkit-print-color-adjust: exact;
1096   .pie-linea { line-height: 1.4; }
1097 }
1098
1099 // ----- Impresión -----
1100 @media print {
1101   @page { size: letter; margin: 0.25in; }
1102   .hoja { width: auto; }
1103   .salto { break-after: page; }
1104   .pagina { padding: 0; }
1105 }
1106 `],
1107 })
1108 export class PdfVistaComponent implements OnChanges {
1109   @Input() registro: Record<string, any> | undefined;
1110
1111   /** Códigos de la hoja física para EXPLORACIÓN FÍSICA (los nombres deben
1112    * coincidir con las opciones de evaluacionSecundaria.exploracionFisica). */
1113   exploracionFisica = [
1114     { codigo: 'D', nombre: 'Deformaciones' },
1115     { codigo: 'CO', nombre: 'Contusiones' },
1116     { codigo: 'A', nombre: 'Abrasiones' },
1117     { codigo: 'P', nombre: 'Penetraciones' },
1118     { codigo: 'MP', nombre: 'Movimientos paradójicos' },
1119     { codigo: 'C', nombre: 'Crepitación' },
1120     { codigo: 'H', nombre: 'Heridas' },
1121     { codigo: 'F', nombre: 'Fracturas' },
1122     { codigo: 'ES', nombre: 'Enfisema subcutáneo' },
1123     { codigo: 'Q', nombre: 'Quemaduras' },
1124     { codigo: 'L', nombre: 'Laceraciones' },
1125     { codigo: 'E', nombre: 'Edema' },
1126     { codigo: 'AS', nombre: 'Alteraciones de sensibilidad' },
1127     { codigo: 'AM', nombre: 'Alteraciones de movilidad' },
1128     { codigo: 'DO', nombre: 'Dolor' },
1129     { codigo: 'AMP', nombre: 'Amputación' },
1130   ];
1131   filasSignos: Record<string, string>[] = [];
1132   filasFarmaco: Record<string, string>[] = [];
1133   filasVehiculos: Record<string, string>[] = [];
1134   imagenesHospitalReceptor: string[] = [];
1135
1136   ngOnChanges(): void {
1137     // Pre-calculadas para que *ngFor no reciba un arreglo nuevo en cada ciclo
1138     // de detección de cambios.
1139     this.filasSignos = this.rellenar('signosVitales', 3);
1140     this.filasFarmaco = this.rellenar('manejoFarmacologico', 4);
1141     this.filasVehiculos = this.rellenar('vehiculosInvolucrados', 4);
1142     this.imagenesHospitalReceptor = (this.registro?['hospitalReceptor']?['imagenesEkgRxLaboratorios'] as string[] | undefi
1143     » ned) ?? [];
1144   }
1145
1146   /** Valor plano de un campo; '' si está vacío (la hoja se imprime en blanco). */
1147   v(seccion: string, campo: string): string {
1148     const valor = this.registro?.[seccion]?.[campo];
1149     if (valor === undefined || valor === null) return '';
1150     if (Array.isArray(valor)) return valor.join(', ');

```

```
1150     return String(valor);
1151   }
1152
1153   /**
1154    * true si la opción está seleccionada: en checkbox-grupo (arreglo) si la
1155    * contiene, en radio-grupo si es el valor, y en checkbox booleano (sin
1156    * opción) si el valor es true.
1157    */
1158   marcado(seccion: string, campo: string, opcion?: string): boolean {
1159     const valor = this.registro?.[seccion]?.[campo];
1160     if (opcion === undefined) return valor === true;
1161     if (Array.isArray(valor)) return valor.includes(opcion);
1162     return valor === opcion;
1163   }
1164
1165   /** Opciones del catálogo SECCIONES para iterar las casillas en el template. */
1166   ops(seccionClave: string, campoClave: string): string[] {
1167     const seccion = SECCIONES.find(s => s.clave === seccionClave);
1168     return seccion?.campos.find(c => c.clave === campoClave)?.opciones ?? [];
1169   }
1170
1171   private rellenar(clave: string, minimo: number): Record<string, string>[] {
1172     const filas = [...((this.registro?.[clave] as Record<string, string>[] | undefined) ?? [])];
1173     while (filas.length < minimo) filas.push({});
1174     return filas;
1175   }
1176 }
```

```

1 import { ComponentFixture, TestBed, fakeAsync, tick } from '@angular/core/testing';
2 import { provideNativeDateAdapter } from '@angular/material/core';
3 import { FormularioComponent } from './formulario.component';
4 import { REGISTRO_SERVICE, RegistroService } from '../core/services/registro.service';
5 import { crearRegistroVacio } from '../core/models/registro.model';
6 import { SECCIONES } from '../core/data/secciones.data';
7
8 describe('FormularioComponent', () => {
9   let fixture: ComponentFixture<FormularioComponent>;
10   let servicioFalso: jasmine.SpyObj<RegistroService>;
11
12   beforeEach(async () => {
13     servicioFalso = jasmine.createSpyObj<RegistroService>('RegistroService', ['guardar', 'obtener', 'listar', 'eliminar']);
14     servicioFalso.listar.and.resolveTo([]);
15     servicioFalso.guardar.and.callFake(async r => r);
16
17     await TestBed.configureTestingModule({
18       imports: [FormularioComponent],
19       providers: [{ provide: REGISTRO_SERVICE, useValue: servicioFalso }, provideNativeDateAdapter()],
20     }).compileComponents();
21
22     fixture = TestBed.createComponent(FormularioComponent);
23     fixture.detectChanges();
24   });
25
26   it('inicia en el paso 0 mostrando solo la primera sección', () => {
27     expect(fixture.componentInstance.pasoActual).toBe(0);
28     const pasos = fixture.nativeElement.querySelectorAll('app-seccion-paso');
29     expect(pasos.length).toBe(1);
30     expect(fixture.nativeElement.textContent).toContain(SECCIONES[0].titulo);
31   });
32
33   it('avanzar() muestra la siguiente sección y retroceder() regresa', () => {
34     fixture.componentInstance.avanzar();
35     fixture.detectChanges();
36     expect(fixture.nativeElement.textContent).toContain(SECCIONES[1].titulo);
37
38     fixture.componentInstance.retroceder();
39     fixture.detectChanges();
40     expect(fixture.nativeElement.textContent).toContain(SECCIONES[0].titulo);
41   });
42
43   it('el último paso sigue mostrando app-seccion-paso (la última sección del catálogo)', () => {
44     fixture.componentInstance.irAPaso(fixture.componentInstance.totalPasos - 1);
45     fixture.detectChanges();
46     expect(fixture.nativeElement.querySelectorAll('app-seccion-paso').length).toBe(1);
47     expect(fixture.nativeElement.textContent).toContain(SECCIONES[SECCIONES.length - 1].titulo);
48   });
49
50   it('el select de saltar a sección lista todas las secciones', () => {
51     const select: HTMLSelectElement = fixture.nativeElement.querySelector('select.salto-seccion');
52     expect(select).toBeTruthy();
53     expect(select.options.length).toBe(SECCIONES.length);
54   });
55
56   it('Anterior se deshabilita en el primer paso; en el último paso "Siguiente" se reemplaza por "Enviar al Historial"', () => {
57     const anterior: HTMLButtonElement = fixture.nativeElement.querySelector('button.paso-anterior');
58     expect(anterior.disabled).toBeTrue();
59     expect(fixture.nativeElement.querySelector('button.paso-siguiente')).toBeTruthy();
60     expect(fixture.nativeElement.querySelector('button.enviar-historial')).toBeFalsy();
61
62     fixture.componentInstance.irAPaso(fixture.componentInstance.totalPasos - 1);
63     fixture.detectChanges();
64     expect(anterior.disabled).toBeFalse();
65     expect(fixture.nativeElement.querySelector('button.paso-siguiente')).toBeFalsy();
66     expect(fixture.nativeElement.querySelector('button.enviar-historial')).toBeTruthy();
67   });
68
69   it('"Enviar al Historial" guarda, exporta el PDF y emite enviarAlHistorial', fakeAsync(() => {
70     spyOn(window, 'print');
71     let emitido = false;
72     fixture.componentInstance.enviarAlHistorial.subscribe(() => (emitido = true));
73

```

```

74     fixture.componentInstance.irAPaso(fixture.componentInstance.totalPasos - 1);
75     fixture.detectChanges();
76
77     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.enviar-historial');
78     boton.click();
79     tick();
80
81     expect(servicioFalso.guardar).toHaveBeenCalled();
82     expect(window.print).toHaveBeenCalled();
83     expect(emitido).toBeTrue();
84   }));
85
86   it('un cambio en el formulario dispara guardar() (autosave) después del debounce', fakeAsync(() => {
87     tick();
88     fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.setValue('12345');
89     tick(1500);
90     expect(servicioFalso.guardar).toHaveBeenCalled();
91   }));
92
93   it('el botón "Guardar borrador" también dispara guardar() de inmediato', () => {
94     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.guardar-borrador');
95     boton.click();
96     expect(servicioFalso.guardar).toHaveBeenCalled();
97   });
98
99   it('el botón "Exportar PDF" llama a window.print()', () => {
100     spyOn(window, 'print');
101     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.exportar-pdf');
102     boton.click();
103     expect(window.print).toHaveBeenCalled();
104   });
105
106   it('el botón "Exportar PDF" toma una foto (snapshot) del formulario para app-pdf-vista en vez de leer form.getRawValue() e
    » n cada ciclo', () => {
107     spyOn(window, 'print');
108     expect(fixture.componentInstance.snapshotParaImpresion).toBeUndefined();
109
110     fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.setValue('PDF-1');
111     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.exportar-pdf');
112     boton.click();
113
114     expect((fixture.componentInstance.snapshotParaImpresion as any).datosGenerales.folio).toBe('PDF-1');
115   });
116
117   it('el botón "Exportar Excel" llama a ExcelExportadorService.exportar()', () => {
118     spyOn(fixture.componentInstance['excelExportador'], 'exportar');
119     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.exportar-excel');
120     boton.click();
121     expect(fixture.componentInstance['excelExportador'].exportar).toHaveBeenCalled();
122   });
123
124   it('muestra un indicador de "Guardado" tras un guardado exitoso', fakeAsync(() => {
125     fixture.componentInstance.guardarBorrador();
126     tick();
127     expect(fixture.componentInstance.estadoGuardado).toContain('Guardado');
128   }));
129
130   it('muestra "Error al guardar" cuando registroService.guardar() rechaza la promesa', fakeAsync(() => {
131     servicioFalso.guardar.and.rejectWith(new Error('fallo de guardado'));
132     fixture.componentInstance.guardarBorrador();
133     tick();
134     expect(fixture.componentInstance.estadoGuardado).toBe('Error al guardar');
135   }));
136
137   it('el botón "Nuevo registro" limpia el formulario y un guardado posterior persiste un id distinto al original', fakeAsync
    » (() => {
138     const idOriginal = fixture.componentInstance['registroActual'].id;
139     fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.setValue('ALGO');
140
141     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.nuevo-registro');
142     boton.click();
143     fixture.detectChanges();
144
145     expect(fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.value).toBe('');

```

```

146
147     fixture.componentInstance.guardarBorrador();
148     tick();
149     const registroGuardado = servicioFalso.guardar.calls.mostRecent().args[0];
150     expect(registroGuardado.id).not.toBe(idOriginal);
151   });
152
153   it('el botón "Nuevo registro" deja el autosave funcionando sobre el formulario nuevo', fakeAsync(() => {
154     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.nuevo-registro');
155     boton.click();
156     fixture.detectChanges();
157     tick();
158
159     servicioFalso.guardar.calls.reset();
160     fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.setValue('NUEVO');
161     tick(1500);
162     expect(servicioFalso.guardar).toHaveBeenCalled();
163   });
164
165   it('el folio capturado en datosGenerales se mapea al campo plano registro.folio al guardar y al exportar a Excel', () => {
166     fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.setValue('F-999');
167
168     fixture.componentInstance.guardarBorrador();
169     expect(servicioFalso.guardar).toHaveBeenCalledWith(jasmine.objectContaining({ folio: 'F-999' }));
170
171     const exportarSpy = spyOn(fixture.componentInstance['excelExportador'], 'exportar');
172     fixture.componentInstance.exportarExcel();
173     expect(exportarSpy).toHaveBeenCalledWith(jasmine.objectContaining({ folio: 'F-999' }));
174   });
175 });
176
177 describe('FormularioComponent con un registro existente', () => {
178   let fixture: ComponentFixture<FormularioComponent>;
179   let servicioFalso: jasmine.SpyObj<RegistroService>;
180   const registroGuardado = { ...crearRegistroVacio(), folio: 'FOLIO-EXISTENTE' };
181
182   beforeEach(async () => {
183     servicioFalso = jasmine.createSpyObj<RegistroService>('RegistroService', ['guardar', 'obtener', 'listar', 'eliminar']);
184     servicioFalso.listar.and.resolveTo([registroGuardado]);
185     servicioFalso.guardar.and.callFake(async r => r);
186
187     await TestBed.configureTestingModule({
188       imports: [FormularioComponent],
189       providers: [{ provide: REGISTRO_SERVICE, useValue: servicioFalso }, provideNativeDateAdapter()],
190     }).compileComponents();
191
192     fixture = TestBed.createComponent(FormularioComponent);
193   });
194
195   it('reconstruye el formulario con los valores del registro cargado en listar()', fakeAsync(() => {
196     fixture.detectChanges();
197     tick();
198     expect(fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.value).toBe('FOLIO-EXISTENTE');
199   }));
200
201   it('omite la recarga si el usuario ya editó el formulario mientras listar() estaba pendiente', fakeAsync(() => {
202     fixture.detectChanges();
203     // Simula que el usuario empezó a escribir durante la espera asíncrona de listar(),
204     // antes de que la promesa se resuelva (todavía no hay tick()).
205     fixture.componentInstance.form.markAsDirty();
206     tick();
207     expect(fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.value).toBe('');
208   }));
209 });
210
211 describe('FormularioComponent con registroInicial (abierto desde el Historial)', () => {
212   let fixture: ComponentFixture<FormularioComponent>;
213   let servicioFalso: jasmine.SpyObj<RegistroService>;
214   const registroDelHistorial = { ...crearRegistroVacio(), folio: 'FOLIO-HISTORIAL' };
215   const masReciente = { ...crearRegistroVacio(), folio: 'FOLIO-MAS-RECIENTE' };
216
217   beforeEach(async () => {
218     servicioFalso = jasmine.createSpyObj<RegistroService>('RegistroService', ['guardar', 'obtener', 'listar', 'eliminar']);
219     // listar() sí tiene registros – si registroInicial no lo bloqueara, este

```



```
220 // se cargaría por encima del que se pidió abrir desde el Historial.
221 servicioFalso.listar.and.resolveTo([masReciente]);
222 servicioFalso.guardar.and.callFake(async r => r);
223
224 await TestBed.configureTestingModule({
225   imports: [FormularioComponent],
226   providers: [{ provide: REGISTRO_SERVICE, useValue: servicioFalso }, provideNativeDateAdapter()],
227 }).compileComponents();
228
229 fixture = TestBed.createComponent(FormularioComponent);
230 fixture.componentInstance.registroInicial = registroDelHistorial;
231 });
232
233 it('usa el registro pasado por @Input en vez de cargar "el más reciente" de listar()', fakeAsync(() => {
234   fixture.detectChanges();
235   tick();
236   expect(fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.value).toBe('FOLIO-HISTORIAL');
237   expect(servicioFalso.listar).not.toHaveBeenCalled();
238 }));
239
240 it('arranca en el paso 0 y con el autosave suscrito', fakeAsync(() => {
241   fixture.detectChanges();
242   tick();
243   expect(fixture.componentInstance.pasoActual).toBe(0);
244
245   fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.setValue('EDITADO');
246   tick(1500);
247   expect(servicioFalso.guardar).toHaveBeenCalled();
248 }));
249 });
250
251 describe('FormularioComponent cuando listar() rechaza la promesa (localStorage corrupto)', () => {
252   let fixture: ComponentFixture<FormularioComponent>;
253   let servicioFalso: jasmine.SpyObj<RegistroService>;
254
255   beforeEach(async () => {
256     servicioFalso = jasmine.createSpyObj<RegistroService>('RegistroService', ['guardar', 'obtener', 'listar', 'eliminar']);
257     servicioFalso.listar.and.rejectWith(new Error('localStorage corrupto'));
258     servicioFalso.guardar.and.callFake(async r => r);
259
260     await TestBed.configureTestingModule({
261       imports: [FormularioComponent],
262       providers: [{ provide: REGISTRO_SERVICE, useValue: servicioFalso }, provideNativeDateAdapter()],
263     }).compileComponents();
264
265     fixture = TestBed.createComponent(FormularioComponent);
266   });
267
268   it('el autosave sigue quedando suscrito y llama a guardar() aunque listar() rechace', fakeAsync(() => {
269     fixture.detectChanges();
270     tick();
271     fixture.componentInstance.form.get('datosGenerales')!.get('folio')!.setValue('X');
272     tick(1500);
273     expect(servicioFalso.guardar).toHaveBeenCalled();
274   }));
275 });
```



```

1 import { Component, EventEmitter, Inject, Input, OnInit, Output } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { FormBuilder, FormGroup, ReactiveFormsModule } from '@angular/forms';
4 import { debounceTime } from 'rxjs';
5 import { MatButtonModule } from '@angular/material/button';
6 import { MatIconModule } from '@angular/material/icon';
7 import { MatToolbarModule } from '@angular/material/toolbar';
8 import { SeccionPasoComponent } from '../../shared/seccion-paso/seccion-paso.component';
9 import { PdfVistaComponent } from '../../exportacion/pdf-vista/pdf-vista.component';
10 import { ExcelExportadorService } from '../../exportacion/excel-exportador.service';
11 import { SECCIONES } from '../../core/data/secciones.data';
12 import { construirFormularioRegistro } from '../../core/forms/construir-formulario';
13 import { crearRegistroVacio, RegistroAtencionPrehospitalaria } from '../../core/models/registro.model';
14 import { REGISTRO_SERVICE, RegistroService } from '../../core/services/registro.service';
15
16 @Component({
17   selector: 'app-formulario',
18   standalone: true,
19   imports: [
20     CommonModule, ReactiveFormsModule, MatButtonModule, MatIconModule, MatToolbarModule,
21     SeccionPasoComponent, PdfVistaComponent,
22   ],
23   template: `
24     <mat-toolbar class="encabezado">
25       <span>UMAM – Registro de Atención Prehospitalaria</span>
26     </mat-toolbar>
27
28     <div class="encabezado-paso">
29       <div class="progreso-texto">
30         <span>Paso {{ pasoActual + 1 }} de {{ totalPasos }}</span>
31         <select class="salto-seccion" [value]="pasoActual" (change)="irAPaso(+$any($event.target).value)" aria-label="Ir a s
32         ección">
33           <option *ngFor="let titulo of titulosPasos; let i = index" [value]="i">{{ titulo }}</option>
34         </select>
35       </div>
36       <div class="barra-progreso">
37         <div class="barra-progreso-relleno" [style.width.%]="((pasoActual + 1) / totalPasos) * 100"></div>
38       </div>
39
40       <div class="layout" [formGroup]="form">
41         <main class="contenido">
42           <app-seccion-paso
43             [seccion]="secciones[pasoActual]"
44             [grupo]="grupoDeSeccion(secciones[pasoActual].clave)"
45           ></app-seccion-paso>
46         </main>
47       </div>
48
49       <div class="barra-acciones">
50         <button type="button" class="paso-anterior" mat-stroked-button [disabled]="pasoActual === 0" (click)="retroceder()">
51           <mat-icon>arrow_back</mat-icon> Anterior
52         </button>
53         <button *ngIf="pasoActual < totalPasos - 1" type="button" class="paso-siguiente" mat-raised-button color="primary" (cl
54         ick)="avanzar()">
55           Siguiente <mat-icon>arrow_forward</mat-icon>
56         </button>
57         <button *ngIf="pasoActual === totalPasos - 1" type="button" class="enviar-historial" mat-raised-button color="primary"
58         (click)="enviarYVerHistorial()">
59           Enviar al Historial <mat-icon>send</mat-icon>
60         </button>
61         <span class="separador"></span>
62         <button type="button" class="guardar-borrador" mat-icon-button (click)="guardarBorrador()" aria-label="Guardar borrado
63         r" title="Guardar borrador">
64           <mat-icon>save</mat-icon>
65         </button>
66         <button type="button" class="exportar-pdf" mat-icon-button (click)="exportarPdf()" aria-label="Exportar PDF" title="Ex
67         portar PDF">
68           <mat-icon>picture_as_pdf</mat-icon>
69         </button>
70         <button type="button" class="exportar-excel" mat-icon-button (click)="exportarExcel()" aria-label="Exportar Excel" tit
71         le="Exportar Excel">
72           <mat-icon>grid_on</mat-icon>
73         </button>

```

```

69     <button type="button" class="nuevo-registro" mat-icon-button (click)="nuevoRegistro()" aria-label="Nuevo registro" tit
» le="Nuevo registro">
70         <mat-icon>add</mat-icon>
71     </button>
72     <span class="estado-guardado" [class.error]="estadoGuardado === 'Error al guardar'">{{ estadoGuardado }}</span>
73 </div>
74
75 <div class="solo-impresion">
76     <app-pdf-vista [registro]="snapshotParaImpresion"></app-pdf-vista>
77 </div>
78 `
79 styles: [
80     .encabezado { background: var(--umam-header-bg, #1892d3); color: var(--umam-header-fg, #fff); }
81     .encabezado-paso { position: sticky; top: 0; z-index: 5; background: #fff; padding: 10px 16px 0; border-bottom: 1px soli
» d var(--umam-section-border, #b9def2); }
82     .progreso-texto { display: flex; align-items: center; justify-content: space-between; gap: 12px; margin-bottom: 8px; }
83     .progreso-texto > span { font-weight: 600; color: var(--umam-header-bg, #1892d3); white-space: nowrap; }
84     .salto-seccion { flex: 1; min-width: 0; max-width: 340px; min-height: 40px; padding: 4px 8px; border: 1px solid var(--um
» am-section-border, #b9def2); border-radius: 8px; font-size: 0.9rem; color: #333; background: #fff; }
85     .barra-progreso { height: 4px; background: var(--umam-section-bg, #dbeef9); border-radius: 2px 2px 0 0; overflow: hidden
» ; }
86     .barra-progreso-relleno { height: 100%; background: var(--umam-header-bg, #1892d3); transition: width 0.2s; }
87     .layout { padding: 16px; padding-bottom: 88px; }
88     .contenido { max-width: 900px; margin: 0 auto; }
89     .barra-acciones { position: fixed; bottom: 0; left: 0; right: 0; z-index: 5; display: flex; align-items: center; gap: 8p
» x; padding: 10px 16px; background: white; border-top: 1px solid var(--umam-section-border, #b9def2); }
90     .paso-siguiente, .enviar-historial { min-width: 130px; }
91     .separador { flex: 1; }
92     .estado-guardado { font-size: 0.8rem; color: #555; }
93     .estado-guardado.error { color: #c0392b; font-weight: 600; }
94     .solo-impresion { display: none; }
95     @media print {
96         .encabezado, .encabezado-paso, .barra-acciones, .layout { display: none !important; }
97         .solo-impresion { display: block !important; }
98     }
99 `],
100 })
101 export class FormularioComponent implements OnInit {
102     /**
103      * Cuando App abre un trámite desde el Historial, lo pasa aquí en vez de dejar
104      * que ngOnInit cargue "el más reciente" de localStorage – ver ngOnInit.
105      */
106     @Input() registroInicial?: RegistroAtencionPrehospitalaria;
107
108     /** Se dispara al terminar "Enviar al Historial" – App usa esto para cambiar de vista. */
109     @Output() enviarAlHistorial = new EventEmitter<void>();
110
111     secciones = SECCIONES;
112     form: FormGroup;
113     estadoGuardado = '';
114     pasoActual = 0;
115     totalPasos = SECCIONES.length;
116     titulosPasos = SECCIONES.map(s => s.titulo);
117     /**
118      * Foto del formulario tomada justo antes de imprimir. La vista de impresión
119      * solo importa en el instante de exportar/imprimir, así que evitamos leer
120      * form.getRawValue() (recorre las ~600 controles del formulario) en cada
121      * ciclo de detección de cambios vía binding directo en la plantilla –
122      * costoso y constante en una app cuya prioridad #1 es la velocidad de
123      * captura en celular.
124      */
125     snapshotParaImpresion: Record<string, unknown> | undefined;
126     private registroActual: RegistroAtencionPrehospitalaria;
127
128     constructor(
129         private fb: FormBuilder,
130         @Inject(REGISTRO_SERVICE) private registroService: RegistroService,
131         private excelExportador: ExcelExportadorService,
132     ) {
133         this.registroActual = crearRegistroVacio();
134         this.form = construirFormularioRegistro(this.fb, this.registroActual);
135     }
136
137     async ngOnInit(): Promise<void> {

```

```

138 // Se abrió un trámite puntual desde el Historial: se usa ese registro tal
139 // cual, sin dejar que la carga de "el más reciente" de abajo lo pise.
140 if (this.registroInicial) {
141   this.cargarRegistro(this.registroInicial);
142   return;
143 }
144 try {
145   const existentes = await this.registroService.listar();
146   // Solo reemplazamos this.form por el registro cargado si el usuario no ha
147   // empezado a editar el formulario inicial (vacío) durante esta espera asíncrona.
148   // Si ya está "dirty", el usuario ya escribió algo mientras listar() resolvía;
149   // sobrescribir this.form aquí perdería esa edición en curso, así que la
150   // conservamos y omitimos la recarga en vez de arriesgar pérdida de datos.
151   if (existentes.length > 0 && !this.form.dirty) {
152     this.registroActual = existentes[existentes.length - 1];
153     this.form = construirFormularioRegistro(this.fb, this.registroActual);
154   }
155 } catch (error) {
156   // Si listar() rechaza (p. ej. localStorage corrupto), seguimos con el
157   // borrador vacío en vez de dejar que el error se propague – lo importante
158   // es que el autosave de abajo (finally) siempre termine suscrito, sin
159   // importar si la carga del borrador existente tuvo éxito o no.
160   console.error('No se pudieron cargar los registros guardados; se continúa con un borrador nuevo.', error);
161 } finally {
162   this.suscribirAutosave();
163 }
164 }
165
166 /**
167  * Empieza un registro nuevo desde cero: formulario en blanco, un id nuevo,
168  * y el autosave vuelto a suscribir sobre el formulario nuevo (la suscripción
169  * anterior queda apuntando al FormGroup viejo, que ya no recibe entradas del
170  * usuario una vez que [formGroup] en la plantilla pasa a apuntar aquí).
171  * Así la app puede usarse para un segundo paciente sin tener que limpiar
172  * localStorage a mano.
173  */
174 nuevoRegistro(): void {
175   this.cargarRegistro(crearRegistroVacio());
176 }
177
178 /**
179  * Reemplaza el formulario en edición por el registro dado – mismo mecanismo
180  * que "Nuevo registro" (id nuevo o no, formulario reconstruido desde cero,
181  * autosave vuelto a suscribir sobre el FormGroup nuevo), usado también para
182  * abrir un trámite puntual desde el Historial.
183  */
184 cargarRegistro(registro: RegistroAtencionPrehospitalaria): void {
185   this.registroActual = registro;
186   this.form = construirFormularioRegistro(this.fb, this.registroActual);
187   this.estadoGuardado = '';
188   this.pasoActual = 0;
189   this.suscribirAutosave();
190 }
191
192 avanzar(): void {
193   this.irAPaso(this.pasoActual + 1);
194 }
195
196 retroceder(): void {
197   this.irAPaso(this.pasoActual - 1);
198 }
199
200 irAPaso(paso: number): void {
201   if (paso < 0 || paso > this.totalPasos - 1) return;
202   this.pasoActual = paso;
203   window.scrollTo(0, 0);
204 }
205
206 private suscribirAutosave(): void {
207   this.form.valueChanges.pipe(debounceTime(1000)).subscribe(() => this.guardarBorrador());
208 }
209
210 guardarBorrador(): void {
211   this.guardar();

```

```
212 }
213
214 private async guardar(): Promise<void> {
215   const registro = this.construirRegistroDesdeFormulario();
216   try {
217     await this.registroService.guardar(registro);
218     this.estadoGuardado = `Guardado ${new Date().toLocaleTimeString()}`;
219   } catch (error) {
220     console.error('Error al guardar el borrador', error);
221     this.estadoGuardado = 'Error al guardar';
222   }
223 }
224
225 exportarPdf(): void {
226   this.snapshotParaImpresion = this.form.getRawValue();
227   window.print();
228 }
229
230 /**
231  * Botón del último paso del wizard: guarda el trámite, dispara la descarga
232  * del PDF (mismo window.print() de "Exportar PDF" – el usuario elige
233  * "Guardar como PDF" en el diálogo del navegador, igual que en el resto de
234  * la app) y le avisa a App que muestre el Historial, donde el trámite
235  * recién guardado ya va a aparecer.
236  */
237 async enviarYVerHistorial(): Promise<void> {
238   await this.guardar();
239   this.exportarPdf();
240   this.enviarAlHistorial.emit();
241 }
242
243 exportarExcel(): void {
244   const registro = this.construirRegistroDesdeFormulario();
245   this.excelExportador.exportar(registro);
246 }
247
248 /**
249  * Combina el valor crudo del formulario con el registro base para persistir/exportar.
250  * folio/estado/ciudad viven como campos planos en la raíz de RegistroAtencionPrehospitalaria
251  * (para mantener simple la forma almacenada por RegistroService), pero en el formulario/UI
252  * están anidados bajo la sección 'datosGenerales' (para que el acordeón y los exportadores
253  * los traten como una sección más) – así que los mapeamos de vuelta explícitamente aquí.
254  */
255 private construirRegistroDesdeFormulario(): RegistroAtencionPrehospitalaria {
256   const valor = this.form.getRawValue();
257   const datosGenerales = valor.datosGenerales ?? {};
258   return {
259     ...this.registroActual,
260     ...valor,
261     folio: datosGenerales.folio ?? '',
262     estado: datosGenerales.estado ?? '',
263     ciudad: datosGenerales.ciudad ?? '',
264   };
265 }
266
267 grupoDeSeccion(clave: string): FormGroup {
268   return this.form.get(clave) as FormGroup;
269 }
270 }
```

```

1 import { ComponentFixture, TestBed, fakeAsync, tick } from '@angular/core/testing';
2 import { HistorialComponent } from './historial.component';
3 import { REGISTRO_SERVICE, RegistroService } from '../../../core/services/registro.service';
4 import { ExcelExportadorService } from '../../../exportacion/excel-exportador.service';
5 import { crearRegistroVacio, RegistroAtencionPrehospitalaria } from '../../../core/models/registro.model';
6
7 function registroDePrueba(datos: Partial<RegistroAtencionPrehospitalaria> = {}): RegistroAtencionPrehospitalaria {
8   return { ...crearRegistroVacio(), ...datos };
9 }
10
11 describe('HistorialComponent', () => {
12   let fixture: ComponentFixture<HistorialComponent>;
13   let servicioFalso: jasmine.SpyObj<RegistroService>;
14   let excelExportadorFalso: jasmine.SpyObj<ExcelExportadorService>;
15
16   beforeEach(async () => {
17     servicioFalso = jasmine.createSpyObj<RegistroService>('RegistroService', ['guardar', 'obtener', 'listar', 'eliminar']);
18     excelExportadorFalso = jasmine.createSpyObj<ExcelExportadorService>('ExcelExportadorService', ['exportar']);
19
20     await TestBed.configureTestingModule({
21       imports: [HistorialComponent],
22       providers: [
23         { provide: REGISTRO_SERVICE, useValue: servicioFalso },
24         { provide: ExcelExportadorService, useValue: excelExportadorFalso },
25       ],
26     }).compileComponents();
27
28     fixture = TestBed.createComponent(HistorialComponent);
29   });
30
31   it('muestra "Todavía no hay trámites guardados." cuando el historial está vacío', fakeAsync(() => {
32     servicioFalso.listar.and.resolveTo([]);
33     fixture.detectChanges();
34     tick();
35     fixture.detectChanges();
36
37     expect(fixture.nativeElement.textContent).toContain('Todavía no hay trámites guardados.');
```

Número 6564121593

```

38     expect(fixture.nativeElement.querySelector('.tabla-historial')).toBeFalsy();
39   });
40
41   it('lista los trámites guardados del más reciente al más antiguo, con folio/fecha/paciente/ciudad', fakeAsync(() => {
42     const viejo = registroDePrueba({
43       folio: 'F-1', ciudad: 'Cd. Juárez', fechaCreacion: '2026-01-01T10:00:00.000Z',
44       datosPaciente: { nombreOMediaFiliacion: 'Juan Pérez' },
45     });
46     const nuevo = registroDePrueba({
47       folio: 'F-2', ciudad: 'Chihuahua', fechaCreacion: '2026-02-01T10:00:00.000Z',
48       datosPaciente: { nombreOMediaFiliacion: 'Ana López' },
49     });
50     servicioFalso.listar.and.resolveTo([viejo, nuevo]);
51     fixture.detectChanges();
52     tick();
53     fixture.detectChanges();
54
55     const filas = fixture.nativeElement.querySelectorAll('.tabla-historial tbody tr');
56     expect(filas.length).toBe(2);
57     expect(filas[0].textContent).toContain('F-2');
58     expect(filas[0].textContent).toContain('Ana López');
59     expect(filas[0].textContent).toContain('Chihuahua');
60     expect(filas[1].textContent).toContain('F-1');
61     expect(filas[1].textContent).toContain('Juan Pérez');
62   });
63
64   it('muestra "(sin nombre)" cuando el trámite no tiene nombre de paciente capturado', fakeAsync(() => {
65     servicioFalso.listar.and.resolveTo([registroDePrueba({ folio: 'F-3' })]);
66     fixture.detectChanges();
67     tick();
68     fixture.detectChanges();
69
70     expect(fixture.nativeElement.querySelector('.tabla-historial tbody tr').textContent).toContain('(sin nombre)');
71   });
72
73   it('el botón "Abrir" emite el registro elegido', fakeAsync(() => {
74     const registro = registroDePrueba({ folio: 'F-4' });

```

```
75     servicioFalso.listar.and.resolveTo([registro]);
76     fixture.detectChanges();
77     tick();
78     fixture.detectChanges();
79
80     let emitido: RegistroAtencionPrehospitalaria | undefined;
81     fixture.componentInstance.abrir.subscribe(r => (emitido = r));
82
83     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.abrir');
84     boton.click();
85
86     expect(emitido).toBe(registro);
87   }));
88
89   it('el botón "Exportar PDF" arma un snapshot anidado (datosGenerales.folio) y llama a window.print()', fakeAsync(() => {
90     const registro = registroDePrueba({ folio: 'F-5', estado: 'Chihuahua', ciudad: 'Cd. Juárez' });
91     servicioFalso.listar.and.resolveTo([registro]);
92     fixture.detectChanges();
93     tick();
94     fixture.detectChanges();
95
96     spyOn(window, 'print');
97     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.exportar-pdf');
98     boton.click();
99
100    expect(window.print).toHaveBeenCalled();
101    expect((fixture.componentInstance.snapshotParaImpresion as any).datosGenerales.folio).toBe('F-5');
102  }));
103
104   it('el botón "Exportar Excel" llama a ExcelExportadorService.exportar() con el registro de esa fila', fakeAsync(() => {
105     const registro = registroDePrueba({ folio: 'F-6' });
106     servicioFalso.listar.and.resolveTo([registro]);
107     fixture.detectChanges();
108     tick();
109     fixture.detectChanges();
110
111     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.exportar-excel');
112     boton.click();
113
114     expect(excelExportadorFalso.exportar).toHaveBeenCalledWith(registro);
115   }));
116
117   it('el botón "Eliminar" pide confirmación, borra el registro y lo quita de la lista', fakeAsync(() => {
118     const registro = registroDePrueba({ folio: 'F-7' });
119     servicioFalso.listar.and.resolveTo([registro]);
120     servicioFalso.eliminar.and.resolveTo();
121     fixture.detectChanges();
122     tick();
123     fixture.detectChanges();
124
125     spyOn(window, 'confirm').and.returnValue(true);
126     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.eliminar');
127     boton.click();
128     tick();
129     fixture.detectChanges();
130
131     expect(window.confirm).toHaveBeenCalled();
132     expect(servicioFalso.eliminar).toHaveBeenCalledWith(registro.id);
133     expect(fixture.nativeElement.textContent).toContain('Todavía no hay trámites guardados.');
```

Desarrollado Ingeniero Mario Isai Moreno Duran · Número 6564121593

```
149     expect(fixture.componentInstance.registros.length).toBe(1);
150   });
151
152   it('si listar() rechaza la promesa, muestra el estado vacío en vez de propagar el error', fakeAsync(() => {
153     servicioFalso.listar.and.rejectWith(new Error('localStorage corrupto'));
154     fixture.detectChanges();
155     tick();
156     fixture.detectChanges();
157
158     expect(fixture.nativeElement.textContent).toContain('Todavía no hay trámites guardados.');
```

```
159   });
160 });
```



```

1 import { Component, EventEmitter, Inject, OnInit, Output } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { FormBuilder } from '@angular/forms';
4 import { MatButtonModule } from '@angular/material/button';
5 import { MatIconModule } from '@angular/material/icon';
6 import { MatToolbarModule } from '@angular/material/toolbar';
7 import { PdfVistaComponent } from '../exportacion/pdf-vista/pdf-vista.component';
8 import { ExcelExportadorService } from '../exportacion/excel-exportador.service';
9 import { construirFormularioRegistro } from '../../../core/forms/construir-formulario';
10 import { RegistroAtencionPrehospitalaria } from '../../../core/models/registro.model';
11 import { REGISTRO_SERVICE, RegistroService } from '../../../core/services/registro.service';
12
13 /**
14  * Historial de trámites guardados (RegistroService.listar() ya soportaba
15  * varios registros desde el principio; solo faltaba una vista para verlos).
16  * Cada fila deja abrir el trámite para seguir editándolo, exportar su PDF o
17  * Excel tal cual quedó guardado, o eliminarlo.
18  */
19 @Component({
20   selector: 'app-historial',
21   standalone: true,
22   imports: [CommonModule, MatButtonModule, MatIconModule, MatToolbarModule, PdfVistaComponent],
23   template: `
24     <mat-toolbar class="encabezado">
25       <span>UMAM – Historial de Trámites</span>
26     </mat-toolbar>
27
28     <div class="contenido">
29       <p *ngIf="cargando" class="estado">Cargando historial...</p>
30       <p *ngIf="!cargando && registros.length === 0" class="estado">Todavía no hay trámites guardados.</p>
31
32       <div class="tabla-envoltura" *ngIf="!cargando && registros.length > 0">
33         <table class="tabla-historial">
34           <thead>
35             <tr>
36               <th>Folio</th>
37               <th>Fecha</th>
38               <th>Paciente</th>
39               <th>Ciudad</th>
40               <th>Documentos</th>
41             </tr>
42           </thead>
43           <tbody>
44             <tr *ngFor="let registro of registros">
45               <td>{{ registro.folio | '-' }}</td>
46               <td>{{ registro.fechaCreacion | date: 'dd/MM/yyyy HH:mm' }}</td>
47               <td>{{ nombrePaciente(registro) }}</td>
48               <td>{{ registro.ciudad | '-' }}</td>
49               <td class="acciones-fila">
50                 <button type="button" class="abrir" mat-stroked-button (click)="abrirRegistro(registro)" title="Abrir para e
51                 <mat-icon>edit</mat-icon> Abrir
52                 </button>
53                 <button type="button" class="exportar-pdf" mat-icon-button (click)="exportarPdf(registro)" aria-label="Expor
54                 <mat-icon>picture_as_pdf</mat-icon>
55                 </button>
56                 <button type="button" class="exportar-excel" mat-icon-button (click)="exportarExcel(registro)" aria-label="E
57                 <mat-icon>grid_on</mat-icon>
58                 </button>
59                 <button type="button" class="eliminar" mat-icon-button (click)="eliminar(registro)" aria-label="Eliminar" ti
60                 <mat-icon>delete</mat-icon>
61                 </button>
62               </td>
63             </tr>
64           </tbody>
65         </table>
66       </div>
67     </div>
68
69     <div class="solo-impresion">
70       <app-pdf-vista [registro]="snapshotParaImpresion"></app-pdf-vista>

```



```

71     </div>
72     `,
73     styles: [
74       .encabezado { background: var(--umam-header-bg, #1892d3); color: var(--umam-header-fg, #fff); }
75       .contenido { max-width: 900px; margin: 0 auto; padding: 16px; }
76       .estado { color: #555; text-align: center; padding: 32px 0; }
77       .tabla-envoltura { overflow-x: auto; border: 1px solid var(--umam-section-border, #b9def2); border-radius: 12px; }
78       .tabla-historial { width: 100%; border-collapse: collapse; background: #fff; }
79       .tabla-historial th, .tabla-historial td { padding: 10px 12px; text-align: left; white-space: nowrap; border-bottom: 1px
80     solid var(--umam-section-border, #b9def2); }
81       .tabla-historial th { background: var(--umam-section-bg, #dbeef9); color: var(--umam-header-bg, #1892d3); font-size: 0.8
82     rem; text-transform: uppercase; letter-spacing: 0.03em; }
83       .tabla-historial tbody tr:last-child td { border-bottom: none; }
84       .acciones-fila { display: flex; align-items: center; gap: 4px; }
85       .abrir { min-height: 40px; }
86       .solo-impresion { display: none; }
87       @media print {
88         .encabezado, .contenido { display: none !important; }
89         .solo-impresion { display: block !important; }
90     },
91   ]),
92   export class HistorialComponent implements OnInit {
93     @Output() abrir = new EventEmitter<RegistroAtencionPrehospitalaria>();
94
95     registros: RegistroAtencionPrehospitalaria[] = [];
96     cargando = true;
97     /** Foto tomada justo antes de imprimir – mismo patrón que FormularioComponent. */
98     snapshotParaImpresion: Record<string, unknown> | undefined;
99
100    constructor(
101      private fb: FormBuilder,
102      @Inject(REGISTRO_SERVICE) private registroService: RegistroService,
103      private excelExportador: ExcelExportadorService,
104    ) {}
105
106    async ngOnInit(): Promise<void> {
107      try {
108        const todos = await this.registroService.listar();
109        // Más reciente primero.
110        this.registros = [...todos].sort((a, b) => b.fechaCreacion.localeCompare(a.fechaCreacion));
111      } catch (error) {
112        console.error('No se pudo cargar el historial de trámites.', error);
113        this.registros = [];
114      } finally {
115        this.cargando = false;
116      }
117    }
118
119    nombrePaciente(registro: RegistroAtencionPrehospitalaria): string {
120      const nombre = (registro.datosPaciente as Record<string, unknown> | undefined)?.['nombreOMediaFiliacion'];
121      return typeof nombre === 'string' && nombre.trim() ? nombre : '(sin nombre)';
122    }
123
124    abrirRegistro(registro: RegistroAtencionPrehospitalaria): void {
125      this.abrir.emit(registro);
126    }
127
128    /**
129     * No hay un FormGroup en vivo para un trámite del historial, así que se
130     * arma uno temporal solo para obtener su getRawValue() en la misma forma
131     * anidada (datosGenerales.folio/estado/ciudad, etc.) que espera
132     * PdfVistaComponent – igual que hace FormularioComponent.exportarPdf().
133     */
134    exportarPdf(registro: RegistroAtencionPrehospitalaria): void {
135      const formularioTemporal = construirFormularioRegistro(this.fb, registro);
136      this.snapshotParaImpresion = formularioTemporal.getRawValue();
137      window.print();
138    }
139
140    exportarExcel(registro: RegistroAtencionPrehospitalaria): void {
141      this.excelExportador.exportar(registro);
142    }

```

```
143   async eliminar(registro: RegistroAtencionPrehospitalaria): Promise<void> {  
144     const confirmado = confirm(`¿Eliminar el trámite con folio "${registro.folio || '(sin folio)'}"? Esta acción no se puede  
    » deshacer.`);  
145     if (!confirmado) return;  
146     await this.registroService.eliminar(registro.id);  
147     this.registros = this.registros.filter(r => r.id !== registro.id);  
148   }  
149 }
```

Desarrollado Ingeniero Mario Isai Moreno Duran · Número 6564121593

```

73     // seleccionar la otra opción reemplaza el valor
74     chips[0].click();
75     fixture.detectChanges();
76     expect(control.value).toBe('Permeable');
77
78     // tocar el chip activo lo deselectiona (regresa a '')
79     chips[0].click();
80     fixture.detectChanges();
81     expect(control.value).toBe('');
82 });
83
84 it('renderiza el checkbox simple como chip individual', () => {
85     const campo: CampoFormulario = { clave: 'eyectado', etiqueta: 'Eyectado', tipo: 'checkbox' };
86     const control = new FormControl(false);
87     fixture.componentInstance.campo = campo;
88     fixture.componentInstance.control = control;
89     fixture.detectChanges();
90
91     const chip: HTMLButtonElement = fixture.nativeElement.querySelector('button.chip');
92     expect(chip).toBeTruthy();
93     expect(chip.textContent).toContain('Eyectado');
94
95     chip.click();
96     fixture.detectChanges();
97     expect(control.value).toBe(true);
98
99     chip.click();
100    fixture.detectChanges();
101    expect(control.value).toBe(false);
102 });
103
104 it('renderiza un <select> para tipo "select" con las opciones dadas', async () => {
105     const campo: CampoFormulario = { clave: 'sexo', etiqueta: 'Sexo', tipo: 'select', opciones: ['Masculino', 'Femenino'] };
106     const control = new FormControl('');
107     fixture.componentInstance.campo = campo;
108     fixture.componentInstance.control = control;
109     fixture.detectChanges();
110
111     // mat-select no renderiza un <select> nativo; usamos el harness de Material
112     // para abrir el panel (proyectado en un overlay del CDK) y verificar las opciones.
113     const select = fixture.nativeElement.querySelector('mat-select');
114     expect(select).toBeTruthy();
115
116     const loader: HarnessLoader = TestBedHarnessEnvironment.loader(fixture);
117     const selectHarness = await loader.getHarness(MatSelectHarness);
118     await selectHarness.open();
119     const opciones = await selectHarness.getOptions();
120     expect(opciones.length).toBe(2);
121     expect(await opciones[0].getText()).toBe('Masculino');
122     expect(await opciones[1].getText()).toBe('Femenino');
123 });
124
125 it('tipo "imagenes" ofrece botones separados de "Tomar foto" (cámara) y "Subir imagen" (archivo/galería)', () => {
126     const campo: CampoFormulario = { clave: 'imagenesEkgrLaboratorios', etiqueta: 'Imágenes', tipo: 'imagenes' };
127     fixture.componentInstance.campo = campo;
128     fixture.componentInstance.control = new FormControl<string[]>([]);
129     fixture.detectChanges();
130
131     const botones: HTMLButtonElement[] = Array.from(fixture.nativeElement.querySelectorAll('.acciones-imagenes button'));
132     expect(botones.map(b => b.textContent!.trim()).toEqual([' Tomar foto', ' Subir imagen']));
133
134     const inputs: HTMLInputElement[] = Array.from(fixture.nativeElement.querySelectorAll('input[type="file"]'));
135     expect(inputs.length).toBe(2);
136     // El de cámara pide "capture" para que el celular abra la cámara directo en vez
137     // de la galería; el de subir archivo no lo lleva y sí acepta varios a la vez.
138     expect(inputs[0].getAttribute('capture')).toBe('environment');
139     expect(inputs[0].multiple).toBeFalse();
140     expect(inputs[1].hasAttribute('capture')).toBeFalse();
141     expect(inputs[1].multiple).toBeTrue();
142 });
143
144 it('renderiza miniaturas al agregar una imagen (desde cualquiera de los dos inputs) y permite quitarlas', (done) => {
145     const campo: CampoFormulario = { clave: 'imagenesEkgrLaboratorios', etiqueta: 'Imágenes', tipo: 'imagenes' };
146     const control = new FormControl<string[]>([]);

```

```

147     fixture.componentInstance.campo = campo;
148     fixture.componentInstance.control = control;
149     fixture.detectChanges();
150
151     const input: HTMLInputElement = fixture.nativeElement.querySelector('input[type="file"]');
152     expect(input).toBeTruthy();
153
154     const archivo = new File(['contenido'], 'ekg.png', { type: 'image/png' });
155     Object.defineProperty(input, 'files', { value: [archivo] });
156     input.dispatchEvent(new Event('change'));
157
158     // FileReader es asíncrono incluso en jsdom/karma; esperamos a que el control se actualice.
159     const esperar = setInterval(() => {
160         if (control.value!.length === 1) {
161             clearInterval(esperar);
162             expect(control.value![0]).toContain('data:image/png;base64');
163             fixture.detectChanges();
164
165             const miniaturas = fixture.nativeElement.querySelectorAll('.miniatura');
166             expect(miniaturas.length).toBe(1);
167
168             const quitar: HTMLButtonElement = fixture.nativeElement.querySelector('.quitar-imagen');
169             quitar.click();
170             expect(control.value).toEqual([]);
171             done();
172         }
173     }, 10);
174 });
175
176 it('renderiza un mat-datepicker para tipo "fecha": arranca desde el string guardado y al elegir una fecha vuelve a escribi
    » r un string "YYYY-MM-DD" en el FormControl externo', () => {
177     const campo: CampoFormulario = { clave: 'fecha', etiqueta: 'Fecha', tipo: 'fecha' };
178     const control = new FormControl('2026-03-05');
179     fixture.componentInstance.campo = campo;
180     fixture.componentInstance.control = control;
181     fixture.detectChanges();
182
183     expect(fixture.nativeElement.querySelector('mat-datepicker-toggle')).toBeTruthy();
184     const interno = fixture.componentInstance.controlFechaHora.value!;
185     expect(interno.getFullYear()).toBe(2026);
186     expect(interno.getMonth()).toBe(2); // marzo = índice 2
187     expect(interno.getDate()).toBe(5);
188
189     // Simula al usuario eligiendo una fecha en el calendario.
190     fixture.componentInstance.controlFechaHora.setValue(new Date(2026, 8, 20));
191     expect(control.value).toBe('2026-09-20');
192 });
193
194 it('renderiza un mat-timepicker para tipo "hora": arranca desde el string guardado y al elegir una hora vuelve a escribir
    » un string "HH:mm" en el FormControl externo', () => {
195     const campo: CampoFormulario = { clave: 'horaSalida', etiqueta: 'Hora de salida', tipo: 'hora' };
196     const control = new FormControl('08:05');
197     fixture.componentInstance.campo = campo;
198     fixture.componentInstance.control = control;
199     fixture.detectChanges();
200
201     expect(fixture.nativeElement.querySelector('mat-timepicker-toggle')).toBeTruthy();
202     const interno = fixture.componentInstance.controlFechaHora.value!;
203     expect(interno.getHours()).toBe(8);
204     expect(interno.getMinutes()).toBe(5);
205
206     // Simula al usuario eligiendo una hora en el selector.
207     const elegida = new Date(2000, 0, 1);
208     elegida.setHours(23, 45, 0, 0);
209     fixture.componentInstance.controlFechaHora.setValue(elegida);
210     expect(control.value).toBe('23:45');
211 });
212
213 it('"Tomar foto" activa la cámara en vivo (getUserMedia) y muestra el <video> con los botones Capturar/Cancelar', async ()
    => {
214     const pistaFalsa = jasmine.createSpyObj('MediaStreamTrack', ['stop']);
215     const streamFalso = { getTracks: () => [pistaFalsa] } as unknown as MediaStream;
216     spyOn(navigator.mediaDevices, 'getUserMedia').and.resolveTo(streamFalso);
217

```

```

218     const campo: CampoFormulario = { clave: 'imagenesEkgRxLaboratorios', etiqueta: 'Imágenes', tipo: 'imagenes' };
219     fixture.componentInstance.campo = campo;
220     fixture.componentInstance.control = new FormControl<string[]>([]);
221     fixture.detectChanges();
222
223     const botonTomarFoto: HTMLButtonElement = fixture.nativeElement.querySelector('.acciones-imagenes button');
224     botonTomarFoto.click();
225     await fixture.whenStable();
226     fixture.detectChanges();
227
228     expect(navigator.mediaDevices.getUserMedia).toHaveBeenCalled({ video: { facingMode: 'environment' } });
229     expect(fixture.componentInstance.mostrandoCamara).toBeTrue();
230     expect(fixture.nativeElement.querySelector('.camara video')).toBeTruthy();
231     expect(fixture.nativeElement.querySelector('.acciones-imagenes')).toBeFalsy();
232
233     const botonCancelar: HTMLButtonElement = fixture.nativeElement.querySelector('.camara-acciones button:last-child');
234     botonCancelar.click();
235     fixture.detectChanges();
236
237     expect(pistaFalsa.stop).toHaveBeenCalled();
238     expect(fixture.componentInstance.mostrandoCamara).toBeFalse();
239 });
240
241 it('"Capturar" agrega la foto tomada de la cámara en vivo al arreglo del control y cierra la cámara', async () => {
242     const pistaFalsa = jasmine.createSpyObj('MediaStreamTrack', ['stop']);
243     const streamFalso = { getTracks: () => [pistaFalsa] } as unknown as MediaStream;
244     spyOn(navigator.mediaDevices, 'getUserMedia').and.resolveTo(streamFalso);
245
246     const campo: CampoFormulario = { clave: 'imagenesEkgRxLaboratorios', etiqueta: 'Imágenes', tipo: 'imagenes' };
247     const control = new FormControl<string[]>([]);
248     fixture.componentInstance.campo = campo;
249     fixture.componentInstance.control = control;
250     fixture.detectChanges();
251
252     fixture.nativeElement.querySelector('.acciones-imagenes button').click();
253     await fixture.whenStable();
254     fixture.detectChanges();
255
256     const botonCapturar: HTMLButtonElement = fixture.nativeElement.querySelector('.camara-acciones button:first-child');
257     botonCapturar.click();
258     fixture.detectChanges();
259
260     expect(control.value!.length).toBe(1);
261     expect(control.value![0]).toContain('data:image/jpeg;base64');
262     expect(pistaFalsa.stop).toHaveBeenCalled();
263     expect(fixture.componentInstance.mostrandoCamara).toBeFalse();
264 });
265
266 it('si getUserMedia rechaza (permiso denegado), muestra un mensaje de error en vez de la vista previa', async () => {
267     spyOn(navigator.mediaDevices, 'getUserMedia').and.rejectWith(new Error('Permiso denegado'));
268
269     const campo: CampoFormulario = { clave: 'imagenesEkgRxLaboratorios', etiqueta: 'Imágenes', tipo: 'imagenes' };
270     fixture.componentInstance.campo = campo;
271     fixture.componentInstance.control = new FormControl<string[]>([]);
272     fixture.detectChanges();
273
274     fixture.nativeElement.querySelector('.acciones-imagenes button').click();
275     await fixture.whenStable();
276     fixture.detectChanges();
277
278     expect(fixture.componentInstance.mostrandoCamara).toBeFalse();
279     expect(fixture.nativeElement.textContent).toContain('No se pudo activar la cámara');
280 });
281
282 it('si el navegador no soporta getUserMedia, "Tomar foto" recae en el input nativo con "capture"', async () => {
283     spyOnProperty(navigator, 'mediaDevices').and.returnValue(undefined as unknown as MediaDevices);
284
285     const campo: CampoFormulario = { clave: 'imagenesEkgRxLaboratorios', etiqueta: 'Imágenes', tipo: 'imagenes' };
286     fixture.componentInstance.campo = campo;
287     fixture.componentInstance.control = new FormControl<string[]>([]);
288     fixture.detectChanges();
289
290     const entradaFallback: HTMLInputElement = fixture.nativeElement.querySelector('input[capture="environment"]');
291     const espia = spyOn(entradaFallback, 'click');

```

```

292
293     fixture.nativeElement.querySelector('.acciones-imagenes button').click();
294     await fixture.whenStable();
295
296     expect(espia).toHaveBeenCalled();
297     expect(fixture.componentInstance.mostrandoCamara).toBeFalse();
298   });
299
300   it('un campo "fecha"/"hora" sin valor guardado arranca con el datepicker/timepicker interno en null', () => {
301     const campo: CampoFormulario = { clave: 'fecha', etiqueta: 'Fecha', tipo: 'fecha' };
302     const control = new FormControl('');
303     fixture.componentInstance.campo = campo;
304     fixture.componentInstance.control = control;
305     fixture.detectChanges();
306
307     expect(fixture.componentInstance.controlFechaHora.value).toBeNull();
308   });
309
310   it('tipo "firma" permite dibujar con el dedo/mouse/lápiz óptico (Pointer Events) y guarda el trazo como PNG al soltar', ()
311   => {
312     const campo: CampoFormulario = { clave: 'firmaQuienEntrega', etiqueta: 'Firma de quien entrega al paciente', tipo: 'firm
313     a' };
314     const control = new FormControl('');
315     fixture.componentInstance.campo = campo;
316     fixture.componentInstance.control = control;
317     fixture.detectChanges();
318
319     expect(fixture.nativeElement.textContent).toContain('Firma de quien entrega al paciente');
320     const lienzo: HTMLCanvasElement = fixture.nativeElement.querySelector('.lienzo-firma');
321     expect(lienzo).toBeTruthy();
322     expect(control.value).toBe('');
323
324     const rect = lienzo.getBoundingClientRect();
325     lienzo.dispatchEvent(new PointerEvent('pointerdown', { clientX: rect.left + 10, clientY: rect.top + 10 }));
326     lienzo.dispatchEvent(new PointerEvent('pointermove', { clientX: rect.left + 50, clientY: rect.top + 50 }));
327     lienzo.dispatchEvent(new PointerEvent('pointerup'));
328
329     expect(control.value).toContain('data:image/png;base64');
330   });
331
332   it('detecta el tipo de puntero (lápiz óptico de tableta, dedo o mouse) y lo muestra en pantalla', () => {
333     const campo: CampoFormulario = { clave: 'firmaQuienEntrega', etiqueta: 'Firma', tipo: 'firma' };
334     const control = new FormControl('');
335     fixture.componentInstance.campo = campo;
336     fixture.componentInstance.control = control;
337     fixture.detectChanges();
338
339     expect(fixture.nativeElement.querySelector('.detector-puntero')).toBeFalsy();
340
341     const lienzo: HTMLCanvasElement = fixture.nativeElement.querySelector('.lienzo-firma');
342     const rect = lienzo.getBoundingClientRect();
343     lienzo.dispatchEvent(new PointerEvent('pointerdown', { clientX: rect.left + 10, clientY: rect.top + 10, pointerType: 'pe
344     n', pressure: 0.8 }));
345     fixture.detectChanges();
346
347     expect(fixture.componentInstance.tipoPuntero).toBe('pen');
348     expect(fixture.nativeElement.querySelector('.detector-puntero').textContent).toContain('Lápiz óptico / tableta detectada
349     ');
350
351     lienzo.dispatchEvent(new PointerEvent('pointerup'));
352   });
353
354   it('"Borrar firma" limpia el lienzo y vacía el control', () => {
355     const campo: CampoFormulario = { clave: 'firmaQuienEntrega', etiqueta: 'Firma', tipo: 'firma' };
356     const control = new FormControl('data:image/png;base64,ALGOYAFIRMADO');
357     fixture.componentInstance.campo = campo;
358     fixture.componentInstance.control = control;
359     fixture.detectChanges();
360
361     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('.acciones-firma button');
362     expect(boton.textContent).toContain('Borrar firma');
363     boton.click();
364
365     expect(control.value).toBe('');

```

```
362   });
363
364   it('si el control ya trae una firma guardada, no la borra ni la modifica al inicializar el lienzo', () => {
365     const campo: CampoFormulario = { clave: 'firmaQuienEntrega', etiqueta: 'Firma', tipo: 'firma' };
366     const firmaGuardada = 'data:image/png;base64,iVBORw0KGgoAAAANSUhEugAAAAEAAAABCAQAAC1HawCAAAAC01EQVR42mNk+A8AAQUBAScY42Y
    » AAAAASUVORK5CYII=';
367     const control = new FormControl(firmaGuardada);
368     fixture.componentInstance.campo = campo;
369     fixture.componentInstance.control = control;
370
371     expect(() => fixture.detectChanges()).not.toThrow();
372     expect(control.value).toBe(firmaGuardada);
373   });
374 });
```



```

1 import { AfterViewInit, Component, ElementRef, Input, OnDestroy, ViewChild } from '@angular/core';
2 import { FormControl, FormsModule, ReactiveFormsModule } from '@angular/forms';
3 import { CommonModule } from '@angular/common';
4 import { Subscription } from 'rxjs';
5 import { MatFormFieldModule } from '@angular/material/form-field';
6 import { MatInputModule } from '@angular/material/input';
7 import { MatSelectModule } from '@angular/material/select';
8 import { MatDatepickerModule } from '@angular/material/datepicker';
9 import { MatTimepickerModule } from '@angular/material/timepicker';
10 import { CampoFormulario } from '../../core/models/campo-formulario.model';
11
12 /** 'YYYY-MM-DD' (el mismo formato que ya guardaba <input type="date">) -> Date local, o null. */
13 function textoAFecha(valor: unknown): Date | null {
14   if (typeof valor !== 'string' || !valor) return null;
15   const [anio, mes, dia] = valor.split('-').map(Number);
16   if (!anio || !mes || !dia) return null;
17   return new Date(anio, mes - 1, dia);
18 }
19
20 /** Date -> 'YYYY-MM-DD' local (evita el corrimiento de día de toISOString() en huso horario negativo). */
21 function fechaATexto(fecha: Date | null): string {
22   if (!fecha) return '';
23   const anio = fecha.getFullYear();
24   const mes = String(fecha.getMonth() + 1).padStart(2, '0');
25   const dia = String(fecha.getDate()).padStart(2, '0');
26   return `${anio}-${mes}-${dia}`;
27 }
28
29 /** 'HH:mm' (el mismo formato que ya guardaba <input type="time">) -> Date de referencia, o null. */
30 function textoAHora(valor: unknown): Date | null {
31   if (typeof valor !== 'string' || !valor) return null;
32   const [horas, minutos] = valor.split(':').map(Number);
33   if (Number.isNaN(horas) || Number.isNaN(minutos)) return null;
34   const fecha = new Date(2000, 0, 1);
35   fecha.setHours(horas, minutos, 0, 0);
36   return fecha;
37 }
38
39 /** Date -> 'HH:mm'. */
40 function horaATexto(fecha: Date | null): string {
41   if (!fecha) return '';
42   const horas = String(fecha.getHours()).padStart(2, '0');
43   const minutos = String(fecha.getMinutes()).padStart(2, '0');
44   return `${horas}:${minutos}`;
45 }
46
47 @Component({
48   selector: 'app-campo-formulario',
49   standalone: true,
50   imports: [
51     CommonModule, FormsModule, ReactiveFormsModule,
52     MatFormFieldModule, MatInputModule, MatSelectModule, MatDatepickerModule, MatTimepickerModule,
53   ],
54   template: `
55     <div class="campo" [ngSwitch]="campo.tipo">
56
57       <mat-form-field *ngSwitchCase="'texto'" appearance="outline" class="campo-ancho-completo">
58         <mat-label>{{ campo.etiqueta }}</mat-label>
59         <input matInput type="text" [formControl]="control" />
60       </mat-form-field>
61
62       <mat-form-field *ngSwitchCase="'numero'" appearance="outline" class="campo-ancho-medio">
63         <mat-label>{{ campo.etiqueta }}</mat-label>
64         <input matInput type="number" inputmode="decimal" [formControl]="control" />
65         <span matTextSuffix *ngIf="campo.sufijo">{{ campo.sufijo }}</span>
66       </mat-form-field>
67
68       <mat-form-field *ngSwitchCase="'fecha'" appearance="outline" class="campo-ancho-medio">
69         <mat-label>{{ campo.etiqueta }}</mat-label>
70         <input matInput [matDatepicker]="selectorFecha" [formControl]="controlFechaHora" />
71         <mat-datepicker-toggle matIconSuffix [for]="selectorFecha"></mat-datepicker-toggle>
72         <mat-datepicker #selectorFecha></mat-datepicker>
73       </mat-form-field>
74

```

```

75     <mat-form-field *ngSwitchCase="'hora'" appearance="outline" class="campo-ancho-medio">
76       <mat-label>{{ campo.etiqueta }}</mat-label>
77       <input matInput [matTimepicker]="selectorHora" [formControl]="controlFechaHora" />
78       <mat-timepicker-toggle matIconSuffix [for]="selectorHora"></mat-timepicker-toggle>
79       <mat-timepicker #selectorHora interval="5m"></mat-timepicker>
80     </mat-form-field>
81
82     <mat-form-field *ngSwitchCase="'textarea'" appearance="outline" class="campo-ancho-completo">
83       <mat-label>{{ campo.etiqueta }}</mat-label>
84       <textarea matInput rows="3" [formControl]="control"></textarea>
85     </mat-form-field>
86
87     <button
88       *ngSwitchCase="'checkbox'"
89       type="button"
90       class="chip"
91       [class.chip-activo]="control.value === true"
92       (click)="alternarBooleano()">
93       {{ campo.etiqueta }}
94     </button>
95
96     <div *ngSwitchCase="'checkbox-grupo'" class="grupo-opciones">
97       <span class="grupo-titulo">{{ campo.etiqueta }}</span>
98       <div class="chips">
99         <button
100           *ngFor="let opcion of campo.opciones"
101           type="button"
102           class="chip"
103           [class.chip-activo]="estaMarcada(opcion)"
104           (click)="alternarOpcion(opcion)">
105           {{ opcion }}
106         </button>
107       </div>
108     </div>
109
110     <div *ngSwitchCase="'radio-grupo'" class="grupo-opciones">
111       <span class="grupo-titulo">{{ campo.etiqueta }}</span>
112       <div class="chips">
113         <button
114           *ngFor="let opcion of campo.opciones"
115           type="button"
116           class="chip"
117           [class.chip-activo]="control.value === opcion"
118           (click)="seleccionarRadio(opcion)">
119           {{ opcion }}
120         </button>
121       </div>
122     </div>
123
124     <mat-form-field *ngSwitchCase="'select'" appearance="outline" class="campo-ancho-medio">
125       <mat-label>{{ campo.etiqueta }}</mat-label>
126       <mat-select [formControl]="control">
127         <mat-option *ngFor="let opcion of campo.opciones" [value]="opcion">{{ opcion }}</mat-option>
128       </mat-select>
129     </mat-form-field>
130
131     <div *ngSwitchCase="'imagenes'" class="grupo-imagenes">
132       <span class="grupo-titulo">{{ campo.etiqueta }}</span>
133       <div class="miniaturas" *ngIf="imagenes().length > 0">
134         <div class="miniatura" *ngFor="let imagen of imagenes(); let i = index">
135           <img [src]="imagen" alt="" />
136           <button type="button" class="quitar-imagen" (click)="quitarImagen(i)" aria-label="Quitar imagen"> </button>
137         </div>
138       </div>
139     <!-- Vista previa en vivo de la cámara mientras está activa. -->
140     <div class="camara" *ngIf="mostrandoCamara">
141       <video #videoCamara autoplay playsinline muted></video>
142       <div class="camara-acciones">
143         <button type="button" class="chip chip-activo" (click)="capturarFoto()"> Capturar</button>
144         <button type="button" class="chip" (click)="cerrarCamara()"> Cancelar</button>
145       </div>
146     </div>
147
148     <p class="error-camara" *ngIf="errorCamara">{{ errorCamara }}</p>

```

```

149
150     <div class="acciones-imagenes" *ngIf="!mostrandoCamara">
151         <button type="button" class="chip" (click)="abrirCamara()"> Tomar foto</button>
152         <button type="button" class="chip" (click)="entradaArchivo.click()"> Subir imagen</button>
153     </div>
154     <!--
155     "Tomar foto" activa la cámara del dispositivo en vivo (getUserMedia)
156     en vez de delegar al selector nativo del sistema operativo – así el
157     usuario ve la vista previa y decide cuándo capturar sin salir de la
158     app. Si el navegador no soporta getUserMedia (o el contexto no es
159     seguro: ni HTTPS ni localhost), abrirCamara() recae en este input
160     oculto con "capture", que en celular sigue abriendo la cámara nativa.
161     El de subir archivo es independiente (galeria/archivos, admite
162     varias imágenes a la vez).
163     -->
164     <input #entradaCamaraFallback type="file" accept="image/*" capture="environment" hidden (change)="agregarImagenes($event)" />
165     <input #entradaArchivo type="file" accept="image/*" multiple hidden (change)="agregarImagenes($event)" />
166 </div>
167
168 <!--
169 Firma dibujada con el dedo, mouse o lápiz óptico (Pointer Events cubre
170 los tres por igual). El lienzo mantiene su propio tamaño en píxeles
171 fijo (width/height) para que la imagen final salga nítida; se dibuja
172 a escala con getBoundingClientRect() para que el trazo quede alineado
173 aunque el lienzo se vea más angosto en pantallas chicas.
174 -->
175 <div *ngSwitchCase="'firma'" class="campo-firma">
176     <span class="grupo-titulo">{{ campo.etiqueta }}</span>
177     <canvas
178         #lienzoFirma
179         class="lienzo-firma"
180         width="360"
181         height="120"
182         (pointerdown)="iniciarTrazo($event)"
183         (pointermove)="continuarTrazo($event)"
184         (pointerup)="terminarTrazo()"
185         (pointerleave)="terminarTrazo()"
186         (pointercancel)="terminarTrazo()"
187     ></canvas>
188     <div class="acciones-firma">
189         <button type="button" class="chip" (click)="limpiarFirma()">Borrar firma</button>
190         <span class="detector-puntero" *ngIf="tipoPuntero">{{ etiquetaPuntero() }}</span>
191     </div>
192 </div>
193
194 </div>
195 `
196 styles: [`
197     .campo { margin-bottom: 4px; }
198     .campo-ancho-completo, .campo-ancho-medio { width: 100%; }
199     .grupo-opciones { display: flex; flex-direction: column; gap: 6px; margin-bottom: 10px; }
200     .grupo-titulo { font-weight: 600; font-size: 0.85rem; color: var(--umam-header-bg, #1892d3); text-transform: uppercase;
201     letter-spacing: 0.02em; }
202     .chips { display: flex; flex-wrap: wrap; gap: 8px; }
203     .chip {
204         min-height: 44px;
205         padding: 8px 16px;
206         border: 1.5px solid var(--umam-section-border, #b9def2);
207         border-radius: 999px;
208         background: #fff;
209         color: #333;
210         font-size: 0.9rem;
211         font-family: inherit;
212         cursor: pointer;
213         transition: background 0.15s, border-color 0.15s;
214         -webkit-tap-highlight-color: transparent;
215     }
216     .chip:active { background: var(--umam-section-bg, #dbeef9); }
217     .chip-activo {
218         background: var(--umam-header-bg, #1892d3);
219         border-color: var(--umam-header-bg, #1892d3);
220         color: #fff;
221         font-weight: 600;

```

```

221     }
222     .grupo-imagenes { display: flex; flex-direction: column; gap: 6px; margin-bottom: 10px; }
223     .acciones-imagenes { display: flex; flex-wrap: wrap; gap: 8px; }
224     .camara { display: flex; flex-direction: column; gap: 6px; }
225     .camara video { width: 100%; max-width: 360px; max-height: 280px; border-radius: 8px; background: #000; object-fit: cover; }
226   r; }
227   .camara-acciones { display: flex; flex-wrap: wrap; gap: 8px; }
228   .error-camara { color: #c0392b; font-size: 0.85rem; margin: 0; }
229   .miniaturas { display: flex; flex-wrap: wrap; gap: 8px; }
230   .miniatura { position: relative; width: 72px; height: 72px; }
231   .miniatura img { width: 100%; height: 100%; object-fit: cover; border-radius: 6px; border: 1px solid var(--umam-section-
232     border, #b9def2); }
233   .quitar-imagen {
234     position: absolute; top: -6px; right: -6px; width: 20px; height: 20px;
235     border-radius: 50%; border: none; background: #c0392b; color: #fff;
236     font-size: 11px; line-height: 1; cursor: pointer;
237   }
238   .campo-firma { display: flex; flex-direction: column; gap: 6px; margin-bottom: 10px; }
239   .lienzo-firma {
240     width: 100%; max-width: 360px; height: 120px;
241     border: 1.5px dashed var(--umam-section-border, #b9def2); border-radius: 8px;
242     background: #fff; touch-action: none; cursor: crosshair;
243   }
244   .acciones-firma { display: flex; flex-wrap: wrap; align-items: center; gap: 8px; }
245   .detector-puntero { font-size: 0.8rem; color: #2e7d32; font-weight: 600; }
246   `},
247   },
248   })
249   export class CampoFormularioComponent implements AfterViewInit, OnDestroy {
250     private _campo!: CampoFormulario;
251     private _control!: FormControl;
252     /**
253      * Control interno tipo Date que alimenta mat-datepicker / mat-timepicker.
254      * El control externo (this.control) sigue guardando 'YYYY-MM-DD' / 'HH:mm'
255      * como texto plano – igual que antes de agregar los selectores de
256      * Material – para no tocar el modelo, el autosave a localStorage ni la
257      * exportación a PDF/Excel, que siguen leyendo/escribiendo esos strings tal
258      * cual. Se sincroniza en los setters (no en ngOnChanges) para que también
259      * funcione cuando las specs asignan `campo`/`control` directo a la
260      * instancia, sin pasar por un binding de plantilla.
261      */
262     controlFechaHora = new FormControl<Date | null>(null);
263     private suscripcionFechaHora?: Subscription;
264     /** Vista previa de cámara en vivo para el tipo 'imagenes' (ver abrirCamara()). */
265     mostrandoCamara = false;
266     errorCamara = '';
267     @ViewChild('videoCamara') private videoCamaraRef?: ElementRef<HTMLVideoElement>;
268     @ViewChild('entradaCamaraFallback') private entradaCamaraFallbackRef!: ElementRef<HTMLInputElement>;
269     private streamCamara?: MediaStream;
270     /** Firma dibujada (tipo 'firma'): trazo en curso con dedo/mouse/lápiz óptico. */
271     @ViewChild('lienzoFirma') private lienzoFirmaRef?: ElementRef<HTMLCanvasElement>;
272     private dibujandoTrazo = false;
273     private ultimoPunto?: { x: number; y: number };
274     /** Tipo de dispositivo del último trazo – confirma en pantalla que se detectó la tableta/lápiz. */
275     tipoPuntero: 'pen' | 'touch' | 'mouse' | null = null;
276     @Input() set campo(valor: CampoFormulario) {
277       this._campo = valor;
278       this.sincronizarFechaHora();
279     }
280     get campo(): CampoFormulario {
281       return this._campo;
282     }
283     @Input() set control(valor: FormControl) {
284       this._control = valor;
285       this.sincronizarFechaHora();
286     }
287     get control(): FormControl {
288       return this._control;
289     }
290   }
291 }
292

```

```

293   ngAfterViewInit(): void {
294     // Si ya había una firma guardada (se abrió un trámite existente para
295     // seguir editándolo), se precarga en el lienzo para que se vea "firmado"
296     // en vez de aparecer en blanco.
297     this.cargarFirmaGuardada();
298   }
299
300   ngOnDestroy(): void {
301     this.suscripcionFechaHora?.unsubscribe();
302     this.cerrarCamara();
303   }
304
305   private sincronizarFechaHora(): void {
306     if (!this._campo || !this._control) return;
307     if (this._campo.tipo !== 'fecha' && this._campo.tipo !== 'hora') return;
308
309     this.suscripcionFechaHora?.unsubscribe();
310     const aFecha = this._campo.tipo === 'fecha' ? textoAFecha : textoAHora;
311     const aTexto = this._campo.tipo === 'fecha' ? fechaATexto : horaATexto;
312
313     this.controlFechaHora.setValue(aFecha(this._control.value), { emitEvent: false });
314     this.suscripcionFechaHora = this.controlFechaHora.valueChanges.subscribe(valor => {
315       this._control.setValue(aTexto(valor));
316     });
317   }
318
319   estaMarcada(opcion: string): boolean {
320     const valor = (this.control.value ?? []) as string[];
321     return valor.includes(opcion);
322   }
323
324   alternarOpcion(opcion: string): void {
325     const valor = [...((this.control.value ?? []) as string[])];
326     const indice = valor.indexOf(opcion);
327     if (indice >= 0) {
328       valor.splice(indice, 1);
329     } else {
330       valor.push(opcion);
331     }
332     this.control.setValue(valor);
333   }
334
335   seleccionarRadio(opcion: string): void {
336     // Tocar el chip ya activo lo deselectiona: en el papel muchas casillas
337     // pueden quedar sin marcar, así que el radio necesita poder vaciarse.
338     this.control.setValue(this.control.value === opcion ? '' : opcion);
339   }
340
341   alternarBooleano(): void {
342     this.control.setValue(this.control.value !== true);
343   }
344
345   imagenes(): string[] {
346     return (this.control.value ?? []) as string[];
347   }
348
349   /**
350    * Convierte cada archivo elegido a un data URI base64 y lo agrega al arreglo
351    * del control. Se guarda como base64 (no como File) porque todo el registro
352    * viaja tal cual a localStorage vía JSON.stringify – un File no sobrevive
353    * esa serialización.
354    */
355   agregarImagenes(evento: Event): void {
356     const input = evento.target as HTMLInputElement;
357     const archivos = Array.from(input.files ?? []);
358     for (const archivo of archivos) {
359       const lector = new FileReader();
360       lector.onload = () => {
361         this.control.setValue([...this.imagenes(), String(lector.result)]);
362       };
363       lector.readAsDataURL(archivo);
364     }
365     input.value = '';
366   }

```

```

367
368 quitarImagen(indice: number): void {
369     const valor = [...this.imagenes()];
370     valor.splice(indice, 1);
371     this.control.setValue(valor);
372 }
373
374 /**
375  * Activa la cámara del dispositivo en vivo y la muestra en un <video> dentro
376  * del propio campo. Si el navegador no soporta getUserMedia (o el contexto
377  * no es seguro – ni HTTPS ni localhost), o si el usuario niega el permiso,
378  * recae en el input con "capture", que en celular sigue abriendo la cámara
379  * nativa del sistema.
380  */
381 async abrirCamara(): Promise<void> {
382     this.errorCamara = '';
383     if (!navigator.mediaDevices?.getUserMedia) {
384         this.entradaCamaraFallbackRef?.nativeElement.click();
385         return;
386     }
387     try {
388         this.streamCamara = await navigator.mediaDevices.getUserMedia({ video: { facingMode: 'environment' } });
389         this.mostrandoCamara = true;
390         // El <video> recién aparece en el DOM al activar mostrandoCamara (*ngIf);
391         // se espera al siguiente tick para que @ViewChild ya lo haya capturado.
392         setTimeout(() => {
393             if (this.videoCamaraRef) {
394                 this.videoCamaraRef.nativeElement.srcObject = this.streamCamara!;
395             }
396         });
397     } catch (error) {
398         console.error('No se pudo activar la cámara', error);
399         this.errorCamara = 'No se pudo activar la cámara. Revisa los permisos del navegador o usa "Subir imagen."';
400     }
401 }
402
403 /** Toma la imagen actual del <video> en vivo y la agrega como una imagen más. */
404 capturarFoto(): void {
405     const video = this.videoCamaraRef?.nativeElement;
406     if (!video) return;
407     const lienzo = document.createElement('canvas');
408     lienzo.width = video.videoWidth || 1;
409     lienzo.height = video.videoHeight || 1;
410     lienzo.getContext('2d')?.drawImage(video, 0, 0, lienzo.width, lienzo.height);
411     this.control.setValue([...this.imagenes(), lienzo.toDataURL('image/jpeg', 0.85)]);
412     this.cerrarCamara();
413 }
414
415 cerrarCamara(): void {
416     this.streamCamara?.getTracks().forEach(pista => pista.stop());
417     this.streamCamara = undefined;
418     this.mostrandoCamara = false;
419 }
420
421 private cargarFirmaGuardada(): void {
422     const canvas = this.lienzoFirmaRef?.nativeElement;
423     const valor = this._control?.value;
424     if (!canvas || typeof valor !== 'string' || !valor) return;
425     const contexto = canvas.getContext('2d');
426     const imagen = new Image();
427     imagen.onload = () => contexto?.drawImage(imagen, 0, 0, canvas.width, canvas.height);
428     imagen.src = valor;
429 }
430
431 iniciarTrazo(evento: PointerEvent): void {
432     this.dibujandoTrazo = true;
433     this.tipoPuntero = (evento.pointerType as 'pen' | 'touch' | 'mouse') || 'mouse';
434     this.ultimoPunto = this.puntoDelEvento(evento);
435     // setPointerCapture ata el resto del trazo a este lienzo aunque el puntero
436     // se mueva rápido y salga de sus límites a medio trazo – común en
437     // tabletas con lápiz óptico, donde el cursor puede "saltar" entre eventos.
438     // Puede rechazar el pointerId (p. ej. en pruebas con eventos sintéticos o
439     // en navegadores que no lo soportan del todo); no es crítico, se ignora.
440     try {

```

```

441     this.lienzoFirmaRef?.nativeElement.setPointerCapture?.(evento.pointerId);
442   } catch {
443     // sin captura, el trazo sigue funcionando igual mientras el puntero no salga del lienzo.
444   }
445   evento.preventDefault();
446 }
447
448 continuarTrazo(evento: PointerEvent): void {
449   if (!this.dibujandoTrazo || !this.ultimoPunto) return;
450   const canvas = this.lienzoFirmaRef?.nativeElement;
451   const contexto = canvas?.getContext('2d');
452   if (!canvas || !contexto) return;
453
454   const punto = this.puntoDelEvento(evento);
455   // Cuando el lápiz óptico reporta presión real, el trazo sale más grueso
456   // donde se presiona más fuerte – como con pluma en papel. El mouse/dedo
457   // sin sensor de presión reportan 0.5 mientras están presionados, así que
458   // igual sale un trazo de grosor constante y razonable.
459   const presion = evento.pressure > 0 ? evento.pressure : 0.5;
460   contexto.strokeStyle = '#1a1a1a';
461   contexto.lineWidth = 1.2 + presion * 2.6;
462   contexto.lineCap = 'round';
463   contexto.lineJoin = 'round';
464   contexto.beginPath();
465   contexto.moveTo(this.ultimoPunto.x, this.ultimoPunto.y);
466   contexto.lineTo(punto.x, punto.y);
467   contexto.stroke();
468   this.ultimoPunto = punto;
469   // Evita que el navegador interprete el trazo como gesto de scroll/zoom en celular.
470   evento.preventDefault();
471 }
472
473 terminarTrazo(): void {
474   if (!this.dibujandoTrazo) return;
475   this.dibujandoTrazo = false;
476   this.ultimoPunto = undefined;
477   const canvas = this.lienzoFirmaRef?.nativeElement;
478   if (canvas) this.control.setValue(canvas.toDataURL('image/png'));
479 }
480
481 limpiarFirma(): void {
482   const canvas = this.lienzoFirmaRef?.nativeElement;
483   const contexto = canvas?.getContext('2d');
484   contexto?.clearRect(0, 0, canvas!.width, canvas!.height);
485   this.control.setValue('');
486   this.tipoPuntero = null;
487 }
488
489 etiquetaPuntero(): string {
490   switch (this.tipoPuntero) {
491     case 'pen': return 'Lápiz óptico / tableta detectada';
492     case 'touch': return 'Dedo detectado';
493     default: return 'Mouse detectado';
494   }
495 }
496
497 /** Convierte las coordenadas del puntero (en pantalla) a coordenadas del lienzo. */
498 private puntoDelEvento(evento: PointerEvent): { x: number; y: number } {
499   const canvas = this.lienzoFirmaRef!.nativeElement;
500   const rect = canvas.getBoundingClientRect();
501   return {
502     x: ((evento.clientX - rect.left) / rect.width) * canvas.width,
503     y: ((evento.clientY - rect.top) / rect.height) * canvas.height,
504   };
505 }
506 }

```



```
1 import { ComponentFixture, TestBed } from '@angular/core/testing';
2 import { FormBuilder, FormGroup } from '@angular/forms';
3 import { SeccionPasoComponent } from './seccion-paso.component';
4 import { SECCIONES } from '../../../../../core/data/secciones.data';
5 import { construirFormularioRegistro } from '../../../../../core/forms/construir-formulario';
6 import { crearRegistroVacio } from '../../../../../core/models/registro.model';
7
8 describe('SeccionPasoComponent', () => {
9   let fixture: ComponentFixture<SeccionPasoComponent>;
10   let formularioCompleto: FormGroup;
11
12   beforeEach(async () => {
13     await TestBed.configureTestingModule({
14       imports: [SeccionPasoComponent],
15     }).compileComponents();
16     formularioCompleto = construirFormularioRegistro(new FormBuilder(), crearRegistroVacio());
17     fixture = TestBed.createComponent(SeccionPasoComponent);
18   });
19
20   it('renderiza el título de la sección y un app-campo-formulario por cada campo', () => {
21     const seccion = SECCIONES.find(s => s.clave === 'control');
22     fixture.componentInstance.seccion = seccion;
23     fixture.componentInstance.grupo = formularioCompleto.get('control') as FormGroup;
24     fixture.detectChanges();
25
26     expect(fixture.nativeElement.textContent).toContain('III. Control');
27     const campos = fixture.nativeElement.querySelectorAll('app-campo-formulario');
28     expect(campos.length).toBe(seccion.campos.length);
29   });
30
31   it('renderiza una app-tabla-repetible por cada tabla de la sección', () => {
32     const seccion = SECCIONES.find(s => s.clave === 'evaluacionSecundaria');
33     fixture.componentInstance.seccion = seccion;
34     fixture.componentInstance.grupo = formularioCompleto.get('evaluacionSecundaria') as FormGroup;
35     fixture.detectChanges();
36
37     const tablas = fixture.nativeElement.querySelectorAll('app-tabla-repetible');
38     expect(tablas.length).toBe(seccion.tablas!.length);
39   });
40 });
```



```

1 import { Component, Input } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { FormArray, FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';
4 import { CampoFormularioComponent } from '../campo-formulario/campo-formulario.component';
5 import { TablaRepetibleComponent } from '../tabla-repetible/tabla-repetible.component';
6 import { SeccionFormulario } from '../../core/models/campo-formulario.model';
7
8 @Component({
9   selector: 'app-seccion-paso',
10  standalone: true,
11  imports: [CommonModule, ReactiveFormsModule, CampoFormularioComponent, TablaRepetibleComponent],
12  template: `
13    <section class="paso" [formGroup]="grupo">
14      <h2 class="titulo-seccion">{{ seccion.titulo }}</h2>
15
16      <div class="grid-campos">
17        <div
18          *ngFor="let campo of seccion.campos"
19          class="celda"
20          [class.celda-completa]="esAnchoCompleto(campo.tipo)">
21          <app-campo-formulario [campo]="campo" [control]="controlDeCampo(campo.clave)"></app-campo-formulario>
22        </div>
23      </div>
24
25      <app-tabla-repetible
26        *ngFor="let tabla of seccion.tablas"
27        [tabla]="tabla"
28        [filas]="filasDeTabla(tabla.clave)"
29      ></app-tabla-repetible>
30    </section>
31  `,
32  styles: [`
33    .paso { background: #fff; border: 1px solid var(--umam-section-border, #b9def2); border-radius: 12px; padding: 16px; }
34    .titulo-seccion { margin: 0 0 16px; font-size: 1.15rem; color: var(--umam-header-bg, #1892d3); border-bottom: 2px solid
35    » var(--umam-section-border, #b9def2); padding-bottom: 8px; }
36    .grid-campos { display: grid; grid-template-columns: repeat(auto-fit, minmax(160px, 1fr)); column-gap: 12px; align-items
37    » : start; }
38    .celda-completa { grid-column: 1 / -1; }
39  `],
40 })
41 export class SeccionPasoComponent {
42   @Input() seccion!: SeccionFormulario;
43   @Input() grupo!: FormGroup;
44
45   esAnchoCompleto(tipo: string): boolean {
46     // Los campos cortos (horas, números, fechas, selects) comparten renglón en la
47     // cuadrícula; texto libre y grupos de chips necesitan el ancho completo.
48     return ![ 'hora', 'numero', 'fecha', 'select' ].includes(tipo);
49   }
50
51   controlDeCampo(clave: string): FormControl {
52     return this.grupo.get(clave) as FormControl;
53   }
54
55   filasDeTabla(clave: string): FormArray {
56     return this.grupo.parent!.get(clave) as FormArray;
57   }
58 }

```

```
1 import { ComponentFixture, TestBed } from '@angular/core/testing';
2 import { FormArray, FormBuilder } from '@angular/forms';
3 import { TablaRepetibleComponent } from './tabla-repetible.component';
4 import { TablaRepetible } from '../../core/models/campo-formulario.model';
5
6 describe('TablaRepetibleComponent', () => {
7   let fixture: ComponentFixture<TablaRepetibleComponent>;
8   const fb = new FormBuilder();
9   const tabla: TablaRepetible = {
10     clave: 'vehiculosInvolucrados',
11     titulo: 'Vehículos involucrados',
12     columnas: [
13       { clave: 'tipoMarca', etiqueta: 'Tipo y marca', tipo: 'texto' },
14       { clave: 'placas', etiqueta: 'Placas', tipo: 'texto' },
15     ],
16   };
17
18   beforeEach(async () => {
19     await TestBed.configureTestingModule({
20       imports: [TablaRepetibleComponent],
21     }).compileComponents();
22     fixture = TestBed.createComponent(TablaRepetibleComponent);
23     fixture.componentInstance.tabla = tabla;
24     fixture.componentInstance.filas = fb.array([]);
25   });
26
27   it('empieza sin filas y el botón "agregar fila" agrega una fila con las columnas de la tabla', () => {
28     fixture.detectChanges();
29     expect(fixture.componentInstance.filas.length).toBe(0);
30
31     const boton: HTMLButtonElement = fixture.nativeElement.querySelector('button.agregar-fila');
32     boton.click();
33     fixture.detectChanges();
34
35     expect(fixture.componentInstance.filas.length).toBe(1);
36     const primeraFila = fixture.componentInstance.filas.at(0);
37     expect(primeraFila.get('tipoMarca')).toBeTruthy();
38     expect(primeraFila.get('placas')).toBeTruthy();
39   });
40
41   it('el botón "quitar" elimina la fila correspondiente', () => {
42     fixture.componentInstance.filas.push(fixture.componentInstance['fb'].group({ tipoMarca: [''], placas: [''] }));
43     fixture.detectChanges();
44     expect(fixture.componentInstance.filas.length).toBe(1);
45
46     const botonQuitar: HTMLButtonElement = fixture.nativeElement.querySelector('button.quitar-fila');
47     botonQuitar.click();
48     fixture.detectChanges();
49
50     expect(fixture.componentInstance.filas.length).toBe(0);
51   });
52 });
```

```

1 import { Component, Input } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { AbstractControl, FormArray, FormBuilder, FormControl, FormGroup, ReactiveFormsModule } from '@angular/forms';
4 import { MatButtonModule } from '@angular/material/button';
5 import { MatIconModule } from '@angular/material/icon';
6 import { CampoFormularioComponent } from '../campo-formulario/campo-formulario.component';
7 import { TablaRepetible } from '../../core/models/campo-formulario.model';
8 import { nuevaFilaTabla } from '../../core/forms/construir-formulario';
9
10 @Component({
11   selector: 'app-tabla-repetible',
12   standalone: true,
13   imports: [CommonModule, ReactiveFormsModule, MatButtonModule, MatIconModule, CampoFormularioComponent],
14   template: `
15     <div class="tabla-repetible">
16       <h4>{{ tabla.titulo }}</h4>
17       <div class="fila" *ngFor="let fila of filas.controls; let i = index" [formGroup]="asFormGroup(fila)">
18         <app-campo-formulario
19           *ngFor="let columna of tabla.columnas"
20           [campo]="columna"
21           [control]="controlDeColumna(fila, columna.clave)"
22         ></app-campo-formulario>
23         <button type="button" class="quitar-fila" mat-icon-button (click)="quitarFila(i)" aria-label="Quitar fila">
24           <mat-icon>delete</mat-icon>
25         </button>
26       </div>
27       <button type="button" class="agregar-fila" mat-stroked-button (click)="agregarFila()">
28         <mat-icon>add</mat-icon> Agregar fila
29       </button>
30     </div>
31   `,
32   styles: [
33     .tabla-repetible { margin: 16px 0; }
34     .fila { display: flex; flex-wrap: wrap; gap: 8px; align-items: center; margin-bottom: 8px; }
35   ],
36 })
37 export class TablaRepetibleComponent {
38   @Input() tabla!: TablaRepetible;
39   @Input() filas!: FormArray;
40
41   private fb = new FormBuilder();
42
43   agregarFila(): void {
44     this.filas.push(nuevaFilaTabla(this.fb, this.tabla.columnas));
45   }
46
47   quitarFila(indice: number): void {
48     this.filas.removeAt(indice);
49   }
50
51   asFormGroup(control: unknown): FormGroup {
52     return control as FormGroup;
53   }
54
55   controlDeColumna(fila: AbstractControl, clave: string): FormControl {
56     return (fila as FormGroup).get(clave) as FormControl;
57   }
58 }

```

```
1 @use '@angular/material' as mat;
2
3 // Theme driven by Angular Material's M3 blue palette (current Material major
4 // version expects M3 palettes, not a hand-rolled M2 map). The custom
5 // properties below are the actual colors sampled from the UMAM paper form
6 // (celeste/blue section headers and table fills) and are what the rest of
7 // the app's raw CSS (print view, section headers) reads from – keep them in
8 // sync with $blue-palette's 500-ish tone if the Material theme ever changes.
9 $umam-theme: mat.define-theme((
10   color: (
11     theme-type: light,
12     primary: mat.$blue-palette,
13     tertiary: mat.$cyan-palette,
14   ),
15   density: (scale: 0),
16 ));
17
18 :root {
19   @include mat.all-component-themes($umam-theme);
20   --umam-header-bg: #1892d3;
21   --umam-header-fg: #ffffff;
22   --umam-section-border: #b9def2;
23 }
```

```
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8">
5   <title>AppTmp</title>
6   <base href="/">
7   <meta name="viewport" content="width=device-width, initial-scale=1">
8   <link rel="icon" type="image/x-icon" href="favicon.ico">
9   <link href="https://fonts.googleapis.com/css2?family=Roboto:wght@300;400;500&display=swap" rel="stylesheet">
10  <link href="https://fonts.googleapis.com/icon?family=Material+Icons" rel="stylesheet">
11 </head>
12 <body>
13   <app-root></app-root>
14 </body>
15 </html>
```

```
1 import { bootstrapApplication } from '@angular/platform-browser';
2 import { appConfig } from './app/app.config';
3 import { App } from './app/app';
4
5 bootstrapApplication(App, appConfig)
6   .catch((err) => console.error(err));
```

```
1 @use './app/shared/theme/umam-theme';
2
3 // Angular Material theming is defined once, in ./app/shared/theme/umam-theme.scss
4 // (imported above), which is the single source of truth for the app's Material
5 // theme (primary: blue, tertiary: cyan, sampled from the UMAM paper form colors).
6 // Learn more about theming at https://material.angular.dev/guide/theming
7
8 body {
9   // Default the application to a light color theme. This can be changed to
10  // `dark` to enable the dark color theme, or to `light dark` to defer to the
11  // user's system settings.
12  color-scheme: light;
13
14  // Set a default background, font and text colors for the application using
15  // Angular Material's system-level CSS variables. Learn more about these
16  // variables at https://material.angular.dev/guide/system-variables
17  background-color: var(--mat-sys-surface);
18  color: var(--mat-sys-on-surface);
19  font: var(--mat-sys-body-medium);
20
21  // Reset the user agent margin.
22  margin: 0;
23 }
24 /* You can add global styles to this file, and also import other style files */
25
26 html, body { height: 100%; }
27 body { margin: 0; font-family: Roboto, "Helvetica Neue", sans-serif; }
28
29 @media print {
30   body {
31     background: none;
32     print-color-adjust: exact;
33     -webkit-print-color-adjust: exact;
34   }
35   @page { margin: 12mm; }
36 }
```

```
1 # Plantilla de configuración. Cópiala como .env (que no se sube a git)
2 # y ajusta los valores según el entorno.
3
4 # Puerto donde escucha la API.
5 PORT=3000
6 # Origen del frontend autorizado por CORS.
7 CORS_ORIGIN=http://localhost:4200
8 # Tamaño máximo del JSON de un registro (incluye firmas e imágenes en base64).
9 BODY_LIMIT=50mb
10
11 # Conexión a PostgreSQL (coincide con docker-compose.yml).
12 DB_HOST=localhost
13 DB_PORT=5432
14 DB_USER=umam
15 DB_PASSWORD=umam
16 DB_NAME=umam
17 # true: TypeORM crea/actualiza las tablas al arrancar.
18 # En producción: false y usar migraciones.
19 DB_SYNCHRONIZE=true
```



```
1 node_modules
2 dist
3 coverage
4 .env
```

```
1 {  
2   "singleQuote": true,  
3   "trailingComma": "all"  
4 }
```

```
1 # Backend UMAM (NestJS + PostgreSQL)
2
3 API REST que persiste los "Registros de Atención Prehospitalaria" del
4 frontend Angular. Cada sección del formulario se guarda como una columna
5 JSONB (los campos se definen en `src/app/core/data/secciones.data.ts` del
6 frontend), así que agregar campos al formulario no requiere migraciones.
7
8 ## Uso
9
10 ```powershell
11 cd backend
12 npm install
13 copy .env.example .env
14 docker compose up -d      # Postgres 17 en localhost:5432 (usuario/clave/bd: umam)
15 npm run start:dev         # API en http://localhost:3000/api
16 ```
17
18 Con `DB_SYNCHRONIZE=true` (valor por defecto) TypeORM crea la tabla
19 `registros` al arrancar. En producción ponlo en `false` y usa migraciones.
20
21 ## Endpoints
22
23 | Método | Ruta | Descripción |
24 |-----|-----|-----|
25 | GET | `/api/registros` | Lista todos (más antiguo → más reciente) |
26 | GET | `/api/registros/:id` | Obtiene uno (404 si no existe) |
27 | PUT | `/api/registros/:id` | Crea o actualiza (el id lo genera el frontend) |
28 | DELETE | `/api/registros/:id` | Elimina (204) |
29
30 Las firmas e imágenes viajan como data URLs dentro del JSON; el límite del
31 body se ajusta con `BODY_LIMIT` (por defecto `50mb`).
32
33 ## Pruebas
34
35 ```powershell
36 npm run test:e2e # requiere Postgres corriendo
37 ```
```

```
1 # Base de datos PostgreSQL para desarrollo local.
2 # Levantar: docker compose up -d      Apagar: docker compose down
3 # Los datos sobreviven a reinicios gracias al volumen umam_pgdata
4 # (docker compose down -v los borra).
5 services:
6   postgres:
7     image: postgres:17
8     container_name: umam_postgres
9     # Usuario, contraseña y base de datos que se crean al primer arranque;
10    # deben coincidir con DB_USER / DB_PASSWORD / DB_NAME del .env.
11    environment:
12      POSTGRES_USER: umam
13      POSTGRES_PASSWORD: umam
14      POSTGRES_DB: umam
15    # Expone Postgres en localhost:5432 para que el backend se conecte.
16    ports:
17      - "5432:5432"
18    volumes:
19      - umam_pgdata:/var/lib/postgresql/data
20
21 volumes:
22   umam_pgdata:
```

```
1 // @ts-check
2 import eslint from '@eslint/js';
3 import eslintPluginPrettierRecommended from 'eslint-plugin-prettier/recommended';
4 import globals from 'globals';
5 import tseslint from 'typescript-eslint';
6
7 export default tseslint.config(
8   {
9     ignores: ['eslint.config.mjs'],
10   },
11   eslint.configs.recommended,
12   ...tseslint.configs.recommendedTypeChecked,
13   eslintPluginPrettierRecommended,
14   {
15     languageOptions: {
16       globals: {
17         ...globals.node,
18         ...globals.jest,
19       },
20       sourceType: 'commonjs',
21       parserOptions: {
22         projectService: true,
23         tsconfigRootDir: import.meta.dirname,
24       },
25     },
26   },
27   {
28     rules: {
29       '@typescript-eslint/no-explicit-any': 'off',
30       '@typescript-eslint/no-floating-promises': 'warn',
31       '@typescript-eslint/no-unsafe-argument': 'warn'
32     },
33   },
34 );
```

```
1 {  
2   "$schema": "https://json.schemastore.org/nest-cli",  
3   "collection": "@nestjs/schematics",  
4   "sourceRoot": "src",  
5   "compilerOptions": {  
6     "deleteOutDir": true  
7   }  
8 }
```

```

1  {
2    "name": "backend",
3    "version": "0.0.1",
4    "description": "",
5    "author": "",
6    "private": true,
7    "license": "UNLICENSED",
8    "scripts": {
9      "build": "nest build",
10     "format": "prettier --write \"src/**/*.ts\" \"test/**/*.ts\"",
11     "start": "nest start",
12     "start:dev": "nest start --watch",
13     "start:debug": "nest start --debug --watch",
14     "start:prod": "node dist/main",
15     "lint": "eslint \"{src,apps,libs,test}/**/*.ts\" --fix",
16     "test": "jest",
17     "test:watch": "jest --watch",
18     "test:cov": "jest --coverage",
19     "test:debug": "node --inspect-brk -r tsconfig-paths/register -r ts-node/register node_modules/.bin/jest --runInBand",
20     "test:e2e": "jest --config ./test/jest-e2e.json"
21   },
22   "dependencies": {
23     "@nestjs/common": "^11.0.1",
24     "@nestjs/config": "^4.0.4",
25     "@nestjs/core": "^11.0.1",
26     "@nestjs/platform-express": "^11.0.1",
27     "@nestjs/typeorm": "^11.0.3",
28     "class-transformer": "^0.5.1",
29     "class-validator": "^0.15.1",
30     "pg": "^8.23.1",
31     "reflect-metadata": "^0.2.2",
32     "rxjs": "^7.8.1",
33     "typeorm": "^1.1.2"
34   },
35   "devDependencies": {
36     "@eslint/eslintrc": "^3.2.0",
37     "@eslint/js": "^9.18.0",
38     "@nestjs/cli": "^11.0.0",
39     "@nestjs/schematics": "^11.0.0",
40     "@nestjs/testing": "^11.0.1",
41     "@types/express": "^5.0.0",
42     "@types/jest": "^30.0.0",
43     "@types/node": "^22.10.7",
44     "@types/supertest": "^6.0.2",
45     "eslint": "^9.18.0",
46     "eslint-config-prettier": "^10.0.1",
47     "eslint-plugin-prettier": "^5.2.2",
48     "globals": "^16.0.0",
49     "jest": "^30.0.0",
50     "prettier": "^3.4.2",
51     "source-map-support": "^0.5.21",
52     "supertest": "^7.0.0",
53     "ts-jest": "^29.2.5",
54     "ts-loader": "^9.5.2",
55     "ts-node": "^10.9.2",
56     "tsconfig-paths": "^4.2.0",
57     "typescript": "^5.7.3",
58     "typescript-eslint": "^8.20.0"
59   },
60   "jest": {
61     "moduleFileExtensions": [
62       "js",
63       "json",
64       "ts"
65     ],
66     "rootDir": "src",
67     "testRegex": ".*\\.spec\\.ts$",
68     "transform": {
69       "^.+\\.?(t|j)s$": "ts-jest"
70     },
71     "collectCoverageFrom": [
72       "**/*.?(t|j)s"
73     ],
74     "coverageDirectory": "../coverage",

```

```
75     "testEnvironment": "node"
76   }
77 }
```

Desarrollado Ingeniero Mario Isai Moreno Duran
Número 6564121593


```
1 {  
2   "extends": "./tsconfig.json",  
3   "exclude": ["node_modules", "test", "dist", "**/*spec.ts"]  
4 }
```

```
1 {
2   "compilerOptions": {
3     "module": "nodenext",
4     "moduleResolution": "nodenext",
5     "resolvePackageJsonExports": true,
6     "esModuleInterop": true,
7     "isolatedModules": true,
8     "declaration": true,
9     "removeComments": true,
10    "emitDecoratorMetadata": true,
11    "experimentalDecorators": true,
12    "allowSyntheticDefaultImports": true,
13    "target": "ES2023",
14    "sourceMap": true,
15    "outDir": "./dist",
16    "baseUrl": "./",
17    "incremental": true,
18    "skipLibCheck": true,
19    "strictNullChecks": true,
20    "forceConsistentCasingInFileNames": true,
21    "noImplicitAny": true,
22    "strictBindCallApply": true,
23    "noFallthroughCasesInSwitch": true
24  }
25 }
```

```
1  /**
2   * Módulo raíz de la aplicación. En NestJS cada "módulo" agrupa una parte de
3   * la app; este junta la configuración, la conexión a la base de datos y los
4   * módulos de negocio (por ahora solo RegistrosModule).
5   */
6  import { Module } from '@nestjs/common';
7  import { ConfigModule, ConfigService } from '@nestjs/config';
8  import { TypeOrmModule } from '@nestjs/typeorm';
9  import { RegistrosModule } from '../registros/registros.module';
10
11  @Module({
12    imports: [
13      // Lee el archivo .env y deja sus variables disponibles en toda la app
14      // (isGlobal) a través de ConfigService.
15      ConfigModule.forRoot({ isGlobal: true }),
16
17      // Conexión a PostgreSQL con TypeORM (el ORM que traduce entre clases de
18      // TypeScript y tablas SQL). Se usa forRootAsync para esperar a que
19      // ConfigService ya haya leído el .env antes de conectarse.
20      TypeOrmModule.forRootAsync({
21        inject: [ConfigService],
22        useFactory: (config: ConfigService) => ({
23          type: 'postgres',
24          // Datos de conexión: salen del .env; los valores por defecto coinciden
25          // con los del docker-compose.yml para que funcione sin configurar nada.
26          host: config.get('DB_HOST', 'localhost'),
27          port: Number(config.get('DB_PORT', 5432)),
28          username: config.get('DB_USER', 'umam'),
29          password: config.get('DB_PASSWORD', 'umam'),
30          database: config.get('DB_NAME', 'umam'),
31          // Registra solas las entidades declaradas con TypeOrmModule.forFeature()
32          // en cada módulo, sin tener que listarlas aquí una por una.
33          autoLoadEntities: true,
34          // Crea/actualiza las tablas a partir de las entidades. Cómodo en desarrollo;
35          // en producción conviene ponerlo en false y usar migraciones.
36          synchronize: config.get('DB_SYNCHRONIZE', 'true') !== 'false',
37        }),
38      ]),
39
40      // Módulo con todo lo de los registros: entidad, servicio y controlador.
41      RegistrosModule,
42    ],
43  })
44  export class AppModule {}
```

```
1  /**
2   * Punto de entrada del backend: crea la aplicación NestJS, la configura y
3   * la pone a escuchar peticiones HTTP.
4   */
5  import { ValidationPipe } from '@nestjs/common';
6  import { NestFactory } from '@nestjs/core';
7  import { NestExpressApplication } from '@nestjs/platform-express';
8  import { AppModule } from './app.module';
9
10 async function bootstrap() {
11   // Crea la app a partir del módulo raíz (AppModule), que a su vez carga la
12   // conexión a Postgres y el módulo de registros. Se tipa como Express para
13   // poder usar useBodyParser() más abajo.
14   const app = await NestFactory.create<NestExpressApplication>(AppModule);
15
16   // Todas las rutas quedan bajo /api (p. ej. /api/registros), para separarlas
17   // claramente de cualquier otra cosa que se sirva desde el mismo dominio.
18   app.setGlobalPrefix('api');
19
20   // CORS: el frontend Angular corre en otro origen (localhost:4200) y el
21   // navegador bloquearía sus peticiones si el backend no lo autoriza aquí.
22   app.enableCors({ origin: process.env.CORS_ORIGIN ?? 'http://localhost:4200' });
23
24   // Las firmas e imágenes (EKG, Rx, laboratorios) viajan como data URLs en base64
25   // dentro del JSON del registro, así que el límite por defecto (100kb) no alcanza.
26   app.useBodyParser('json', { limit: process.env.BODY_LIMIT ?? '50mb' });
27
28   // Valida automáticamente el cuerpo de cada petición contra su DTO
29   // (ver dto/guardar-registro.dto.ts) y responde 400 si algo no cumple.
30   // whitelist: descarta las propiedades que el DTO no declara, para que no
31   // se cuele basura a la base de datos.
32   app.useGlobalPipes(new ValidationPipe({ whitelist: true }));
33
34   // Puerto configurable por variable de entorno (archivo .env); 3000 por defecto.
35   await app.listen(process.env.PORT ?? 3000);
36 }
37 bootstrap();
```

```
1 import { IsArray, IsDateString, IsObject, IsOptional, IsString, IsUUID } from 'class-validator';
2
3 /** Contenido de una sección: { claveDelCampo: valor }. */
4 type Seccion = Record<string, unknown>;
5
6 /**
7  * DTO (Data Transfer Object): describe qué forma debe tener el JSON que el
8  * frontend manda en PUT /api/registros/:id. El ValidationPipe global (main.ts)
9  * revisa cada petición contra estas reglas antes de que llegue al controlador:
10  * si algo no cumple responde 400, y cualquier propiedad que no esté declarada
11  * aquí se descarta (whitelist).
12  *
13  * Los decoradores son de class-validator:
14  * - @IsOptional: la propiedad puede no venir.
15  * - @IsString / @IsObject / @IsArray / @IsUUID / @IsDateString: si viene,
16  *   debe ser de ese tipo.
17  *
18  * El contenido de cada sección no se valida campo por campo a propósito: los
19  * campos los define el frontend (secciones.data.ts) y cambian seguido.
20  */
21 export class GuardarRegistroDto {
22   // Obligatorio: identifica el registro (lo genera el frontend).
23   @IsUUID()
24   id: string;
25
26   // Obligatorio: fecha en formato ISO 8601, p. ej. "2026-10-08T21:26:39.635Z".
27   @IsDateString()
28   fechaCreacion: string;
29
30   // Datos generales.
31   @IsOptional() @IsString() estado?: string;
32   @IsOptional() @IsString() ciudad?: string;
33   @IsOptional() @IsString() folio?: string;
34
35   // Secciones: cada una debe ser un objeto.
36   @IsOptional() @IsObject() datosServicio?: Seccion;
37   @IsOptional() @IsObject() control?: Seccion;
38   @IsOptional() @IsObject() datosPaciente?: Seccion;
39   @IsOptional() @IsObject() causaTraumatica?: Seccion;
40   @IsOptional() @IsObject() causaClinica?: Seccion;
41   @IsOptional() @IsObject() parto?: Seccion;
42   @IsOptional() @IsObject() evaluacionInicial?: Seccion;
43   @IsOptional() @IsObject() evaluacionSecundaria?: Seccion;
44   @IsOptional() @IsObject() anamnesis?: Seccion;
45   @IsOptional() @IsObject() tratamiento?: Seccion;
46   @IsOptional() @IsObject() traslado?: Seccion;
47   @IsOptional() @IsObject() observaciones?: Seccion;
48   @IsOptional() @IsObject() datosLegales?: Seccion;
49   @IsOptional() @IsObject() hospitalReceptor?: Seccion;
50   @IsOptional() @IsObject() consentimientoInformado?: Seccion;
51
52   // Tablas repetibles: arreglo cuyos elementos (filas) deben ser objetos.
53   @IsOptional() @IsArray() @IsObject({ each: true }) signosVitales?: Seccion[];
54   @IsOptional() @IsArray() @IsObject({ each: true }) manejoFarmacologico?: Seccion[];
55   @IsOptional() @IsArray() @IsObject({ each: true }) vehiculosInvolucrados?: Seccion[];
56 }
```

```

1 import { Column, Entity, Index, PrimaryColumn, UpdateDateColumn } from 'typeorm';
2
3 /** Contenido de una sección: { claveDelCampo: valor }, tal como lo manda el frontend. */
4 type Seccion = Record<string, unknown>;
5
6 /**
7  * Un "Registro de Atención Prehospitalaria". Cada propiedad decorada con
8  * @Column se convierte en una columna de la tabla `registros` en Postgres.
9  *
10 * Los campos de cada sección se definen dinámicamente en el frontend
11 * (secciones.data.ts), así que cada sección se guarda como JSONB en vez de
12 * una columna por campo: agregar o quitar campos en el formulario no
13 * requiere cambiar la base de datos. JSONB además permite consultar dentro
14 * del JSON con SQL si algún día se necesita (p. ej. buscar por nombre de paciente).
15 *
16 * Las propiedades tienen los mismos nombres que la interfaz
17 * RegistroAtencionPrehospitalaria del frontend (registro.model.ts), así el
18 * JSON que va y viene es exactamente el mismo.
19 */
20 @Entity('registros')
21 export class Registro {
22   // Llave primaria. El id (UUID) lo genera el frontend al crear el borrador,
23   // por eso es PrimaryColumn y no PrimaryGeneratedColumn.
24   @PrimaryColumn('uuid')
25   id: string;
26
27   // Cuando se creó el registro (lo manda el frontend). Indexado porque la
28   // lista se ordena por esta fecha.
29   @Index()
30   @Column({ type: 'timestampz' })
31   fechaCreacion: Date;
32
33   // La llena TypeORM sola cada vez que se guarda el registro; sirve para
34   // saber cuándo fue la última modificación.
35   @UpdateDateColumn({ type: 'timestampz' })
36   fechaActualizacion: Date;
37
38   // Datos generales (sección I). Viven como columnas normales en vez de JSONB
39   // porque son los más útiles para buscar/filtrar.
40   @Column({ default: '' })
41   estado: string;
42
43   @Column({ default: '' })
44   ciudad: string;
45
46   @Index()
47   @Column({ default: '' })
48   folio: string;
49
50   // Secciones del formulario (II a XVII), una columna JSONB por sección.
51   // default: {} para que un registro recién creado nunca tenga null aquí.
52   @Column({ type: 'jsonb', default: {} }) datosServicio: Seccion;
53   @Column({ type: 'jsonb', default: {} }) control: Seccion;
54   @Column({ type: 'jsonb', default: {} }) datosPaciente: Seccion;
55   @Column({ type: 'jsonb', default: {} }) causaTraumatica: Seccion;
56   @Column({ type: 'jsonb', default: {} }) causaClinica: Seccion;
57   @Column({ type: 'jsonb', default: {} }) parto: Seccion;
58   @Column({ type: 'jsonb', default: {} }) evaluacionInicial: Seccion;
59   @Column({ type: 'jsonb', default: {} }) evaluacionSecundaria: Seccion;
60   @Column({ type: 'jsonb', default: {} }) anamnesis: Seccion;
61   @Column({ type: 'jsonb', default: {} }) tratamiento: Seccion;
62   @Column({ type: 'jsonb', default: {} }) traslado: Seccion;
63   @Column({ type: 'jsonb', default: {} }) observaciones: Seccion;
64   @Column({ type: 'jsonb', default: {} }) datosLegales: Seccion;
65   @Column({ type: 'jsonb', default: {} }) hospitalReceptor: Seccion;
66   @Column({ type: 'jsonb', default: {} }) consentimientoInformado: Seccion;
67
68   // Tablas repetibles: arreglo JSONB donde una fila = un objeto { claveColumna: valor }.
69   @Column({ type: 'jsonb', default: [] }) signosVitales: Seccion[];
70   @Column({ type: 'jsonb', default: [] }) manejoFarmacologico: Seccion[];
71   @Column({ type: 'jsonb', default: [] }) vehiculosInvolucrados: Seccion[];
72 }

```

```
1 import {
2   BadRequestException,
3   Body,
4   Controller,
5   Delete,
6   Get,
7   HttpStatusCode,
8   Param,
9   ParseUUIDPipe,
10  Put,
11 } from '@nestjs/common';
12 import { GuardarRegistroDto } from '../dto/guardar-registro.dto';
13 import { RegistrosService } from '../registros.service';
14
15 /**
16  * Controlador HTTP: define las rutas de /api/registros y qué método del
17  * servicio atiende cada una. Aquí solo se reciben y validan las peticiones;
18  * el trabajo con la base de datos lo hace RegistrosService.
19  *
20  * ParseUUIDPipe en los :id hace que un id mal formado responda 400 sin
21  * llegar a consultar la base de datos.
22  */
23 @Controller('registros')
24 export class RegistrosController {
25   constructor(private readonly registros: RegistrosService) {}
26
27   /** GET /api/registros → lista todos los registros (la usa el Historial y el formulario al iniciar). */
28   @Get()
29   listar() {
30     return this.registros.listar();
31   }
32
33   /** GET /api/registros/:id → un registro, o 404 si no existe. */
34   @Get('/:id')
35   obtener(@Param('id', ParseUUIDPipe) id: string) {
36     return this.registros.obtener(id);
37   }
38
39   /**
40    * PUT /api/registros/:id → crea o actualiza el registro (autoguardado).
41    * Se usa PUT y no POST porque el id ya viene del frontend: "guarda este
42    * registro en esta dirección". @Body() ya llega validado contra el DTO.
43    */
44   @Put('/:id')
45   guardar(@Param('id', ParseUUIDPipe) id: string, @Body() dto: GuardarRegistroDto) {
46     // Evita guardar por error el contenido de un registro sobre otro.
47     if (dto.id !== id) throw new BadRequestException('El id del cuerpo no coincide con el de la URL');
48     return this.registros.guardar(dto);
49   }
50
51   /** DELETE /api/registros/:id → elimina el registro y responde 204 (sin contenido). */
52   @Delete('/:id')
53   @HttpStatusCode(204)
54   eliminar(@Param('id', ParseUUIDPipe) id: string) {
55     return this.registros.eliminar(id);
56   }
57 }
```

```
1  /**
2   * Módulo de registros: agrupa la entidad (tabla), el servicio (lógica de
3   * acceso a datos) y el controlador (rutas HTTP) de los Registros de
4   * Atención Prehospitalaria. AppModule lo importa para activarlo.
5   */
6  import { Module } from '@nestjs/common';
7  import { TypeOrmModule } from '@nestjs/typeorm';
8  import { Registro } from '../registro.entity';
9  import { RegistrosController } from '../registros.controller';
10 import { RegistrosService } from '../registros.service';
11
12 @Module({
13   // Registra la entidad Registro en este módulo; esto es lo que permite
14   // inyectar su Repository en RegistrosService con @InjectRepository.
15   imports: [TypeOrmModule.forFeature([Registro])],
16   // Clases que reciben las peticiones HTTP.
17   controllers: [RegistrosController],
18   // Clases que NestJS crea e inyecta donde se necesiten (inyección de dependencias).
19   providers: [RegistrosService],
20 })
21 export class RegistrosModule {}
```



```
1 import { Injectable, NotFoundException } from '@nestjs/common';
2 import { InjectRepository } from '@nestjs/typeorm';
3 import { Repository } from 'typeorm';
4 import { GuardarRegistroDto } from '../dto/guardar-registro.dto';
5 import { Registro } from '../registro.entity';
6
7 /**
8  * Lógica de acceso a datos de los registros. El controlador no habla con la
9  * base de datos directamente: llama a estos métodos, que usan el Repository
10 * de TypeORM (un objeto que ya sabe hacer SELECT/INSERT/UPDATE/DELETE sobre
11 * la tabla `registros`).
12 *
13 * Los métodos equivalen 1 a 1 a la interfaz RegistroService del frontend
14 * (listar, obtener, guardar, eliminar).
15 */
16 @Injectable()
17 export class RegistrosService {
18   // NestJS inyecta aquí el Repository de la entidad Registro (registrado en
19   // RegistrosModule con TypeOrmModule.forFeature).
20   constructor(@InjectRepository(Registro) private readonly repo: Repository<Registro>) {}
21
22   /**
23    * Todos los registros, del más antiguo al más reciente. El orden importa:
24    * el formulario toma el último de la lista como "borrador en curso" al abrir la app.
25    */
26   listar(): Promise<Registro[]> {
27     return this.repo.find({ order: { fechaCreacion: 'ASC' } });
28   }
29
30   /** Un registro por id; si no existe lanza NotFoundException (NestJS responde 404). */
31   async obtener(id: string): Promise<Registro> {
32     const registro = await this.repo.findOneBy({ id });
33     if (!registro) throw new NotFoundException(`Registro ${id} no encontrado`);
34     return registro;
35   }
36
37   /**
38    * Crea o actualiza ("upsert"): el frontend autoguarda el mismo id una y otra
39    * vez mientras se llena el formulario. repo.save() busca el id; si no existe
40    * hace INSERT y si existe hace UPDATE. La fecha llega como texto ISO y se
41    * convierte a Date para la columna timestampz.
42    */
43   guardar(dto: GuardarRegistroDto): Promise<Registro> {
44     return this.repo.save({ ...dto, fechaCreacion: new Date(dto.fechaCreacion) });
45   }
46
47   /** Borra el registro. Si el id no existe no hace nada (no es error). */
48   async eliminar(id: string): Promise<void> {
49     await this.repo.delete({ id });
50   }
51 }
```

```

1 import { Test, TestingModule } from '@nestjs/testing';
2 import { ValidationPipe } from '@nestjs/common';
3 import { NestExpressApplication } from '@nestjs/platform-express';
4 import { randomUUID } from 'crypto';
5 import request from 'supertest';
6 import { AppModule } from '../src/app.module';
7
8 // Requiere Postgres corriendo (docker compose up -d).
9 describe('Registros (e2e)', () => {
10   let app: NestExpressApplication;
11   const id = randomUUID();
12
13   beforeAll(async () => {
14     const moduleFixture: TestingModule = await Test.createTestingModule({
15       imports: [AppModule],
16     }).compile();
17
18     app = moduleFixture.createNestApplication<NestExpressApplication>();
19     app.setGlobalPrefix('api');
20     app.useBodyParser('json', { limit: '50mb' });
21     app.useGlobalPipes(new ValidationPipe({ whitelist: true }));
22     await app.init();
23   });
24
25   afterAll(async () => {
26     await request(app.getHttpServer()).delete(`/api/registros/${id}`);
27     await app.close();
28   });
29
30   const registro = () => ({
31     id,
32     fechaCreacion: new Date().toISOString(),
33     estado: 'CDMX',
34     ciudad: 'CDMX',
35     folio: 'F-001',
36     datosPaciente: { nombreOMediaFiliacion: 'Juan Pérez', edadAnios: 40 },
37     hospitalReceptor: { firmaQuienRecibe: 'data:image/png;base64,' + 'A'.repeat(500_000) },
38     signosVitales: [{ hora: '10:00', fc: 80 }],
39   });
40
41   it('crea, actualiza, lista, obtiene y elimina un registro', async () => {
42     const server = app.getHttpServer();
43
44     await request(server).put(`/api/registros/${id}`).send(registro()).expect(200);
45     await request(server)
46       .put(`/api/registros/${id}`)
47       .send({ ...registro(), folio: 'F-002', campoDesconocido: 'x' })
48       .expect(200);
49
50     const lista = await request(server).get('/api/registros').expect(200);
51     expect(lista.body.filter((r: { id: string }) => r.id === id)).toHaveLength(1);
52
53     const uno = await request(server).get(`/api/registros/${id}`).expect(200);
54     expect(uno.body.folio).toBe('F-002');
55     expect(uno.body.campoDesconocido).toBeUndefined();
56     expect(uno.body.datosPaciente.nombreOMediaFiliacion).toBe('Juan Pérez');
57     expect(uno.body.signosVitales).toEqual([{ hora: '10:00', fc: 80 }]);
58     expect(uno.body.control).toEqual({});
59
60     await request(server).delete(`/api/registros/${id}`).expect(204);
61     await request(server).get(`/api/registros/${id}`).expect(404);
62   });
63
64   it('rechaza un cuerpo cuyo id no coincide con la URL', () => {
65     return request(app.getHttpServer())
66       .put(`/api/registros/${randomUUID()}`)
67       .send(registro())
68       .expect(400);
69   });
70
71   it('rechaza datos inválidos', () => {
72     return request(app.getHttpServer())
73       .put(`/api/registros/${id}`)
74       .send({ ...registro(), datosPaciente: 'no-es-objeto' })

```

```
75     .expect(400);  
76   });  
77   });
```

Desarrollado Ingeniero Mario Isai Moreno Duran
Número 6564121593

```
1 {  
2   "moduleFileExtensions": ["js", "json", "ts"],  
3   "rootDir": ".",  
4   "testEnvironment": "node",  
5   "testRegex": ".e2e-spec.ts$",  
6   "transform": {  
7     "^.+\\.\\.?(t|j)s$": "ts-jest"  
8   }  
9 }
```